Appunti di Metodi Numerici

Carbone Davide [1]

A.A. 2026/2027

[1]Lezioni del prof. A. Borio

Indice

1 Aritmetica di Macchina **2**
- 1.1 Rappresentazione dei numeri (reali) . . . 2
- 1.2 Arrotondamento di numeri reali a numeri di macchina . . . 3
- 1.3 Operazioni di macchina ed equivalenze . . . 5
- 1.4 Cancellazione numerica . . . 7
- 1.5 Condizionamento di un problema numerico . . . 9
  - 1.5.1 Cancellazione numerica come problema mal condizionato . . . 10
- 1.6 Stabilità di un algoritmo . . . 10

2 Sistemi Lineari **13**
- 2.1 Norme . . . 13
- 2.2 Forme speciali di matrici . . . 15
- 2.3 Matrici a blocchi . . . 18
- 2.4 Condizionamento di un sistema lineare . . . 19
- 2.5 Metodi diretti . . . 22
- 2.6 Metodi iterativi . . . 28
  - 2.6.1 Metodo di Jacobi e Gauss-Seidel . . . 30
  - 2.6.2 Generalizzazione di Jacobi e Gauss-Seidel . . . 33
  - 2.6.3 Convergenza di metetodi iterativi . . . 34
  - 2.6.4 Rilassamento del metodo di Gauss-Seidel . . . 35
  - 2.6.5 Metodi di discesa . . . 37
- 2.7 Sistemi lineare precondizionati . . . 47
  - 2.7.1 Metodo del gradiente coniugato precondizionato . . . 48

3 Approssimazione di funzioni e di dati **50**
- 3.1 Interpolazione mediante polinomi . . . 50
  - 3.1.1 Esistenza, unicità e costruzione del polinomio interpolante . . . 50
  - 3.1.2 Condizionamento del polinomio di intepolazione . . . 60
  - 3.1.3 Convergenza del polinomio di interpolazione . . . 62
- 3.2 Interpolazione mediante funzioni polinomiali a tratti . . . 69
  - 3.2.1 Spline interpolanti . . . 71

Capitolo 1

Aritmetica di Macchina

Uno dei più grandi errori che si possono commettere quando si programma al calcolatore è quello di pensare che i risultati numerici ottenuti siano privi di errore. Per motivare questa affermazione definiremo nel seguito cosa si intende per un'aritmetica di macchina e le problematiche ad essa collegate.

1.1 Rappresentazione dei numeri (reali)

Definizione 1.1.1 (Rappresentazione floating-point)**.** Si definisce rappresentazione floating-point in base N di in numero reale a la seguente espressione:

$$a = (-1)^s p N^q, \quad s \in \{0, 1\}, \quad p \geq 0 \in \mathbb{R}, \quad q \in \mathbb{Z}$$

Osservazione: Nonostante spesso ragioneremo e faremo esempi in base $N = 10$ le arichitetture dei computer classici usano $N = 2$.

Osservazione: La rappresentazione di un numero floating point non è unico. Per convezione però si usa la rappresentazione seguente.

Definizione 1.1.2 (Rappresentazione floating-point **normalizzata**)**.** Una rappresentazione floating-point si dice **normalizzata** se p soddisfa:

$$N^{-}1 \leq p < 1$$

In questo caso p (mantissa) e q (esponente o caratteristica) sono univocamente determinati.

Una volta fissata una base N del sistema di numerazione, s, p e q quindi definiscono in modo univoco un numero reale a.
Un computer reale può memorizzare solo un numero finito di cifre per la mantissa e per l'esponente. Non tutti i numeri reali sono allora rappresentabili con notazione floating-point.

Definizione 1.1.3 (Numeri di macchina)**.** Si definiscono **numeri di macchina** i numeri con mantissa ed esponente rappresentabili in un calcolatore.
L'insieme dei numeri di macchina in un calcolatore che memorizza t cifre per la mantissa e q

compreso tra $L < 0$ e $U > 0$ risulta così definito:

$$\mathbb{F} = \{0\} \cap \{(-1)^s 0.a_1 a_2 a_3 \dots a_t \cdot N^q, 0 \leq a_i < N, a_1 \neq 0, L \leq q \leq U\}$$

Definizione 1.1.4 (Aritmetica di macchina)**.** Un'**aritmetica di macchina** è l'insieme delle regole e operazioni matematiche tra numeri di macchina rappresentati in una certa notazione floating-point normalizzata.

Osservazione: Il più piccolo numero rappresentabile è $m = 0.1000 \dots 0 \cdot N^L$.

Definizione 1.1.5. Si definisce **regione di underflow** l'insieme dei numeri reali diversi da 0 appartenenti a $(-m, m)$.

Osservazione: Il più grande numero rappresentabile è $M = 0.N-1N-1N-1 \dots N-1 \cdot N^U$.

Definizione 1.1.6. Si definisce **regione di overflow** l'insieme dei numeri reali appartenti a $(-\inf, -M) \cup (M, +\inf)$.

Osservazione: È importante notare che anche se i dati iniziali e il risultato finale sono numeri di macchina o arrotondabili a numeri di macchina le operazioni intermedie potrebbero dare origine a numeri che appartengono alla regione di overflow (valore resituito 0) o di underflow (valore restituito inf).

1.2 Arrotondamento di numeri reali a numeri di macchina

Se un numero reale possiede una mantissa normalizzata con più di t cifre il calcolatore ricorre ad una tecnica di arrotondamento chiamata "rounding to even".

Definizione 1.2.1 (Tecnica "rounding to even")**.** La mantissa p viene approssimata con la mantissa di macchina $\bar{p}$ più vicina e se p è equidistante da due mantisse di macchina consecutive allora p viene approssimata con quella delle due che ha l'ultima cifra pari.
Più formalmente se

$$p = a_1 a_2 \dots a_t a_{t+1} \dots a_n$$

e

$$p \in (\bar{p_1}, \bar{p_2})$$

con

$$\bar{p_1} = 0.a_1 a_2 \dots a_t \quad e \quad \bar{p_2} = \bar{p_1} + N^{-t}$$

due mantisse di macchina consecutive, allora la mantissa di macchina $\bar{p}$ corrispondente a p è data da

$$\bar{p} = \begin{cases} \bar{p_1}, \text{ se } p \in \left(\bar{p_1}, \frac{\bar{p_1}+\bar{p_2}}{2}\right) \text{ oppure } p = \frac{\bar{p_1}+\bar{p_2}}{2} \text{ e } a_t \text{ è pari} \\ \bar{p_2}, \text{ se } p \in \left(\frac{\bar{p_1}+\bar{p_2}}{2}, \bar{p_2}\right) \text{ oppure } p = \frac{\bar{p_1}+\bar{p_2}}{2} \text{ e } a_t \text{ è dispari} \end{cases}$$

Osservazione: Si introduce la tecnica perché statisticamente arrotondare sempre per eccesso porta ad un errore maggiore. Intituivamente infatti l'errore non viene compensato da altri calcoli. Aggiungi in che senso questo errore ha media nulla dopo aver fatto statistica

Nel seguito $\bar{a}$ sarà il numero di macchina associato al numero reale a. È evidente che ogni volta che si memorizza $\bar{a}$ al posto di a si commette un errore.

Definizione 1.2.2 (Errore)**.** Sia $\tilde{x}$ un'approssimazione del numero x. Si definisce **errore assoluto** associato a $\tilde{x}$

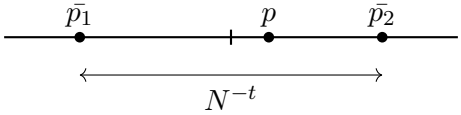
$$e_a = |x - \tilde{x}|$$

e **errore relativo**

$$e_r = \frac{|x - \tilde{x}|}{|x|}, \quad x \neq 0$$

Definizione 1.2.3 (Errore di arrotondamento)**.** Si definisce **errore di arrotondamento** l'errore che si commette quando si sostituisce un numero reale a con un numero di macchina $\bar{a}$.

Determiniamo ora una stima dell'errore assoluto $|a-\bar{a}|$ e dell'errore relativo $\frac{|a-\bar{a}|}{|a|}$ di arrotondamento.
Sia $a = (-1)^s p N^q$ con $N^{-1} \leq p < 1$ e $\bar{a} = (-1)^s \bar{p} N^q$.
Supponiamo che $p \in (\bar{p_1}, \bar{p_2})$, con $\bar{p_1}, \bar{p_2}$ mantisse di macchina consecutive.



Si ha allora necessariamente che

$$|p - \bar{p}| \leq \frac{1}{2} N^{-t}$$

L'errore assoluto risulta quindi

$$|a - \bar{a}| = |(-1)^s p N^q - (-1)^s \bar{p} N^q| = |p - \bar{p}| N^q \leq \frac{1}{2} N^{q-t}$$

e dal momento che $p \geq N^{-1}$

$$\frac{|a - \bar{a}|}{|a|} = \frac{|p - \bar{p}|}{|p|} \leq \frac{1}{2} N^{-t} \frac{1}{|p|} \leq \frac{1}{2} N^{1-t}$$

Osservazione: La distanza tra due numeri di macchina consecutivi è

$$|\bar{a}_1 - \bar{a}_2| = |(-1)^s \bar{p}_1 N^q - (-1)^s \bar{p}_2 N^q| = |\bar{p}_1 - \bar{p}_2| N^q = N^{q-t}$$

Osserviamo quindi che i numeri di macchina non sono equamente distribuiti in quanto si allargano all'aumemtare di q.

Definizione 1.2.4. Si definisce **epsilon di macchina** la quantità

$$eps = N^{1-t}$$

Si definisce **precisione di macchina** la quantità

$$\varepsilon_m = \frac{1}{2}N^{1-t}$$

Osservazione: L'epsilon di macchina è il massimo errore relativo tra due numeri di macchina consecutivi.

La precisione di macchina è una costante specifica di ogni aritmetica floating-point, d'altronde dipende dalla base numerica del calcolatore (N) e dal numero di cifre con cui è rappresentata la mantissa (t).

Definendo $\varepsilon = \frac{(a-\bar{a})}{a}$ otteniamo

$$\bar{a} = a(1+\varepsilon), \quad |\varepsilon| \leq \varepsilon_m$$

Osservazione: Matlab usa un'aritmetica di macchina con $N = 2$, la tecnica di arrotondamento "rounded to even" e 64 bit per la rappresentazione del numero di cui

1. 1 bit per il segno
2. 52 bit per la mantissa p
3. 11 bit per l'esponente q

Inoltre Matlab mette a disposizioni i seguenti valori

1. `realmin` fornisce il più piccolo numero di macchina (modulo)
2. `realmax` fornisce il più grande numero di macchina (modulo)
3. `eps` fornisce il valore dell'epsilon di macchina

Osservazione: Nei calcolatori reali i bit per la mantissa sono 53 dal momento che in base 2 la prima cifra dopo la virgola deve necessariamente essere 0.

Osservazione: Nei computer moderni la precisione di macchina è dell'ordine di 10^{-16}. Da questo fatto equivale a dire che possiamo ritenere affidabile un risultato ottenuto in modo tradizionale solo fino alla 16-esima cifra significativa.

1.3 Operazioni di macchina ed equivalenze

Tralasciando rare eccezioni il risultato di un'operazione tra due numeri di macchina **non** è un numero di macchina.
Un calcolatore infatti non può eseguire esattamente le operazioni aritmetiche esatte ma può solamente effettuare le **operazioni di macchina**.

Definizione 1.3.1. Un'**operazione di macchina** $\odot$ associa a 2 numeri di macchina un terzo numero di macchina, ottenuto arrotondando il risultato della operazione in aritmetica esatta $\cdot$:

$$\bar{a}_1 \odot \bar{a}_2 = \overline{\bar{a}_1 \cdot \bar{a}_2} = (\bar{a}_1 \cdot \bar{a}_2)(1+\varepsilon_\odot), \qquad |\varepsilon_\odot| \leq \varepsilon_m.$$

Osservazione: $|\varepsilon_\odot|$ è l'errore relativo dell'operazione $\odot$ infatti

$$\bar{a}_1 \odot \bar{a}_2 = (\bar{a}_1 \cdot \bar{a}_2) + \varepsilon_\odot(\bar{a}_1 \cdot \bar{a}_2) \Leftrightarrow \varepsilon_\odot = \frac{\bar{a}_1 \odot \bar{a}_2 - \bar{a}_1 \cdot \bar{a}_2}{\bar{a}_1 \cdot \bar{a}_2}$$

Le operazioni $+, -, \times, /$ sono rappresentate dalle corrispettive operazioni di macchina $\oplus, \ominus, \otimes, \oslash$.

Per le operazioni di macchina rimane valida la proprietà commutativa:

$$\bar{a_1} \oplus \bar{a_2} = \bar{a_2} \oplus \bar{a_1}$$

$$\bar{a_1} \otimes \bar{a_2} = \bar{a_2} \otimes \bar{a_1}$$

ma non rimangono valide la proprietà associativa e distributiva:

$$\bar{a_1} \oplus (\bar{a_2} \oplus \bar{a_3}) \neq (\bar{a_1} \oplus \bar{a_2}) \oplus \bar{a_3}$$

$$\bar{a_1} \otimes (\bar{a_2} \otimes \bar{a_3}) \neq (\bar{a_1} \otimes \bar{a_2}) \otimes \bar{a_3}$$

$$\bar{a_1} \otimes (\bar{a_2} \oplus \bar{a_3} \neq (\bar{a_1} \otimes \bar{a_2}) \oplus (\bar{a_1} \otimes \bar{a_3})$$

$$(\bar{a_1} \otimes \bar{a_2}) \oslash \bar{a_3} \neq (\bar{a_1} \oslash \bar{a_3}) \otimes \bar{a_2}$$

Un altro fenomeno di cui bisogna essere a conoscenza è

$$\bar{a_1} \oplus \bar{a_2} = \bar{a_1}, \quad |\bar{a_2}| < |\bar{a_1}|eps$$

quando viene effettuata questa somma infatti le t cifre di mantissa vengono occupate prima con le cifre di $\bar{a_1}$ e poi con alcuni zeri, dal momento che le cifre significative di $\bar{a_2}$ dovrebbero essere collocate dopo la t-esima cifra della nuova mantissa.

$$\overbrace{0.1374623}^{\text{cifre mantissa di } \bar{a_1}} \underbrace{47000000000000000000000000387423847}_{\text{cifre mantissa di } \bar{a_2}}$$
$$\underbrace{\qquad\qquad\qquad\qquad\qquad\qquad\qquad}_{t \text{ cifre}}$$

Esempio 1.3.2. Diamo un esempio pratico del fenomeno appena descritto. Vogliamo effettuare la somma

$$\sum_{i=1}^{10^{16}} 10^{-16} = 1$$

al calcolatore. Un'idea di implementazione in MATLAB potrebbe essere

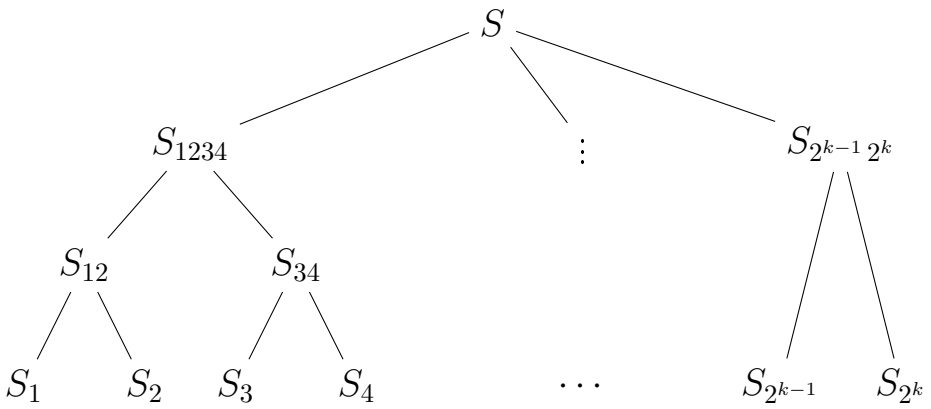
```
s=0
```

```
for i=1:1:10^16
    s=s+10^-16
end
```

Quando $i = \dfrac{10^{16}}{2}$, $S = \dfrac{1}{2}$, allora per $i = \dfrac{10^{16}}{2} + 1$,

$$S = \frac{1}{2} + 10^{-16} = \frac{1}{2}.$$

E così via per ogni $i > \dfrac{10^{16}}{2} + 1$.
Un metodo per effettuare la somma evitando errori è quello di effettuare un algoritmo su un albero binario del tipo



Raggruppando i numeri da sommare in gruppi S_i e sommando in parallelo dal fondo del grafo fino ad ottenere S.

Definizione 1.3.3 (Concetto di equivalenza)**.** Due espressioni $e_1(x)$ ed $e_2(x)$ si definiscono equivalenti nell'aritmetica del calcolatore quando i loro valori in aritmetica di macchina ottenuti tramite le approssimazioni in aritmetica di macchina dei dati $\bar{e_1}(\bar{x}), \bar{e_2}(\bar{x})$ differiscono di una tolleranza relativa dello stesso ordine di grandezza o inferiore della precisione di macchina ε_m, ovvero

$$\frac{|\bar{e_1} - \bar{e_2}|}{|\bar{e_1}|} \lesssim \varepsilon_m$$

1.4 Cancellazione numerica

Definizione 1.4.1 (Cancellazione numerica)**.** Siano $\bar{a_1} = (-1)^{s_1}\bar{p_1}N^{q_1}$, $\bar{a_2} = (-1)^{s_2}\bar{p_2}N^{q_2}$ le rappresentazione di macchina dei due numeri reali $a_1 = (-1)^{s_1}p_1N^{q_1}$, $a_2 = (-1)^{s_2}p_2N^{q_2}$.
Il fenomeno della **cancellazione numerica** consiste nella perdita di cifre della mantissa quando si effettua l'operazione di sottrazioni tra due numeri di macchina e sussistono le seguenti condizioni:

1. $s_1 = s_2$
2. $q_1 = q_2$

3. $\bar{p_1} \approx \bar{p_2}$

4. $\bar{p_1} \neq p_1$ e/o $\bar{p_2} \neq p_2$

Questo fenomeno si presenta perché in un'aritmetica finita, come abbiamo visto in precedenza, le operazioni vengono effettuate su numeri di macchina. Se uno o più numeri di macchina coinvolti presentano una perturbazione rispetto ai dati esatti l'operazione di macchina potrebbe ampliarla. Per capire meglio il fenomeno consideriamo il seguente esempio.

Esempio 1.4.2. Consideriamo i numeri $a_1 = 0.157824831$ e $a_2 = 0.157348212$.
In aritmetica esatta $a_1 - a_2 = 0.000476619 = 0.476619 \cdot 10^{-3}$.
Siano $\bar{a_1} = 0.15782$ e $\bar{a_2} = 0.15735$ le due rappresentazioni di a_1 e a_2 in un'aritmetica finita con $N = 10$ e $t = 5$.
In questa aritmetica $\bar{a_1} \ominus \bar{a_2} = 0.00047 = 0.47000 \cdot 10^{-3}$.
Come possiamo notare abbiamo perso 3 cifre dal risultato in aritmetica esatta.
Calcoliamo gli errori relativi su $\bar{a_1}$, $\bar{a_2}$ e $(\bar{a_1} \ominus \bar{a_2})$:

$$\frac{|a_1 - \bar{a_1}|}{|a_1|} = 0.3 \cdot 10^{-4} < \varepsilon_m$$

$$\frac{|a_2 - \bar{a_2}|}{|a_2|} = 0.1 \cdot 10^{-4} < \varepsilon_m$$

$$\frac{|(a_1 - a_2) - (\bar{a_1} \ominus \bar{a_2})|}{|a_1 - a_2|} = 0.1 \cdot 10^{-1} > \varepsilon_m$$

Nonostante gli errori relativi su $\bar{a_2}$ e $\bar{a_1}$ siano minori della precisione di macchina (e quindi $a_1 \sim \bar{a_1}$ e $a_2 \sim \bar{a_2}$) l'errore relativo della loro differenza calcolata in aritmetica di macchina è maggiore della precisione di macchina e quindi le differenze non sono equivalenti. Notiamo che in questo caso

$$\bar{a_1} \ominus \bar{a_2} = \bar{a_1} - \bar{a_2}$$

e quindi la cancellazione prescinde dal fatto che si usino operazioni di macchina o esatte ma dipende principalmente dal fatto che gli operandi sono perturbati rispetto ai dati esatti. Questo fatto sarà più chiaro nel seguito dopo aver introdotto il condizionamento di un problema. L'operazione di macchina può poi introdurre ulteriore errore ma questo prescinde dal fenomeno della cancellazione numerica.

A volte è possibile manipolare l'espressione algebrica che identifica un problema numerico per evitare la cancellazione numerica.

Esempio 1.4.3. Consideriamo il problema numerico $y = \sqrt{x + \delta} - \sqrt{x}$.
Se $|\delta| << x$ allora il problema di cancellazione numerica può essere eliminato razionalizzando:

$$y = (\sqrt[2]{x + \delta} - \sqrt{x}) \frac{\sqrt{x + \delta} + \sqrt{x}}{\sqrt{x + \delta} + \sqrt{x}} = \frac{\delta}{\sqrt{x + \delta} + \sqrt{x}}$$

Se invece $\delta \approx -x$ il fenomeno della cancellazione numerica non può essere evitato.

Un altro modo per manipolare un'espressione algebrica del tipo $y = f(x)$ per evitare il fenomeno della cancellazione numerica è quello di sostituire ad $f(x)$ il suo sviluppo di Taylor centrato in un valore prossimo a quello di x.

Esempio 1.4.4. Consideriamo il problema numerico:

$$y = \frac{e^x - 1}{x}$$

Per $x \approx 0$ possiamo scrivere il problema come:

$$y = \frac{1}{x}(1 + x + \frac{x^2}{2!} + \ldots + \frac{x^n}{n!} + o(x^n) + 1 = 1 + \frac{x}{2!} + \ldots + \frac{x^{n-1}}{n!} + o(x^{n-1})$$

Ricordando il fenomeno per cui $\bar{a_1} \oplus \bar{a_2} = \bar{a_1}, \quad |\bar{a_2}| < |\bar{a_1}|eps$, possiamo limitarci a sommare i soli termini che danno un contributo reale in aritmetica finita.

Nello studio della propagazione degli errori bisogna distinguere il ruolo del problema dal ruolo dell'algoritmo utilizzato per risolverlo.
Nelle successive due sezioni daremo le definizioni di **condizionamento di un problema numerico** e di **stabilità di un algoritmo** e spiegheremo la loro relazione.

1.5 Condizionamento di un problema numerico

Definizione 1.5.1. Si definisce **problema numerico** una relazione funzionale $y = f(x)$ (**esplicita**) oppure $f(x, y) = 0$ (**implicita**), tra dati x (**input**) e risultati y (**output**).
Qui x e y sono simboli generici: non indicano necessariamente scalari.
I dati x e i risultati y devono essere rappresentabili tramite numeri, vettori o matrici di dimensione finita. Nel caso di vettori o matrici l'errore assoluto o relativo associato a essi viene definito tramite una norma di vettore oppure di matrice.

Definizione 1.5.2 (Condizionamento di un problema numerico)**.** Sia $y = f(x)$ un generico problema numerico e siano:

1. $\bar{x}$ una perturbazione dei dati x di input
2. $\bar{y}$ i risultati ottenuti dai dati $\bar{x}$ in aritmetica esatta

Un problema si dice **ben condizionato** se accade che l'errore relativo associato a $\bar{x}$ è dello stesso ordine o maggiore a quello associato a $\bar{y}$ ovvero se

$$\frac{\|y - \bar{y}\|}{\|y\|} \lesssim \frac{\|x - \bar{x}\|}{\|x\|}$$

In caso contrario si dice **mal condizionato**.

Possiamo studiare il condizionamento di un problema con stime del tipo:

$$\frac{\|y - \bar{y}\|}{\|y\|} \leq K(f, x)\frac{\|x - \bar{x}\|}{\|x\|}$$

Definizione 1.5.3. Si definisce **numero di condizionamento** del problema il più piccolo valore di $K(f,x)$ per il quale la relazione precedente risulta verificata.
Se K è abbastanza piccolo il problema risulta ben condizionato

Quindi in un problema ben condizionato a perturbazioni nei dati corrispondono perturbazioni nei risultati dello stesso ordine di grandezza. In un problema mal condizionato invece a piccolo perturbazioni nei dati corrispondono grandi perturbazioni nei risultati.

1.5.1 Cancellazione numerica come problema mal condizionato

Inquadriamo la cancellazione numerica come problema mal condizionato. In particolare calcoliamo il numero di condizionamento della somma. Dati $a_1, a_2 \in \mathbb{R}$ siano $\bar{a}_1, \bar{a}_2 \in \mathbb{R}$ le loro rappresentazioni di macchina. Poniamo $S = a_1 + a_2$ e $\bar{S} = \bar{a}_1 + \bar{a}_2$. Vogliamo trovare $K(a_1, a_2)$ tale che

$$\frac{|S - \bar{S}|}{|S|} \leq K(a_1, a_2) \cdot \frac{\|\boldsymbol{x} - \bar{\boldsymbol{x}}\|}{\|\boldsymbol{x}\|}$$

per qualche norma p e con $\boldsymbol{x} = (a_1, a_2)$ e $\bar{\boldsymbol{x}} = (\bar{a}_1, \bar{a}_2)$. Partiamo dall'errore relativo

$$|S - \bar{S}| = |a_1 + a_2 - \bar{a}_1 - \bar{a}_2| \leq |a_1 - \bar{a}_1| + |a_2 - \bar{a}_2|$$

Allora

$$\frac{|S - \bar{S}|}{|S|} \leq \frac{|a_1 - \bar{a}_1| + |a_2 - \bar{a}_2|}{|a_1 + a_2|} = \frac{|a_1| + |a_2|}{|a_1 + a_2|} \cdot \frac{|a_1 - \bar{a}_1| + |a_2 - \bar{a}_2|}{|a_1| + |a_2|} = \frac{|a_1| + |a_2|}{|a_1 + a_2|} \cdot \frac{\|\boldsymbol{x} - \bar{\boldsymbol{x}}\|_1}{\|\boldsymbol{x}\|_1}$$

Avendo posto $\boldsymbol{x} = (a_1, a_2)$ e $\bar{\boldsymbol{x}} = (\bar{a}_1, \bar{a}_2)$. Se chiamiamo

$$K(a_1, a_2) = \frac{|a_1| + |a_2|}{|a_1 + a_2|}$$

troviamo che

$$a_1 \to -a_2 \Rightarrow K(a_1, a_2) \to +\infty$$

e questo è esattamente il caso della cancellazione numerica.
Abbiamo quindi dato un fondamento teorico al problema: l'operazione di somma sotto quelle ipotesi è mal condizionata ovvero la perturbazione dei dati di input viene amplificata nell'output. Nell'aritmetica del calcolatore quella perturbazione è data dall'approssimazione di macchina dei dati in input.

1.6 Stabilità di un algoritmo

Come osservavamo nel paragrafo nello scorso paragrafo il condizionamento di un problema dipende dal fatto che i dati siano perturbati e non dal fatto di eseguire operazioni di macchina. Questo ci porta ad osservare che il condizionamento di un problema prescinde dall'algoritmo risolutivo utilizzato per risolverlo. Abbiamo anche detto però che le operazioni di macchina possono introdurre ulteriore errore. Per quantificare questo errore si introduce il concetto di stabilità di un algoritmo di risoluzione.

Definizione 1.6.1. Sia $y = f(x)$ un problema numerico generico. Data un'aritmetica finita siano inoltre:

1. $\bar{x}$ la rappresentazione di macchina dei dati x
2. $\bar{y}$ i risultati ottenuti attraverso un algoritmo a partire da $\bar{x}$ in aritmetica esatta
3. $\tilde{y}$ i risultati ottenuti attraverso un algoritmo a partire da $\bar{x}$ in aritmetica finita

L'algoritmo si dice **numericamente stabile** se accade che l'errore relativo

$$\frac{\|\bar{y} - \tilde{y}\|}{\|\bar{y}\|} \lesssim \varepsilon_m$$

In caso contrario si dice **instabile**.

Osservazione: Un algoritmo è quindi numericamente stabile quando la sequenza delle operazioni non amplifica eccessivamente gli errori di arrotondamento presente nei dati.

Esempio 1.6.2. Consideriamo l'algoritmo per il calcolo di π definito dalla successione

$$x_1 = 2,$$

$$x_n = 2^{n-\frac{1}{2}}\sqrt{1 - \sqrt{1 - 4^{1-n}x_{n-1}^2}}, \qquad n \geq 2.$$

Questo algoritmo è instabile a causa del fenomeno della cancellazione numerica. Infatti, al crescere di n, la quantità

$$z = \sqrt{1 - 4^{1-n}x_{n-1}^2}$$

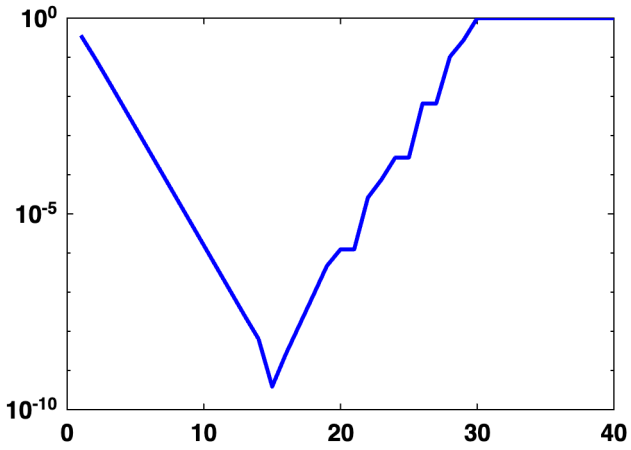
si avvicina sempre di più a 1. Di conseguenza, nell'operazione

$$1 - z$$

si effettua la sottrazione tra due numeri sempre più vicini, con conseguente perdita di cifre significative.
L'errore relativo

$$\frac{|\pi - x_n|}{|\pi|}$$

inizialmente diminuisce, ma successivamente aumenta proprio a causa dell'instabilità numerica dell'algoritmo.



È possibile eliminare la cancellazione numerica razionalizzando l'espressione precedente. In questo modo la stessa successione può essere calcolata mediante

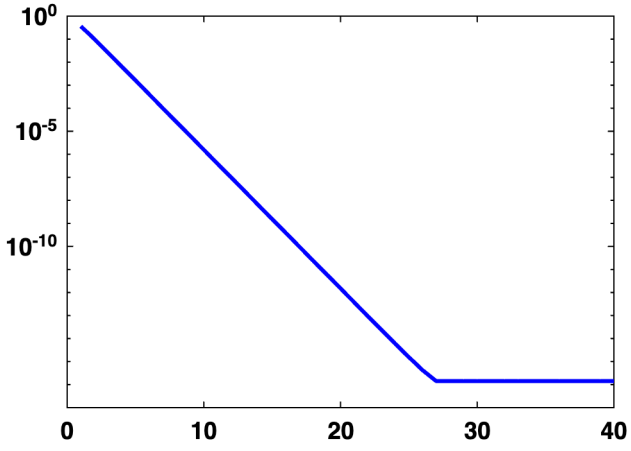
$$x_1 = 2,$$

$$x_n = x_{n-1}\sqrt{\frac{2}{1+\sqrt{1-4^{1-n}x_{n-1}^2}}}, \qquad n \geq 2.$$

In questa forma non compare più la sottrazione tra quantità molto vicine e l'algoritmo risulta stabile.
In questo caso l'errore relativo

$$\frac{|\pi - x_n|}{|\pi|}$$

decresce fino a raggiungere un valore dell'ordine della precisione di macchina, oltre il quale non può diminuire ulteriormente.



Capitolo 2

Sistemi Lineari

Il fine di questo capitolo è quello di presentare al lettore i primi metodi numerici per la risoluzione di sistemi lineari. Prima di addentrarci nella trattazione dei metodi verranno presentate alcune nozioni riguardanti vettori e matrici.

2.1 Norme

Definizione 2.1.1 (Norma di un vettore)**.** Sia $\boldsymbol{v} \in \mathbb{R}^n$ un vettore.

Si definisce norma di un vettore una funzione $f : \mathbb{R}^n \to \mathbb{R}$ che rispetta le seguenti caratteristiche:

1. $f(\boldsymbol{v}) \geq 0 \quad \forall \boldsymbol{v} \in \mathbb{R}^n$ e $f(\boldsymbol{v}) = 0 \Leftrightarrow \boldsymbol{v} = \boldsymbol{0}$
2. $f(\alpha \boldsymbol{v}) = \alpha f(\boldsymbol{v}) \quad \forall \boldsymbol{v} \in \mathbb{R}^n, \quad \forall \alpha \in \mathbb{R}$
3. $f(\boldsymbol{w} + \boldsymbol{v}) \leq f(\boldsymbol{v}) + f(\boldsymbol{w})$

Nel seguito verranno considerate i seguenti tipi di norma di un vettore:

$$||\boldsymbol{x}||_1 = \sum_{i=1}^{n} |x_i|$$

$$||\boldsymbol{x}||_2 = \sqrt{\sum_{i=1}^{n} x_i^2} = \sqrt{\boldsymbol{x}^T \boldsymbol{x}} \quad \text{Norma euclidea}$$

$$||\boldsymbol{x}||_\infty = \max_{1 \leq i \leq n} |x_i|$$

MatLab fornisce funzioni per il calcolo delle norme:

```
norm(x,1); % Fornisce la norma 1 del vettore x
norm(x,2); % Fornisce la norma 2 del vettore x
norm(x); % Fornisce la norma 2 del vettore x
norm(x, inf); % Fornisce la norma infinita del vettore x
```

Osservazione: Su V spazio vettoriale di dimensione finita tutte le norme sono equivalenti ovvero fissate $\|\cdot\|_\star, \|\cdot\|_0 \; \exists C_\star, C^\star > 0$ tali che

$$C_\star \|\boldsymbol{v}\|_0 \leq \|\boldsymbol{v}\|_\star \leq C^\star \|\boldsymbol{v}\|_0 \qquad \forall \boldsymbol{v} \in V$$

Il concetto di norma può essere esteso anche ad un operatore lineare in dimensione finita (e quindi ad una matrice).

Definizione 2.1.2. Sia $L_{\mathbf{A}} : \mathbb{R}^n \to \mathbb{R}^m$, $\boldsymbol{x} \mapsto \mathbf{A}\boldsymbol{x}$, $\mathbf{A} \in \mathbb{R}^{m,n}$ e sia $\|\cdot\|_\star$ una norma di vettore. Si definisce norma operatoriale associata a $\|\cdot\|_\star$ la norma

$$\|L_{\mathbf{A}}\|_\star = \max_{\boldsymbol{x}\in\mathbb{R}^n} \frac{\|\mathbf{A}\boldsymbol{x}\|_\star}{\|\boldsymbol{x}\|_\star} = \max_{\substack{\boldsymbol{x}\in\mathbb{R}^n \\ \|\boldsymbol{x}\|_\star=1}} \|\mathbf{A}\boldsymbol{x}\|_\star$$

Equivalentemente indichiamo con $\|\mathbf{A}\|_\star$ la norma di matrice definita da $\|\mathbf{A}\|_\star = \|L_{\mathbf{A}}\|_\star$.

Definizione 2.1.3. Sia $\mathbf{B} \in \mathbb{R}^{m,m}$ di autovalori $\{\lambda_i\}_{i=1}^m$. Definiamo raggio spettrale di $\mathbf{B}$ la quantità

$$\rho(\mathbf{B}) = \max_{1\le i\le m} |\lambda_i|$$

In particolare siamo interessati alle norme

- $$\|\mathbf{A}\|_1 = \max_{1\le j\le n} \sum_{i=1}^{m} |a_{ij}|$$
- $$\|\mathbf{A}\|_\infty = \max_{1\le i\le m} \sum_{j=1}^{n} |a_{ij}|$$
- $$\|\mathbf{A}\|_2 = \sqrt{\rho(\mathbf{A}^T\mathbf{A})} \qquad \text{(norma spettrale)}$$

Le norme appena definite si ottengono come norme operatoriali associate alle norme vettoriali $p = 1, 2, \infty$.

Osservazione: Notiamo che la norma 1 prende il valore massimo tra le norme dei vettori colonna, mentre la norma infinita prende il valore massimo tra le norme dei vettori righe. Risulta quindi valida la seguente relazione:

$$||\mathbf{A}||_\infty = ||\mathbf{A}^T||_1$$

Segnaliamo l'esistenza della norma 2 di una matrice $||\mathbf{A}||_2$, detta norma spettrale, che verrà trattata in seguito.

```
norm(A,1); % Fornisce la norma 1 della matrice A
norm(A,inf); % Fornisce la norma infinito della matrice A
norm(A,2); % Fornisce la norma spettrale della matrice A
```

Definizione 2.1.4 (**Compatibilità** della norma di matrice con la norma di vettore)**.** Una norma di matrice $||\mathbf{A}||_p$ si dice compatibile con una norma di vettore $||\boldsymbol{x}||_p$ quando sussiste la seguente relazione:

$$||\mathbf{A}\boldsymbol{x}||_p \leq ||\mathbf{A}||_p \cdot ||\boldsymbol{x}||_p$$

In generale tale relazione è verificata per $p = \{1, 2, \infty\}$. Inoltre per ogni norma di matrice compatibile con una norma di vettore vale la seguente relazione

$$\rho(\mathbf{A}) \leq \|\mathbf{A}\|$$

2.2 Forme speciali di matrici

In questo paragrafo elencheremo una serie di forme "speciali" che possono avere le matrici.

Definizione 2.2.1 (**Triangolare superiore**)**.** Una matrice $\mathbf{A} \in \mathbb{R}^{m,n}$ si definisce **triangolare superiore** se $a_{ij} = 0, \quad i > j$

$$\mathbf{A} = \begin{pmatrix} * & * & * \\ 0 & * & * \\ 0 & 0 & * \end{pmatrix}$$

Definizione 2.2.2 (**Triangolare inferiore**)**.** Una matrice $\mathbf{A} \in \mathbb{R}^{m,n}$ si definisce **triangolare inferiore** se $a_{ij} = 0, \quad i < j$

$$\mathbf{A} = \begin{pmatrix} * & 0 & 0 \\ * & * & 0 \\ * & * & * \end{pmatrix}$$

Definizione 2.2.3 (**Diagonale**)**.** Una matrice $\mathbf{A} \in \mathbb{R}^{m,n}$ si definisce **diagonale** se $a_{ij} = 0, \quad i \neq j$

$$\mathbf{A} = \begin{pmatrix} * & 0 & 0 \\ 0 & * & 0 \\ 0 & 0 & * \end{pmatrix}$$

Definizione 2.2.4 (**Tridiagonale**)**.** Una matrice $\mathbf{A} \in \mathbb{R}^{m,n}$ si definisce **tridiagonale** se $a_{ij} = 0, \quad |i - j| > 1$

$$\mathbf{A} = \begin{pmatrix} * & * & 0 \\ * & * & * \\ 0 & * & * \end{pmatrix}$$

Definizione 2.2.5 (Trasposta)**.** Si definisce **trasposta** della matrice $\mathbf{A}$ la matrice $\mathbf{A}^T$ le cui entrate sono definite come $a_{ij}^T = a_{ji}$

Definizione 2.2.6 (Simmetrica)**.** Una matrice $\mathbf{A}$ si dice **simmetrica** se $\mathbf{A} = \mathbf{A}^T$

Definizione 2.2.7 (Inversa)**.** Si definisce **inversa** della matrice $\mathbf{A} \in \mathbb{R}^{n,n}$ la matrice $\mathbf{A}^{-1}$ tale che $\mathbf{A}\mathbf{A}^{-1} = \mathbf{A}^{-1}\mathbf{A} = \mathbf{I}$. Non sempre si può definire l'inversa.

Definizione 2.2.8 (Ortogonale)**.** Una matrice $\mathbf{A} \in \mathbb{R}^{n,n}$ si dice **ortogonale** se $\mathbf{A}\mathbf{A}^T = \mathbf{A}^T\mathbf{A} = \mathbf{I}$. Pertanto l'inversa di una matrice ortogonale è la sua trasposta.

Osservazione: Le colonne di una matrice ortogonale formano una base ortonormale di $\mathbb{R}^n$ rispetto al prodotto scalare standard.

Definizione 2.2.9 (Matrice di permutazione)**.** Si definisce **matrice di permutazione P** una matrice ottenuta permutando le righe della matrice identità. Moltiplicando a destra o a sinistra una matrice o un vettore per una matrice di permutazione si applica la permutazione alla matrice o al vettore. In particolare

- $\mathbf{PA}$ scambia le righe di $\mathbf{A}$
- $\mathbf{AP}$ scambia le colonne di $\mathbf{A}$
- $\mathbf{P}\boldsymbol{v}$ scambia le entrate (righe) di $\boldsymbol{v}$
- $\boldsymbol{v}^T\mathbf{P}$ scambia le entrate (colonne) di $\boldsymbol{v}^T$

Osservazione: Le matrici di permutazioni sono ortogonali.

Esempio 2.2.10. Consideriamo la seguente matrice di permutazione

$$\mathbf{P} = \begin{pmatrix} 0 & 0 & 1 \\ 0 & 1 & 0 \\ 1 & 0 & 0 \end{pmatrix}$$

calcoliamo i prodotti $\mathbf{PA}$ e $\mathbf{P}\boldsymbol{b}$, con $\mathbf{A} \in \mathbb{R}^{3,3}$ e $\boldsymbol{b} \in \mathbb{R}^3$

$$\mathbf{PA} = \begin{pmatrix} 0 & 0 & 1 \\ 0 & 1 & 0 \\ 1 & 0 & 0 \end{pmatrix} \begin{pmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \\ a_{31} & a_{32} & a_{33} \end{pmatrix} = \begin{pmatrix} a_{31} & a_{32} & a_{33} \\ a_{21} & a_{22} & a_{23} \\ a_{11} & a_{12} & a_{13} \end{pmatrix}$$

$$\mathbf{P}\boldsymbol{b} = \begin{pmatrix} 0 & 0 & 1 \\ 0 & 1 & 0 \\ 1 & 0 & 0 \end{pmatrix} \begin{pmatrix} b_1 \\ b_2 \\ b_3 \end{pmatrix} = \begin{pmatrix} b_3 \\ b_2 \\ b_1 \end{pmatrix}$$

Definizione 2.2.11 (Diagonale dominante per righe)**.** Si dice **diagonale dominante per righe** una matrice le cui entrate rispettano la seguente proprietà

$$|a_{ii}| > \sum_{\substack{j=1 \\ j\neq i}}^{n} |a_{ij}| \quad \text{per } i = 1, \dots, n$$

Ovvero se per ogni riga l'elemento sulla diagonale principale è in modulo maggiore della somma dei moduli degli altri elementi della riga

Definizione 2.2.12 (Diagonale dominante per colonne)**.** Si dice **diagonale dominante per colonne** una matrice le cui entrate rispettano la seguente proprietà

$$|a_{jj}| > \sum_{\substack{i=1 \\ i\neq j}}^{n} |a_{ij}| \quad \text{per } j = 1, \dots, n$$

Ovvero se per ogni colonna l'elemento sulla diagonale principale è in modulo maggiore della somma dei moduli degli altri elementi della colonna

Definizione 2.2.13. Una matrice simmetrica $\mathbf{A}$ si dice definita positiva se

$$\boldsymbol{x}^T \mathbf{A} \boldsymbol{x} > 0 \quad \text{per ogni } \boldsymbol{x} \neq \mathbf{0};$$

oppure, equivalentemente, se

$$\lambda_i(\mathbf{A}) > 0 \quad \text{per ogni } i = 1, 2, \dots, n$$

con $\lambda_i(\mathbf{A})$ autovalore di $\mathbf{A}$; oppure, equivalentemente, se

$$\det(\mathbf{A}_k) > 0 \quad \text{per ogni } k = 1, 2, \dots, n$$

ove $\mathbf{A}_k$ è la matrice di ordine k formata dall'intersezione delle prime k righe e k colonne di $\mathbf{A}$.

Teorema 2.2.14. La matrice $\mathbf{A} = \mathbf{B}^T\mathbf{B}$ con $\mathbf{B} \in \mathbb{R}^{n,n}$ avente determinate non nullo è definita positiva.

Dimostrazione. $\mathbf{A}$ è simmetrica per costruzione, infatti

$$\mathbf{A} = \mathbf{B}^T\mathbf{B} \quad \mathbf{A}^T = (\mathbf{B}^T\mathbf{B})^T = \mathbf{B}^T(\mathbf{B}^T)^T = \mathbf{A}$$

Sia $\boldsymbol{x} \neq \mathbf{0}$ e $\boldsymbol{y} = \mathbf{B}\boldsymbol{x}$

$$\boldsymbol{x}^T\mathbf{A}\boldsymbol{x} = \boldsymbol{x}^T(\mathbf{B}^T\mathbf{B})\boldsymbol{x} = \boldsymbol{x}^T\mathbf{B}^T\mathbf{B}\boldsymbol{x} = (\boldsymbol{x}^T\mathbf{B}^T)(\mathbf{B}\boldsymbol{x}) = (\mathbf{B}\boldsymbol{x})^T(\mathbf{B}\boldsymbol{x}) = \boldsymbol{y}^T\boldsymbol{y}$$

$$\boldsymbol{y}^T\boldsymbol{y} = \begin{bmatrix} y_1, \dots, y_n \end{bmatrix} \begin{bmatrix} y_1 \\ \vdots \\ y_n \end{bmatrix} = y_1^2 + y_2^2 + \ldots + y_n^2 = ||\boldsymbol{y}||_2^2$$

$$\Rightarrow \boldsymbol{x}^T\mathbf{A}\boldsymbol{x} = ||\boldsymbol{y}||_2^2 \geq 0 \quad \text{e} \quad ||\boldsymbol{y}||_2^2 = 0 \leftrightarrow \boldsymbol{y} = \mathbf{0}$$

Dal momento che $\boldsymbol{y} \neq \mathbf{0}$ perché, essendo $\boldsymbol{y} = \mathbf{B}\boldsymbol{x}$, $\boldsymbol{x} \neq \mathbf{0}$ e $\mathbf{B}$ di rango massimo, si può concludere che

$$\boldsymbol{x}^T\mathbf{A}\boldsymbol{x} > 0$$

□

2.3 Matrici a blocchi

Definizione 2.3.1 (Matrice a blocchi)**.** Una **matrice a blocchi** è una matrice descritta in termini delle sue sottomatrici anziché in termini dei suoi elementi. Una descrizione di una matrice a blocchi non è unica.

$$\mathbf{A} = \left(\begin{array}{cc|c} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \\ \hline a_{31} & a_{32} & a_{33} \end{array}\right) = \begin{pmatrix} \mathbf{A}_{11} & \mathbf{A}_{12} \\ \mathbf{A}_{21} & \mathbf{A}_{22} \end{pmatrix}$$

La **trasposta** di una **matrice a blocchi** si ottiene nel seguente modo:

$$\mathbf{A}^T = \begin{pmatrix} \mathbf{A}_{11} & \mathbf{A}_{12} & \cdots & \mathbf{A}_{1n} \\ \mathbf{A}_{21} & \mathbf{A}_{22} & \cdots & \mathbf{A}_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ \mathbf{A}_{m1} & \mathbf{A}_{m2} & \cdots & \mathbf{A}_{mn} \end{pmatrix}^T = \begin{pmatrix} \mathbf{A}_{11}^T & \mathbf{A}_{21}^T & \cdots & \mathbf{A}_{m1}^T \\ \mathbf{A}_{12}^T & \mathbf{A}_{22}^T & \cdots & \mathbf{A}_{m2}^T \\ \vdots & \vdots & \ddots & \vdots \\ \mathbf{A}_{1n}^T & \mathbf{A}_{2n}^T & \cdots & \mathbf{A}_{mn}^T \end{pmatrix}$$

Quando il numero dei blocchi e le loro dimensioni sono compatibili, le operazioni tra matrici a blocchi possono essere effettuate trattando i blocchi come scalari.

Esempio 2.3.2. Siano

$$\mathbf{A} = \begin{pmatrix} \mathbf{A}_{11} & \mathbf{A}_{12} \\ \mathbf{A}_{21} & \mathbf{A}_{22} \end{pmatrix}, \quad \mathbf{B} = \begin{pmatrix} \mathbf{B}_{11} & \mathbf{B}_{12} \\ \mathbf{B}_{21} & \mathbf{B}_{22} \end{pmatrix}$$

due matrici con la stessa partizione a blocchi.
Si ha allora

$$\mathbf{A} + \mathbf{B} = \begin{pmatrix} \mathbf{A}_{11} + \mathbf{B}_{11} & \mathbf{A}_{12} + \mathbf{B}_{12} \\ \mathbf{A}_{21} + \mathbf{B}_{21} & \mathbf{A}_{22} + \mathbf{B}_{22} \end{pmatrix}, \quad \alpha\mathbf{A} = \begin{pmatrix} \alpha\mathbf{A}_{11} & \alpha\mathbf{A}_{12} \\ \alpha\mathbf{A}_{21} & \alpha\mathbf{A}_{22} \end{pmatrix}$$

$$\mathbf{AB} = \begin{pmatrix} \mathbf{A}_{11}\mathbf{B}_{11} + \mathbf{A}_{12}\mathbf{B}_{21} & \mathbf{A}_{11}\mathbf{B}_{12} + \mathbf{A}_{12}\mathbf{B}_{22} \\ \mathbf{A}_{21}\mathbf{B}_{11} + \mathbf{A}_{22}\mathbf{B}_{21} & \mathbf{A}_{21}\mathbf{B}_{12} + \mathbf{A}_{22}\mathbf{B}_{22} \end{pmatrix}$$

Quest'ultimo risultato è alla base degli algoritmi di calcolo matriciale dei calcolatori dal momento che riduce il calcolo di un prodotto di matrici a somme di prodotti di matrici più piccoli. È particolarmente utile nel caso di matrici sparse (che presentano larghe aree formate da 0), perchè si individuano dei blocchi formati interamente da zeri che non inficiano nel calcolo

Definizione 2.3.3 (Matrice triangolare a blocchi)**.** Una matrice triangolare superiore (inferiore / superiore) a blocchi è una matrice
quadrata che ha blocchi quadrati sulla diagonale e i cui blocchi sotto

(sopra / sotto) la diagonale principale contengono solo zeri:

$$\mathbf{A} = \begin{pmatrix} \mathbf{A}_{11} & \mathbf{A}_{12} & \cdots & \cdots & \mathbf{A}_{1n} \\ \mathbf{0} & \mathbf{A}_{22} & \cdots & \cdots & \mathbf{A}_{2n} \\ \vdots & \vdots & \ddots & & \vdots \\ \mathbf{0} & \mathbf{0} & \cdots & & \mathbf{A}_{nn} \end{pmatrix}$$

Per le matrici diagonali a blocchi vale il seguente risultato:

$$\det(\mathbf{A}) = \prod_{i=1}^{n} \det(\mathbf{A}_{ii})$$

2.4 Condizionamento di un sistema lineare

Consideriamo il sistema lineare di n equazioni nelle n incognite $x_1, \ldots, x_n$:

$$\begin{cases} a_{11}x_1 + a_{12}x_2 + \cdots + a_{1n}x_n = b_1 \\ a_{21}x_1 + a_{22}x_2 + \cdots + a_{2n}x_n = b_2 \\ \qquad \vdots \qquad \vdots \qquad \ddots \qquad \vdots \\ a_{n1}x_1 + a_{n2}x_2 + \cdots + a_{nn}x_n = b_n \end{cases}$$

Il sistema può essere riscritto in forma matriciale e si ha:

$$\begin{pmatrix} a_{11} & a_{12} & \cdots & a_{1n} \\ a_{21} & a_{22} & \cdots & a_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{n1} & a_{n2} & \cdots & a_{nn} \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{pmatrix} = \begin{pmatrix} b_1 \\ b_2 \\ \vdots \\ b_n \end{pmatrix} \iff \mathbf{A}\boldsymbol{x} = \boldsymbol{b}$$

Assumiamo che il sistema abbia un' unica soluzione, in questo caso la matrice $\mathbf{A}$ si dice **non singolare** ($det(\mathbf{A}) \neq 0$), dal momento che ha rango massimo.
Andiamo a studiare il condizionamento del nostro problema numerico. Forniremo la dimostrazione del caso con $\mathbf{A}$ non perturbata, $\bar{\boldsymbol{b}}$ perturbato e $\bar{\boldsymbol{x}}$ soluzione in aritmetica esatta del sistema $\mathbf{A}\bar{\boldsymbol{x}} = \bar{\boldsymbol{b}}$. Successivamente enunceremo senza dimostrare il caso più generale con $\bar{\mathbf{A}}$ e $\bar{\boldsymbol{b}}$ perturbati e $\bar{\boldsymbol{x}}$ soluzione in aritmetica esatta del sistema $\bar{\mathbf{A}}\bar{\boldsymbol{x}} = \bar{\boldsymbol{b}}$.

Teorema 2.4.1. Dato un problema numerico del tipo $\mathbf{A}\bar{\boldsymbol{x}} = \bar{\boldsymbol{b}}$ vale la seguente relazione

$$\frac{||\boldsymbol{x} - \bar{\boldsymbol{x}}||_p}{||\boldsymbol{x}||_p} \leq K_p(\mathbf{A}) \frac{||\boldsymbol{b} - \bar{\boldsymbol{b}}||_p}{||\boldsymbol{b}||_p}$$

con $K_p(\mathbf{A}) = ||\mathbf{A}||_p ||\mathbf{A}^{-1}||_p$

Dimostrazione. Consideriamo i due problemi numerici

$$\mathbf{A}\bar{\boldsymbol{x}} = \bar{\boldsymbol{b}} \qquad \mathbf{A}\boldsymbol{x} = \boldsymbol{b}$$

Se $\boldsymbol{b} = \mathbf{0}$ allora $\bar{\boldsymbol{b}} = \mathbf{0}$ e l'unica soluzione del sistema lineare è $\boldsymbol{x} = \bar{\boldsymbol{x}} = \mathbf{0}$. Se $\boldsymbol{b} \neq \mathbf{0}$ sottraendo membro a membro otteniamo

$$\mathbf{A}(\bar{\boldsymbol{x}} - \boldsymbol{x}) = \bar{\boldsymbol{b}} - \boldsymbol{b} \Rightarrow \bar{\boldsymbol{x}} - \boldsymbol{x} = \mathbf{A}^{-1}(\bar{\boldsymbol{b}} - \boldsymbol{b}) \Rightarrow$$

$$\Rightarrow ||\bar{\boldsymbol{x}} - \boldsymbol{x}||_p = ||\mathbf{A}^{-1}(\bar{\boldsymbol{b}} - \boldsymbol{b})||_p \leq ||\mathbf{A}^{-1}||_p ||\bar{\boldsymbol{b}} - \boldsymbol{b}||_p$$

$$\mathbf{A}\boldsymbol{x} = \boldsymbol{b} \Rightarrow ||\boldsymbol{b}||_p = ||\mathbf{A}\boldsymbol{x}|| \leq ||\boldsymbol{x}||_p ||\mathbf{A}||_p \Rightarrow ||\boldsymbol{x}||_p \geq \frac{||\boldsymbol{b}||_p}{||\mathbf{A}||_p} \Rightarrow$$

$$\Rightarrow \frac{1}{||\boldsymbol{x}||_p} \leq \frac{||\mathbf{A}||_p}{||\boldsymbol{b}||_p}$$

$$\frac{||\bar{\boldsymbol{x}} - \boldsymbol{x}||}{||\bar{\boldsymbol{x}}||_p} \leq ||\mathbf{A}^{-1}||_p ||\mathbf{A}||_p \frac{||\bar{\boldsymbol{b}} - \boldsymbol{b}||}{||\bar{\boldsymbol{b}}||_p}$$

$$K_p(\mathbf{A}) = ||\mathbf{A}^{-1}||_p ||\mathbf{A}||_p$$

□

Teorema 2.4.2. Dato un problema numerico del tipo $\bar{\mathbf{A}}\bar{\boldsymbol{x}} = \bar{\boldsymbol{b}}$.
Se $||\mathbf{A} - \bar{\mathbf{A}}||_p < \frac{1}{2||\mathbf{A}^{-1}||_p}$ vale la seguente relazione

$$\frac{\|\boldsymbol{x} - \bar{\boldsymbol{x}}\|_p}{\|\boldsymbol{x}\|_p} \leq 2K_p(\mathbf{A}) \left(\frac{\|\mathbf{A} - \bar{\mathbf{A}}\|_p}{\|\mathbf{A}\|_p} + \frac{\|\boldsymbol{b} - \bar{\boldsymbol{b}}\|_p}{\|\boldsymbol{b}\|_p} \right)$$

con

$$K_p(\mathbf{A}) = \|\mathbf{A}\|_p \|\mathbf{A}^{-1}\|_p$$

$K_p(\mathbf{A})$ viene definito **numero di condizionamento** del sistema $\mathbf{A}\boldsymbol{x} = \boldsymbol{b}$.

Osservazione: Per compatibilità si ha che

$$\|\mathbf{AB}\boldsymbol{x}\|_p \leq \|\mathbf{A}\|_p \cdot \|\mathbf{B}\boldsymbol{x}\|_p \leq \|\mathbf{A}\|_p \cdot \|\mathbf{B}\|_p \cdot \|\boldsymbol{x}\|_p \Rightarrow \frac{\|\mathbf{AB}\boldsymbol{x}\|_p}{\|\boldsymbol{x}\|_p} \leq \|\mathbf{A}\|_p \cdot \|\mathbf{B}\|_p$$

Allora

$$\|\mathbf{AB}\|_p = \max_{\boldsymbol{x} \in \mathbb{R}^n} \frac{\|\mathbf{AB}\boldsymbol{x}\|_p}{\|\boldsymbol{x}\|_p} \leq \|\mathbf{A}\|_p \cdot \|\mathbf{B}\|_p$$

Osservazione: Si osservi che

$$K_p(\mathbf{A}) = \|\mathbf{A}\|_p \|\mathbf{A}^{-1}\|_p \geq \|\mathbf{A}\mathbf{A}^{-1}\|_p = \|\mathbf{I}\|_p = \max_{\boldsymbol{x} \in \mathbb{R}^n} \frac{\|\mathbf{I}\boldsymbol{x}\|_p}{\|\boldsymbol{x}\|_p} = \max_{\boldsymbol{x} \in \mathbb{R}^n} \frac{\|\boldsymbol{x}\|_p}{\|\boldsymbol{x}\|_p} = 1$$

e quindi che

1. se $K(\mathbf{A}) \approx 1$, la matrice $\mathbf{A}$ si dice **ben condizionata** $\Rightarrow$ a piccole perturbazioni sui dati corrispondono pertubazioni sui risultati al più dello stesso ordine di grandezza
2. se $K(\mathbf{A}) >> 1$ la matrice $\mathbf{A}$ si dice **mal condizionata** $\Rightarrow$ a piccole perturbazioni sui dati possono corrispondere grandi perturbazioni sui risultati

Esempio 2.4.3.

$$\mathbf{D} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 10^{-10} \end{bmatrix}$$

$K_1(\mathbf{D}) = \|\mathbf{D}\|_1 \|\mathbf{D}^{-1}\|_1 = 10^{10}$.
$\mathbf{D}$ è mal condizionata.

Osservazione: Denotato con $\lambda_i(\mathbf{B})$ il generico autovalore della matrice $\mathbf{B}$, si dimostra che:

$$K_2(\mathbf{A}) = \frac{\sqrt{\max_i |\lambda_i(\mathbf{A}^T\mathbf{A})|}}{\sqrt{\min_i |\lambda_i(\mathbf{A}^T\mathbf{A})|}}$$

Se $\mathbf{A}$ è simmetrica, allora

$$K_2(\mathbf{A}) = \frac{|\max_i \lambda_i(\mathbf{A})|}{|\min_i \lambda_i(\mathbf{A})|}$$

Se $\mathbf{A}$ è simmetrica e definita positiva, allora

$$K_2(\mathbf{A}) = \frac{\max_i \lambda_i(\mathbf{A})}{\min_i \lambda_i(\mathbf{A})}$$

Nel caso simmetrico dire che l'autovalore minimo e massimo hanno moduli di ordine di grandezza diverso vuol dire che $\mathbf{A}$ trasforma lo spazio in modo non uniforme. È proprio questa proprietà che amplifica gli errori di perturbazione degl input.

Due classiche matrici mal condizionate sono la **matrice di Hilbert**

$$\mathbf{H}_n = \begin{pmatrix} 1 & \frac{1}{2} & \cdots & \frac{1}{n} \\ \frac{1}{2} & \frac{1}{3} & \cdots & \frac{1}{n+1} \\ \vdots & \vdots & \ddots & \vdots \\ \frac{1}{n} & \frac{1}{n+1} & \cdots & \frac{1}{2n-1} \end{pmatrix}$$

e la **matrice di Vandermonde**

$$\mathbf{V}_n = \begin{pmatrix} x_1^n & x_1^{n-1} & \cdots & x_1 & 1 \\ x_2^n & x_2^{n-1} & \cdots & x_2 & 1 \\ \vdots & \vdots & \ddots & \vdots & \vdots \\ x_{n+1}^n & x_{n+1}^{n-1} & \cdots & x_{n+1} & 1 \end{pmatrix}$$

.

Il condizionamento di un sistema lineare **non** dipende dalla **norma scelta**. Infatti tutte le norme hanno lo stesso ordine di grandezza.

Seguono una serie di funzioni MatLab utili per il calcolo del condizionamento di un sistema lineare e per generare le matrici di Hilbert e di Vandermonde

```
cond(A,1) % Calcola il numero di condizionamento in norma 1 del sistema Ax = b
cond(A, inf) % Calcola il numero di condizionamento in norma infinito del sistema Ax = b
hilb(n) % genera la matrice di Hilbert di ordine n
vander(x) % Genera la matrice di Vandermonde associata al vettore x con componenti x(1) , ... , x(n+1)
```

2.5 Metodi diretti

In questa e nella prossima sezione presenteremo diversi metodi numerici per la risoluzione di sistemi lineari non singolari. Dividiamo i metodi in due grandi classi: i metodi diretti e quelli iterativi. Dei metodi diretti daremo solo un breve accenno dal momento che questi sono stati già trattati in modo esaustivo nel corso del primo anno. Le principali proprietà dei metodi diretti per la risoluzione del sistema lineare

$$\mathbf{A}\boldsymbol{x} = \boldsymbol{b}$$

sono:

1. $\boldsymbol{x}$ viene determinata mediante un numero finito di passi (al più $n-1$ se n è l'ordine del sistema);
2. $\boldsymbol{x}$ in aritmetica con precisione infinita di calcolo viene determinata in maniera esatta; in aritmetica con precisione finita di calcolo viene determinata con una precisione la cui entità non dipende dalle richieste dell'utente;
3. i metodi diretti modificano la matrice dei coefficienti $\mathbf{A}$ e il termine noto $\boldsymbol{b}$;
4. i metodi diretti sono efficienti per matrici dense e di piccole-medie dimensioni.

Osservazione: A volte modificano la quantità di memoria allocata.

La scelta di un metodo diretto ed efficiente per la risoluzione del sistema lineare

$$\mathbf{A}\boldsymbol{x} = \boldsymbol{b}$$

dipende dalla struttura e dalle proprietà della matrice $\mathbf{A}$.

Matrici diagonali

Se la matrice dei coefficienti $\mathbf{A}$ è **diagonale**, con $a_{kk} \neq 0$, $k = 1, \ldots, n$, la soluzione del sistema lineare

$$\begin{cases} a_{11}x_1 = b_1, \\ a_{22}x_2 = b_2, \\ a_{33}x_3 = b_3, \\ \qquad \ddots \\ a_{nn}x_n = b_n \end{cases}$$

si ricava immediatamente mediante le seguenti n divisioni:

```
for k = 1:n
    x(k) = b(k) / A(k,k);
end
```

Osservazione: L'algortimo per i sistemi diaognali essendo formato da un solo ciclo `for` ha costo computazionale $O(n)$.

Matrici trinagolari

Se la matrice dei coefficienti **A** è **triangolare superiore**, con $a_{kk} \neq 0$, $k = 1, \dots, n$, la soluzione del sistema lineare

$$\begin{cases} a_{11}x_1 + a_{12}x_2 + \cdots + a_{1n}x_n = b_1, \\ \qquad\qquad\qquad\qquad \ddots \\ a_{n-1,n-1}x_{n-1} + a_{n-1,n}x_n = b_{n-1}, \\ \qquad\qquad\qquad\quad a_{nn}x_n = b_n \end{cases}$$

si ottiene ricavando l'incognita x_n dall'ultima equazione, x_{n-1} dalla penultima equazione (dopo aver sostituito in essa il valore di x_n), e continuando verso l'alto fino a ricavare x_1 dalla prima (dopo aver sostituito in essa i valori di $x_2, \dots, x_n$).
Lo schema di calcolo è il seguente:

```
x(n) = b(n) / A(n,n);

for k = n-1:-1:1
    x(k) = (b(k) - sum(A(k,k+1:n) .* x(k+1:n))) / A(k,k);
end
```

Tale procedura è nota con il nome di **metodo di sostituzione all'indietro**. Il costo computazionale in termini di operazioni aritmetiche è essenzialmente $O(n^2/2)$ per n grande.
Se la matrice dei coefficienti **A** è **triangolare inferiore**, con $a_{kk} \neq 0$, $k = 1, \dots, n$, la soluzione del sistema lineare

$$\begin{cases} a_{11}x_1 = b_1, \\ a_{21}x_1 + a_{22}x_2 = b_2, \\ \qquad\qquad\quad \ddots \\ a_{n1}x_1 + a_{n2}x_2 + \cdots + a_{nn}x_n = b_n \end{cases}$$

si ottiene in modo simile al caso triangolare superiore ma partendo dalla prima equazione e arrivando fino all'ultima ovvero attraverso lo schema di calcolo

```
x(1) = b(1) / A(1,1);

for k = 2:n
    x(k) = (b(k) - sum(A(k,1:k-1) .* x(1:k-1))) / A(k,k);
end
```

Tale procedura è nota con il nome di **metodo di sostituzione in avanti**. Anche in questo caso il costo computazionale in termini di operazioni aritmetiche è essenzialmente $O(n^2/2)$ per n grande.

Osservazione: Gli elementi sono in tutto

$$1 + 2 + \cdots + n = \sum_{i=1}^{n} i = \frac{n(n+1)}{2}.$$

Per ogni elemento si fa un numero finito di operazioni quindi ho costo

$$O\left(\alpha\frac{n(n+1)}{2}\right) = O\left(\frac{n(n+1)}{2}\right) = O\left(\frac{n^2}{2}\right).$$

In particolare ad ogni iterazione faccio

- 1 sottrazione
- un numero di moltiplicazioni e somme pari al numero di coefficienti sulla riga k

In tutto $(n-1)$ sottrazioni e

$$\frac{n(n+1)}{2}$$

moltiplicazioni.

Metodo di Gauss

Se la matrice dei coefficienti $\mathbf{A}$ non ha una struttura particolare, il metodo diretto più noto e più utilizzato è senza dubbio il metodo delle eliminazioni di Gauss.
Si ricorda che il metodo delle eliminazioni di Gauss consta essenzialmente di due fasi:

1. trasformazione, in $n-1$ passi, del sistema assegnato $\mathbf{A}\boldsymbol{x} = \boldsymbol{b}$ nel sistema $\mathbf{U}\boldsymbol{x} = \bar{\boldsymbol{b}}$ equivalente a quello assegnato (ovvero che ammette la stessa soluzione $\boldsymbol{x}$), con $\mathbf{U}$ matrice triangolare superiore;
2. risoluzione del sistema $\mathbf{U}\boldsymbol{x} = \bar{\boldsymbol{b}}$ mediante la tecnica di sostituzione all'indietro.

Il costo computazionale del metodo delle eliminazioni di Gauss in termini di operazioni aritmetiche è essenzialmente $O(n^3/3)$ per n grande. Per garantire una **migliore stabilità numerica** dell'algoritmo di Gauss e, quindi, per ridurre l'amplificazione degli errori di arrotondamento che vengono generati nel corso delle trasformazioni, a ogni passo conviene operare un preliminare scambio di equazioni prima di procedere con le trasformazioni. Tale strategia è nota sotto il nome di **pivoting parziale** e consiste al passo k nello scambiare le righe in modo tale da avere a_{kk} massimo evitando così di dividere per un numero vicino allo zero.
Il pivoting è **superfluo** quando:

- $\mathbf{A}$ è a diagonale dominante per colonne;
- $\mathbf{A}$ è simmetrica e definita positiva.

A seguire il codice implementato senza pivoting parziale.

```
for k = 1:n-1 % seleziono colonna
    % Tecnica di pivoting parziale (codice omesso)
    for i = k+1:n % fisso la riga
        m(i,k) = A(i,k) / A(k,k);

        for j = k:n % trasformo la riga
            A(i,j) = A(i,j) - m(i,k) * A(k,j);
        end
    end
end
```

Osservazione: Il comando $\mathbf{A} \setminus \boldsymbol{b}$ in MATLAB calcola la soluzione $\boldsymbol{x}$ del sistema $\mathbf{A}\boldsymbol{x} = \boldsymbol{b}$ con il metodo più efficiente in base al tipo di matrice.

Fattorizzazione PA = LU

Il metodo delle eliminazioni di Gauss con pivoting parziale realizza la seguente fattorizzazione della matrice $\mathbf{A}$:

$$\mathbf{PA} = \mathbf{LU}$$

ove $\mathbf{U}$ è la matrice triangolare superiore ottenuta alla fine del metodo di Gauss, $\mathbf{L}$ è la matrice triangolare inferiore contenente gli $m(i,k)$ per $k = 1 : n-1$ e $i = k+1 : n$ avente diagonale unitaria e $\mathbf{P}$ è una matrice di permutazione, definita dagli scambi richiesti dalla strategia del pivoting.
Il costo computazionale della fattorizzazione $\mathbf{PA} = \mathbf{LU}$ della matrice $\mathbf{A}$ è uguale a quello del metodo delle eliminazioni di Gauss e vale all'incirca $O(n^3/3)$ per n grande. In MATLAB

```
[L,U,P] = lu(A)
```

calcola i fattori $\mathbf{L}$, $\mathbf{U}$ e $\mathbf{P}$ della fattorizzazione $\mathbf{PA} = \mathbf{LU}$ di $\mathbf{A}$.
Si osservi che per la risoluzione di un sistema lineare $\mathbf{A}\boldsymbol{x} = \boldsymbol{b}$ si può procedere utilizzando la fattorizzazione $\mathbf{PA} = \mathbf{LU}$ nel seguente modo:

$$\mathbf{A}\boldsymbol{x} = \boldsymbol{b} \Longrightarrow \mathbf{PA}\boldsymbol{x} = \mathbf{P}\boldsymbol{b} \Longrightarrow \mathbf{L}\underbrace{\mathbf{U}\boldsymbol{x}}_{\boldsymbol{y}} = \mathbf{P}\boldsymbol{b}$$

da cui segue la risoluzione dei due seguenti sistemi triangolari

$$\begin{cases} \mathbf{L}\boldsymbol{y} = \mathbf{P}\boldsymbol{b} & \Longrightarrow \boldsymbol{y} \\ \mathbf{U}\boldsymbol{x} = \boldsymbol{y} & \Longrightarrow \boldsymbol{x} \end{cases}$$

Se i fattori $\mathbf{L}$, $\mathbf{U}$, $\mathbf{P}$ della fattorizzazione $\mathbf{PA} = \mathbf{LU}$ non si devono calcolare perché già noti, il costo computazionale della risoluzione del sistema lineare $\mathbf{A}\boldsymbol{x} = \boldsymbol{b}$ è soltanto $O(n^2)$; altrimenti, il costo è pari a quello della fattorizzazione $\mathbf{PA} = \mathbf{LU}$, cioè $O(n^3/3)$ (essendo, per n grande, il costo della risoluzione dei due sistemi lineari triangolari trascurabile rispetto al costo della fattorizzazione).

Osservazione: Inoltre il metodo di eliminazione di Gauss è utile per calcolare il determinante di una matrice

$$\mathbf{PA} = \mathbf{LU} \Longrightarrow \det(\mathbf{PA}) = \det(\mathbf{LU}) \Longrightarrow \det(\mathbf{P})\det(\mathbf{A}) = \det(\mathbf{L})\det(\mathbf{U}).$$

$$\det(\mathbf{P}) = (-1)^S \qquad S = \text{numero scambi.}$$

$$\det(\mathbf{L}) = 1.$$

$$\det(\mathbf{U}) = \prod_{i=1}^{n} u_{ii}.$$

$$\Longrightarrow \det(\mathbf{A}) = (-1)^S \det(\mathbf{U}).$$

Fattorizzazione di Choleski

Per una matrice $\mathbf{A}$ simmetrica e definita positiva esiste ed è unica la fattorizzazione

$$\mathbf{A} = \mathbf{R}^T\mathbf{R}$$

ove $\mathbf{R}$ è una matrice triangolare superiore con elementi positivi sulla diagonale principale.
Tale fattorizzazione è detta fattorizzazione di Choleski e viene dimostrata nel seguente teorema.

Teorema 2.5.1. Se $\mathbf{A}$ è una matrice simmetrica e definita positiva allora esiste un'unica matrice triangolare inferiore $\mathbf{L}_1 (= \mathbf{R}^T)$ con elementi diagonali positivi tale che

$$\mathbf{A} = \mathbf{L}_1\mathbf{L}_1^T.$$

Dimostrazione. È possibile dimostrare che se i determinanti delle matrici $\mathbf{A}_k$, formate dall'intersezione delle prime k righe e k colonne di una qualsiasi matrice $\mathbf{A}$, sono diversi da zero per $k = 1, \ldots, n-1$, allora esiste ed è unica la fattorizzazione $\mathbf{LU}$ di $\mathbf{A}$.

(A meno di permutazioni)

Poiché per ipotesi $\mathbf{A}$ è simmetrica e definita positiva, e dunque verifica

$$\det(\mathbf{A}_k) > 0$$

per $k = 1, \ldots, n$, allora possiamo senz'altro affermare che esiste un'unica matrice $\mathbf{L}$ triangolare inferiore con diagonale unitaria e un'unica matrice $\mathbf{U}$ triangolare superiore, tali che

$$\mathbf{A} = \mathbf{LU}.$$

Autovalori di $\mathbf{A}_k$ sottoinsieme di valori di $\mathbf{A}$.
Se denotiamo con $\mathbf{D}$ la matrice diagonale i cui elementi principali sono quelli di $\mathbf{U}$ e con $\mathbf{U}_1$ la matrice triangolare superiore con diagonale unitaria tale che $\mathbf{DU}_1 = \mathbf{U}$, allora possiamo scrivere

$$\mathbf{A} = \mathbf{LDU}_1.$$

Poiché $\mathbf{A} = \mathbf{LU}$ è simmetrica ed $\mathbf{L}$ è unica allora $\mathbf{U}_1 = \mathbf{L}^T$; infatti, da

$$\mathbf{A} = \mathbf{LDU}_1$$

segue

$$\mathbf{A} = \mathbf{A}^T = \mathbf{U}_1^T\mathbf{DL}^T,$$

con $\mathbf{U}_1^T$ triangolare inferiore con diagonale unitaria. Per l'unicità di $\mathbf{L}$, si ha allora $\mathbf{L} = \mathbf{U}_1^T$ e $\mathbf{DU}_1 = \mathbf{U} = \mathbf{DL}^T$. Pertanto, risulta univocamente

$$\mathbf{A} = \mathbf{LDL}^T.$$

Inoltre, poiché $\mathbf{A}$ è definita positiva, gli elementi diagonali della matrice $\mathbf{D}$ sono tutti positivi in quanto anche $\mathbf{D}$ è definita positiva; infatti, per $\boldsymbol{x} \neq \mathbf{0}$, si ha

$$0 < \boldsymbol{x}^T\mathbf{A}\boldsymbol{x} = \boldsymbol{x}^T\mathbf{LDL}^T\boldsymbol{x} = (\mathbf{L}^T\boldsymbol{x})^T\mathbf{D}(\mathbf{L}^T\boldsymbol{x}) = \boldsymbol{y}^T\mathbf{D}\boldsymbol{y}$$

con $\boldsymbol{y} = \mathbf{L}^T\boldsymbol{x} \neq \mathbf{0}$, essendo $\boldsymbol{x} \neq \mathbf{0}$ e $\mathbf{L}$ non singolare.

Pertanto, denotando con $\mathbf{D}^{\frac{1}{2}}$ la matrice i cui elementi diagonali sono definiti come la radice quadrata degli elementi diagonali della matrice $\mathbf{D}$, allora

$$\mathbf{D} = \mathbf{D}^{\frac{1}{2}}\mathbf{D}^{\frac{1}{2}}$$

e

$$\mathbf{A} = \mathbf{L}\mathbf{D}\mathbf{L}^T = \mathbf{L}\mathbf{D}^{\frac{1}{2}}\mathbf{D}^{\frac{1}{2}}\mathbf{L}^T = \mathbf{L}_1\mathbf{L}_1^T$$

avendo posto $\mathbf{L}_1 = \mathbf{L}\mathbf{D}^{\frac{1}{2}}$. Per costruzione $\mathbf{L}_1$ è triangolare inferiore con elementi diagonali positivi e, per l'unicità di $\mathbf{L}$ e di $\mathbf{D}$, $\mathbf{L}_1$ è unica. □

Il calcolo del fattore di Choleski $\mathbf{R}$ si ottiene mediante un algoritmo il cui costo computazionale è essenzialmente $O(n^3/6)$ per n grande.
In MATLAB il comando

```
R = chol(A)
```

calcola il fattore $\mathbf{R}$ triangolare superiore della fattorizzazione di Choleski

$$\mathbf{A} = \mathbf{R}^T\mathbf{R},$$

della matrice simmetrica e definita positiva $\mathbf{A}$.
Per la risoluzione del sistema lineare $\mathbf{A}\boldsymbol{x} = \boldsymbol{b}$, con $\mathbf{A}$ simmetrica e definita positiva, nota la fattorizzazione di Choleski $\mathbf{A} = \mathbf{R}^T\mathbf{R}$, un algoritmo efficiente è il seguente:

$$\mathbf{A}\boldsymbol{x} = \boldsymbol{b} \Longrightarrow \mathbf{R}^T\underbrace{\mathbf{R}\boldsymbol{x}}_{\boldsymbol{y}} = \boldsymbol{b} \Longrightarrow \begin{cases} \mathbf{R}^T\boldsymbol{y} = \boldsymbol{b} & \Longrightarrow \boldsymbol{y} \\ \mathbf{R}\boldsymbol{x} = \boldsymbol{y} & \Longrightarrow \boldsymbol{x} \end{cases}$$

Fattorizzazione QR

Un'altra fattorizzazione per una generica matrice $\mathbf{A}$ di dimensioni $m \times n$ è la **fattorizzazione QR**. In particolare, si può dimostrare che esiste una matrice ortogonale $\mathbf{Q}$ di dimensioni $m \times m$ tale che

$$\mathbf{A} = \mathbf{Q}\mathbf{R}$$

dove gli elementi r_{ij} della matrice $\mathbf{R}$ sono nulli per $i > j$. Se $m = n$, $\mathbf{R}$ è una matrice triangolare superiore.
In MATLAB il comando

```
[Q,R] = qr(A)
```

calcola i fattori $\mathbf{Q}$ e $\mathbf{R}$ della fattorizzazione $\mathbf{A} = \mathbf{Q}\mathbf{R}$ di $\mathbf{A}$.
Questa fattorizzazione consente di costruire un algoritmo stabile per la risoluzione del sistema $\mathbf{A}\boldsymbol{x} = \boldsymbol{b}$, con $\mathbf{A}$ di ordine n e non singolare:

$$\mathbf{A}\boldsymbol{x} = \boldsymbol{b} \Longrightarrow \mathbf{Q}\mathbf{R}\boldsymbol{x} = \boldsymbol{b} \Longrightarrow \mathbf{R}\boldsymbol{x} = \mathbf{Q}^T\boldsymbol{b}$$

Tuttavia il calcolo delle matrici $\mathbf{Q}$ ed $\mathbf{R}$ richiede $O(2n^3/3)$ operazioni aritmetiche mentre il metodo di Gauss ne richiede solamente $O(n^3/3)$. Un'altra importante fattorizzazione di una generica matrice $\mathbf{A}$ di dimensioni $m \times n$ è la **decomposizione ai valori singolari (SVD)**. In particolare, si può dimostrare che esistono due matrici ortogonali $\mathbf{U}$ di dimensioni $m \times m$ e $\mathbf{V}$ di dimensioni $n \times n$ tali che

$$\mathbf{A} = \mathbf{U}\mathbf{S}\mathbf{V}^T$$

dove $\mathbf{S}$ è diagonale, ovvero

$$s_{ij} = \begin{cases} \sigma_i & \text{se } i = j, \\ 0 & \text{se } i \neq j \end{cases}$$

con

$$\sigma_1 \geq \sigma_2 \geq \ldots \geq \sigma_p \geq 0, \qquad p = \min\{m, n\}.$$

Gli elementi diagonali della matrice $\mathbf{S}$ sono detti **valori singolari** della matrice $\mathbf{A}$. Essi coincidono con la radice quadrata degli autovalori (tutti non negativi) della matrice $\mathbf{A}^T\mathbf{A}$.
In MATLAB il comando

```
[U,S,V] = svd(A)
```

calcola i fattori $\mathbf{U}$, $\mathbf{S}$, e $\mathbf{V}$ della fattorizzazione $\mathbf{A} = \mathbf{USV}^T$ di $\mathbf{A}$.
Si può dimostrare che il rango della matrice $\mathbf{A}$ coincide con il numero dei valori singolari non nulli.
Anche questa fattorizzazione consente di costruire un algoritmo stabile per la risoluzione del sistema $\mathbf{A}\boldsymbol{x} = \boldsymbol{b}$, con $\mathbf{A}$ di ordine n e non singolare:

$$\mathbf{A}\boldsymbol{x} = \boldsymbol{b} \Longrightarrow \mathbf{USV}^T\boldsymbol{x} = \boldsymbol{b} \Longrightarrow \mathbf{SV}^T\boldsymbol{x} = \mathbf{U}^T\boldsymbol{b} \Longrightarrow \begin{cases} \mathbf{S}\boldsymbol{y} = \mathbf{U}^T\boldsymbol{b} & \Longrightarrow \boldsymbol{y} \\ \boldsymbol{x} = \mathbf{V}\boldsymbol{y} \end{cases}$$

Tuttavia il calcolo delle matrici $\mathbf{U}$, $\mathbf{V}$ ed $\mathbf{S}$ richiede $O(32n^3/3)$ operazioni aritmetiche mentre il metodo di Gauss ne richiede solamente $O(n^3/3)$.

2.6 Metodi iterativi

I metodi diretti non sono generalmente adeguati per sistemi di dimensioni elevate. Tali sistemi, che scaturiscono nella maggior parte dalle applicazioni, generalmente ammettono una matrice dei coefficienti **sparsa**. Una matrice di dimensioni $n \times n$ si ritiene sparsa quando il numero degli elementi diversi dallo zero è di ordine n.
La presenza di sparsità in una matrice rappresenta a priori un vantaggio, dal momento che memorizzando solo gli elementi non nulli, si possono ottenere notevoli risparmi di memoria e di operazioni.
MATLAB per esempio consente di memorizzare le matrici in un formato sparso, secondo cui si prendono in considerazione soltanto gli elementi non nulli.
In MATLAB

- `sparse(A)` consente di memorizzare solo gli elementi non nulli di $\mathbf{A}$;
- `spy(A)` consente di rappresentare graficamente l'insieme degli elementi non nulli di $\mathbf{A}$, ovvero il cosiddetto *pattern*;
- `nnz(A)` fornisce il numero degli elementi non nulli di $\mathbf{A}$;
- `spdiags(A,d,m,n)` crea una matrice sparsa $m \times n$, i cui elementi sulle diagonali, specificate dalle componenti del vettore $\boldsymbol{d}$, coincidono con gli elementi delle colonne di $\mathbf{A}$.

Gli standard di memorizzazione si chiamano CSR (Compressed Sparse Row Format) e COO (Coordinates Format).
Iniziamo dall'illustrare COO. Uso 3 vettori

$$\boldsymbol{r} = [\text{indici di riga dei non-zeri}]$$

$$\boldsymbol{c} = [\text{indici di colonna dei non-zeri}]$$

$$\boldsymbol{v} = [\text{valori dei non zeri}]$$

Esempio 2.6.1.

$$\begin{pmatrix} 1 & 0 & 1 \\ 0 & 0 & 0 \\ 1 & 0 & 0 \end{pmatrix}$$

$$\boldsymbol{r} = [1 \;\; 1 \;\; 3]$$

$$\boldsymbol{c} = [1 \;\; 3 \;\; 1]$$

$$\boldsymbol{v} = [1 \;\; 1 \;\; 1]$$

Il CSR è una variante del COO che comprime il vettore $\boldsymbol{r}$ degli indici di riga.

$$r_i = \text{numero di non zeri precedenti alla riga } i+1$$

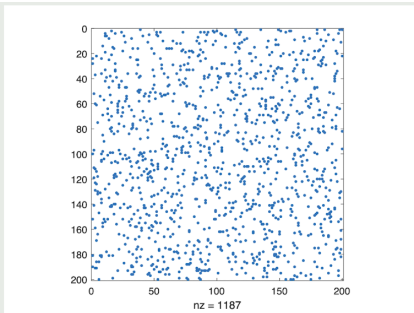
$$\boldsymbol{c} = [\text{indici di colonna}]$$

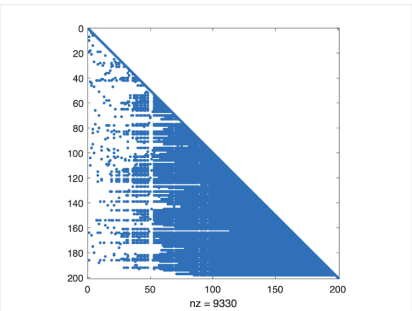
$$\boldsymbol{v} = [\text{valori}]$$

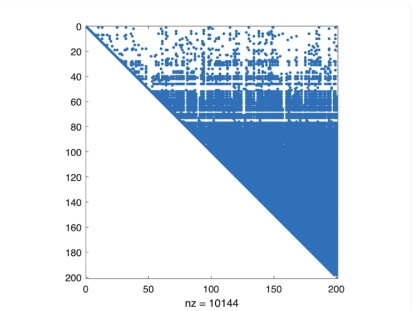
Esempio 2.6.2. Con la stessa matrice dell'esempio precedente $\boldsymbol{r}$ diventa

$$\boldsymbol{r} = [2, 2]$$

Esempio 2.6.3. Sia $\mathbf{A}$ una matrice sparsa di ordine 200×200, con un numero di elementi diversi da zero pari a $nz = 1187$.
Se si calcola la fattorizzazione $\mathbf{PA} = \mathbf{LU}$ della matrice $\mathbf{A}$, le matrici $\mathbf{L}$ e $\mathbf{U}$ non sono sparse a causa del fenomeno di fill-in; per entrambe si ha $nz \approx 10000$.







Per descrivere il generico metodo iterativo per la risoluzione di un sistema lineare, partiamo dal seguente sistema

$$\mathbf{A}\boldsymbol{x} = \boldsymbol{b} \Longleftrightarrow \begin{cases} a_{11}x_1 + a_{12}x_2 + \ldots + a_{1n}x_n = b_1 \\ a_{21}x_1 + a_{22}x_2 + \ldots + a_{2n}x_n = b_2 \\ \vdots \\ a_{n1}x_1 + a_{n2}x_2 + \ldots + a_{nn}x_n = b_n \end{cases}$$

e supponiamo che $a_{ii} \neq 0$ per ogni $i = 1, \ldots, n$. Tale condizione, essendo $\mathbf{A}$ non singolare, si può realizzare mediante opportuni scambi di equazioni.
A partire da un arbitrario vettore iniziale $\boldsymbol{x}^{(0)}$, con un metodo iterativo si determina una sequenza di vettori

$$\left\{\boldsymbol{x}^{(k)}\right\}_{k=1,2,\ldots}$$

che, sotto opportune condizioni, converge alla soluzione esatta $\boldsymbol{x}^*$ del sistema:

$$\boldsymbol{x}^{(0)} \Longrightarrow \left\{\boldsymbol{x}^{(k)}\right\}_{k=1,2,\ldots} : \lim_{k\to\infty} \left(\boldsymbol{x}^* - \boldsymbol{x}^{(k)}\right) = \mathbf{0} \quad \text{(vettore nullo)}$$

2.6.1 Metodo di Jacobi e Gauss-Seidel

Definiamo dapprima due classici metodi iterativi. A tale scopo ricaviamo dall'i-esima equazione per $i = 1, \ldots, n$ l'incognita x_i:

$$\sum_{j=1}^{n} a_{1j}x_j = b_1 \Longrightarrow a_{11}x_1 + \sum_{j=2}^{n} a_{1j}x_j = b_1$$

$$\Longrightarrow x_1 = \frac{1}{a_{11}}\left(b_1 - \sum_{\substack{j=1 \\ j\neq 1}}^{n} a_{1j}x_j\right) \Longrightarrow x_i = \frac{1}{a_{ii}}\left(b_i - \sum_{\substack{j=1 \\ j\neq i}}^{n} a_{ij}x_j\right)$$

$$x_i = \frac{b_i - \sum_{j=1}^{i-1} a_{ij}x_j - \sum_{j=i+1}^{n} a_{ij}x_j}{a_{ii}} = F_i(\boldsymbol{x}), \qquad a_{ii} \neq 0$$

e, a partire dal vettore $\boldsymbol{x}^{(0)}$, generiamo la successione di vettori

$$\left\{\boldsymbol{x}^{(k)}\right\}_{k=1,2,\ldots}$$

la cui i-esima componente, per $i = 1, \ldots, n$, è definita dall'espressione:

$$x_i^{(k)} = \frac{b_i - \sum_{j=1}^{i-1} a_{ij}x_j^{(k-1)} - \sum_{j=i+1}^{n} a_{ij}x_j^{(k-1)}}{a_{ii}} = F_i\left(\boldsymbol{x}^{(k-1)}\right), \qquad k = 1, 2, \ldots$$

Tale formula iterativa definisce il cosiddetto **metodo di Jacobi**.
Ponendo invece

$$x_i^{(k)} = \frac{b_i - \sum_{j=1}^{i-1} a_{ij}x_j^{(k)} - \sum_{j=i+1}^{n} a_{ij}x_j^{(k-1)}}{a_{ii}}, \qquad k = 1, 2, \ldots$$

otteniamo il cosiddetto **metodo di Gauss-Seidel**.

Osservazione: Nella prima sommatoria si prendono gli elementi di ordine (k) perché per $j < i$, ovvero $j = 1 : i - 1$, ho già calcolato $x_j^{(k)}$ e posso far convergere più velocemente il metodo.

Metodo Jacobi
Dato

$$\boldsymbol{x}^{(0)} \in \mathbb{R}^2$$

$$x_1^{(1)} = b_1 - a_{12}x_2^{(0)} \longrightarrow \text{calcolo } x_1^{(1)}$$

$$x_2^{(1)} = b_2 - a_{21}x_1^{(0)} \longrightarrow \text{non uso } x_1^{(1)}$$

Metodo di Gauss-Seidel

$$x_1^{(1)} = b_1 - a_{12}x_2^{(0)} \longrightarrow \text{calcolo } x_1^{(1)}$$

$$x_2^{(1)} = b_2 - a_{21}x_1^{(1)} \longrightarrow \text{uso } x_1^{(1)}$$

Esempio 2.6.4.

$$\begin{cases} 3x_1 - x_2 = 2 \\ x_1 + 2x_2 = 3 \end{cases} \Longrightarrow \begin{cases} x_1 = \dfrac{2 + x_2}{3} \\ x_2 = \dfrac{3 - x_1}{2} \end{cases} \qquad \boldsymbol{x}^* = (1,1)^T$$

METODO DI JACOBI:

$$\begin{cases} x_1^{(k)} = \dfrac{2 + x_2^{(k-1)}}{3} \\ x_2^{(k)} = \dfrac{3 - x_1^{(k-1)}}{2} \end{cases} \qquad \boldsymbol{x}^{(0)} = (0,0)^T$$

$\boldsymbol{x}^{(0)}$	$\boldsymbol{x}^{(1)}$	$\boldsymbol{x}^{(2)}$	$\boldsymbol{x}^{(3)}$	$\boldsymbol{x}^{(4)}$
0	$\frac{2}{3}$	$\frac{7}{6}$	$\frac{19}{18}$	$\dots$
0	$\frac{3}{2}$	$\frac{7}{6}$	$\frac{11}{12}$	$\dots$

$$\frac{\left\|\boldsymbol{x}^* - \boldsymbol{x}^{(3)}\right\|_\infty}{\left\|\boldsymbol{x}^*\right\|_\infty} \approx 0.8 \cdot 10^{-1}$$

METODO DI GAUSS-SEIDEL:

$$\begin{cases} x_1^{(k)} = \dfrac{2 + x_2^{(k-1)}}{3} \\ x_2^{(k)} = \dfrac{3 - x_1^{(k)}}{2} \end{cases} \qquad \boldsymbol{x}^{(0)} = (0,0)^T$$

$\boldsymbol{x}^{(0)}$	$\boldsymbol{x}^{(1)}$	$\boldsymbol{x}^{(2)}$	$\boldsymbol{x}^{(3)}$	$\boldsymbol{x}^{(4)}$
0	$\frac{2}{3}$	$\frac{19}{18}$	$\frac{107}{108}$	$\dots$
0	$\frac{7}{6}$	$\frac{35}{36}$	$\frac{217}{216}$	$\dots$

$$\frac{\left\|\boldsymbol{x}^* - \boldsymbol{x}^{(3)}\right\|_\infty}{\left\|\boldsymbol{x}^*\right\|_\infty} \approx 0.9 \cdot 10^{-2}$$

Quando si implementa un procedimento iterativo occorre definire dei criteri d'arresto.
Un possibile criterio d'arresto consiste senz'altro nel fissare un numero massimo $kmax$ di iterazioni.
Un altro criterio d'arresto riguarda la bontà dell'approssimazione $\boldsymbol{x}^{(k)}$ e consiste nel fissare una tolleranza relativa $tol_r (\geq \varepsilon_m)$ o assoluta tol_a e nell'arrestare il processo all'iterazione $\bar{k}$ se $\boldsymbol{x}^{(\bar{k})}$ soddisfa la seguente disuguaglianza

$$\left\| \boldsymbol{x}^{(\bar{k}+1)} - \boldsymbol{x}^{(\bar{k})} \right\| \leq tol_r \left\| \boldsymbol{x}^{(\bar{k}+1)} \right\|$$

oppure

$$\left\| \boldsymbol{x}^{(\bar{k}+1)} - \boldsymbol{x}^{(\bar{k})} \right\| \leq tol_a$$

Osservazione: Sto verificando idealmente che la successione sia di Cauchy perché $\mathbb{R}^n$ è completo.

Osservazione: Osserviamo che tale criterio d'arresto ha senso quando il metodo converge "rapidamente".

Seguono implememtazioni in MATLAB dei metodi di Jacobi e Gauss-Seidel con iterazioni massime e controllo dell'errore implementati.

```
function [x,k,ier] = Jacobi(A,b,x0,tol,kmax)
if prod(diag(A)) == 0
    x = []; k = []; ier = -1;
    return
end
D = diag(diag(A));
C = A-D;
k = 0; err = Inf;
while k <= kmax && err > tol
    x = D\(b-C*x0);
    err = norm(x-x0)/norm(x);
    k = k + 1;
    x0 = x;
end
if k == kmax+1 && err > tol
    ier = 0;
else
    ier = 1;
end
```

```
function [x,k,ier] = Gauss_Seidel(A,b,x0,tol,kmax)
if prod(diag(A)) == 0
    x = []; k = []; ier = -1;
    return
end
D = tril(A);
C = A-D;
k = 0; err = Inf;
while k <= kmax && err > tol
    x = D\(b-C*x0);
    err = norm(x-x0)/norm(x);
    k = k + 1;
    x0 = x;
```

```
end
if k == kmax+1 && err > tol
    ier = 0;
else
    ier = 1;
end
```

2.6.2 Generalizzazione di Jacobi e Gauss-Seidel

I metodi di Jacobi e di Gauss-Seidel fanno parte di quella classe più generale di procedimenti iterativi che si deducono nel seguente modo:

$$\mathbf{A}\boldsymbol{x} = \boldsymbol{b} \Longleftrightarrow (\mathbf{D}+\mathbf{C})\boldsymbol{x} = \boldsymbol{b} \Longleftrightarrow \mathbf{D}\boldsymbol{x} = -\mathbf{C}\boldsymbol{x} + \boldsymbol{b}$$

$$\Longrightarrow \mathbf{D}\boldsymbol{x}^{(k)} = -\mathbf{C}\boldsymbol{x}^{(k-1)} + \boldsymbol{b}, \qquad k = 1, 2, \dots$$

ove, a ogni iterazione k, $\boldsymbol{x}^{(k)}$ è la soluzione di un sistema lineare con matrice dei coefficienti $\mathbf{D}$ e termine noto $\boldsymbol{b} - \mathbf{C}\boldsymbol{x}^{(k-1)}$.
Pertanto la matrice $\mathbf{D}$ deve essere:

1. non singolare, ovvero tale da garantire l'esistenza e l'unicità della soluzione $\boldsymbol{x}^{(k)}$ a ogni passo k;
2. di forma semplice (triangolare, diagonale), ovvero tale che la soluzione $\boldsymbol{x}^{(k)}$ del sistema possa ottenersi, a ogni passo k, con un algoritmo relativamente semplice e poco costoso;
3. tale da garantire la convergenza, per $k \to \infty$, di $\boldsymbol{x}^{(k)}$ alla soluzione $\boldsymbol{x}^*$ del sistema originario.

Osserviamo che, nel caso del metodo di Jacobi

$$a_{ii}x_i^{(k)} = b_i - \sum_{j=1}^{i-1} a_{ij}x_j^{(k-1)} - \sum_{j=i+1}^{n} a_{ij}x_j^{(k-1)}, \qquad i = 1, \dots, n,$$

le matrici $\mathbf{D}$ e $\mathbf{C}$ dello sdoppiamento $\mathbf{A} = \mathbf{D} + \mathbf{C}$ sono così definite

$$\mathbf{D} = \begin{pmatrix} a_{11} & 0 & \dots & 0 \\ 0 & a_{22} & \dots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \dots & a_{nn} \end{pmatrix}, \qquad \mathbf{C} = \mathbf{A} - \mathbf{D};$$

nel caso del metodo di Gauss-Seidel

$$\sum_{j=1}^{i} a_{ij}x_j^{(k)} = b_i - \sum_{j=i+1}^{n} a_{ij}x_j^{(k-1)}, \qquad i = 1, \dots, n$$

si ha

$$\mathbf{D} = \begin{pmatrix} a_{11} & 0 & \dots & 0 \\ a_{21} & a_{22} & \dots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ a_{n1} & a_{n2} & \dots & a_{nn} \end{pmatrix}, \qquad \mathbf{C} = \mathbf{A} - \mathbf{D}.$$

2.6.3 Convergenza di metetodi iterativi

Per lo studio della convergenza dei metodi iterativi, definiamo il vettore errore

$$\boldsymbol{e}^{(k)} := \boldsymbol{x}^* - \boldsymbol{x}^{(k)}$$

e sottraiamo membro a membro le seguenti due equazioni:

$$\mathbf{D}\boldsymbol{x}^* = \boldsymbol{b} - \mathbf{C}\boldsymbol{x}^*$$
$$\mathbf{D}\boldsymbol{x}^{(k)} = \boldsymbol{b} - \mathbf{C}\boldsymbol{x}^{(k-1)}$$
$$\mathbf{D}\boldsymbol{e}^{(k)} = -\mathbf{C}\boldsymbol{e}^{(k-1)}$$

Abbiamo pertanto

$$\boldsymbol{e}^{(k)} = -\mathbf{D}^{-1}\mathbf{C}\boldsymbol{e}^{(k-1)} = \mathbf{B}\boldsymbol{e}^{(k-1)}$$

ove abbiamo posto

$$\mathbf{B} := -\mathbf{D}^{-1}\mathbf{C} = -\mathbf{D}^{-1}(\mathbf{A} - \mathbf{D}) = \mathbf{I} - \mathbf{D}^{-1}\mathbf{A}$$

La matrice $\mathbf{B}$ è detta **matrice di iterazione**.
Tenendo conto che l'uguaglianza

$$\boldsymbol{e}^{(k)} = \mathbf{B}\boldsymbol{e}^{(k-1)}$$

vale per ogni $k = 1, 2, \ldots$, si ha

$$\boldsymbol{e}^{(k)} = \mathbf{B}\boldsymbol{e}^{(k-1)} = \mathbf{B}^2\boldsymbol{e}^{(k-2)} = \ldots = \mathbf{B}^k\boldsymbol{e}^{(0)}$$

da cui deduciamo che

$$\lim_{k\to\infty} \boldsymbol{e}^{(k)} = \mathbf{0} \quad \text{(vettore nullo)} \quad \forall \boldsymbol{x}^{(0)} \iff \lim_{k\to\infty} \mathbf{B}^k = \mathbf{0} \quad \text{(matrice nulla)}$$

Ricordando che

$$\mathbf{B} = \mathbf{V}^{-1}\mathbf{D}\mathbf{V} \implies \mathbf{B}^2 = \mathbf{V}^{-1}\mathbf{D}\mathbf{V}\mathbf{V}^{-1}\mathbf{D}\mathbf{V} = \mathbf{V}^{-1}\mathbf{D}^2\mathbf{V} \implies \mathbf{B}^k = \mathbf{V}^{-1}\mathbf{D}^k\mathbf{V}$$

ovvero che gli autovalori di $\mathbf{B}^k$ sono λ_i^k con λ_i autovalori di $\mathbf{B}$ e notando che l'unica matrice con autovalori tutti nulli è la matrice nulla si può dimostrare che

$$\lim_{k\to\infty} \mathbf{B}^k = \mathbf{0} \iff \rho(\mathbf{B}) < 1$$

ove $\rho(\mathbf{B})$ è il raggio spettrale della matrice, cioè

$$\rho(\mathbf{B}) = \max_{1\le i\le n} |\lambda_i|$$

con λ_i autovalore di $\mathbf{B}$, allora possiamo dedurre

$$\lim_{k\to\infty} \boldsymbol{e}^{(k)} = \mathbf{0} \quad \forall \boldsymbol{x}^{(0)} \iff \rho(\mathbf{B}) < 1$$

Ricordando che per le norme $1, 2, \infty$ sussiste la proprietà

$$\rho(\mathbf{A}) \le \|\mathbf{A}\|,$$

possiamo senz'altro affermare che, se per almeno una delle suddette norme vale

$$\|\mathbf{B}\| < 1,$$

allora

$$\lim_{k\to\infty} \boldsymbol{e}^{(k)} = \mathbf{0} \qquad \forall \boldsymbol{x}^{(0)}.$$

Convergenza dei metodi di Jacobi e Gauss-Seidel

Teorema 2.6.5. Se $\mathbf{A}$ è a diagonale dominante per righe o per colonne, allora i metodi di Jacobi e di Gauss-Seidel, applicati al sistema lineare $\mathbf{A}\boldsymbol{x} = \boldsymbol{b}$, convergono qualunque sia il vettore iniziale.

Teorema 2.6.6. Se $\mathbf{A}$ è simmetrica e definita positiva, allora il metodo di Gauss-Seidel converge.

Osservazione:

1. La convergenza del metodo di Gauss-Seidel non implica la convergenza del metodo di Jacobi e viceversa. Se entrambi convergono, in generale il metodo di Gauss-Seidel converge più rapidamente.

2. La rapidità di convergenza del metodo iterativo

$$\mathbf{D}\boldsymbol{x}^{(k)} = -\mathbf{C}\boldsymbol{x}^{(k-1)} + \boldsymbol{b}, \qquad k = 1, 2, \ldots$$

dipende dal raggio spettrale $\rho(\mathbf{B})$ della matrice di iterazione

$$\mathbf{B} = -\mathbf{D}^{-1}\mathbf{C} = \mathbf{I} - \mathbf{D}^{-1}\mathbf{A} :$$

quanto più esso è piccolo, tanto più è rapida la convergenza del corrispondente metodo.

Teorema 2.6.7. Se $\mathbf{A}$ è tridiagonale, non singolare con elementi diagonali non nulli, allora i metodi di Jacobi e di Gauss-Seidel sono entrambi convergenti o entrambi divergenti. Nel caso di convergenza, il metodo di Gauss-Seidel converge più velocemente di quello di Jacobi e, in particolare, si ha

$$\rho(\mathbf{B}_{GS}) = \rho(\mathbf{B}_J)^2.$$

2.6.4 Rilassamento del metodo di Gauss-Seidel

Definiamo ora un'altra classe di metodi iterativi detti metodi di rilassamento. A partire dalla formula che definisce il metodo di Gauss-Seidel

$$x_i^{(k)} = \frac{b_i - \sum_{j=1}^{i-1} a_{ij}x_j^{(k)} - \sum_{j=i+1}^{n} a_{ij}x_j^{(k-1)}}{a_{ii}}, \qquad k = 1, 2, \ldots$$

sottraiamo $x_i^{(k-1)}$, per $i = 1, 2, \ldots, n$, da ambo i membri.

$$\begin{aligned} x_i^{(k)} - x_i^{(k-1)} &= \frac{1}{a_{ii}}\left(b_i - \sum_{j=1}^{i-1} a_{ij}x_j^{(k)} - \sum_{j=i+1}^{n} a_{ij}x_j^{(k-1)}\right) - x_i^{(k-1)} \\ &= \frac{1}{a_{ii}}\left(b_i - \sum_{j=1}^{i-1} a_{ij}x_j^{(k)} - \sum_{j=i+1}^{n} a_{ij}x_j^{(k-1)} - a_{ii}x_i^{(k-1)}\right) \\ &= \frac{1}{a_{ii}}\left(b_i - \sum_{j=1}^{i-1} a_{ij}x_j^{(k)} - \sum_{j=i}^{n} a_{ij}x_j^{(k-1)}\right) \end{aligned}$$

ottenendo quindi

$$r_i^{(k-1)} = x_i^{(k)} - x_i^{(k-1)} = \frac{b_i - \sum_{j=1}^{i-1} a_{ij}x_j^{(k)} - \sum_{j=i}^{n} a_{ij}x_j^{(k-1)}}{a_{ii}}, \qquad k = 1, 2, \dots$$

dove $r_i^{(k-1)}$ rappresenta la correzione da apportare a $x_i^{(k-1)}$ per ottenere $x_i^{(k)}$:

$$x_i^{(k)} = x_i^{(k-1)} + r_i^{(k-1)}, \qquad k = 1, 2, \dots, \quad i = 1, \dots, n$$

Quest'ultima relazione definisce ancora il metodo di Gauss-Seidel. Essa tuttavia suggerisce l'introduzione di un parametro $\omega \neq 0$ al fine di migliorare la correzione su $x_i^{(k-1)}$:

$$x_i^{(k)} = x_i^{(k-1)} + \omega r_i^{(k-1)}, \qquad k = 1, 2, \dots, \quad i = 1, \dots, n$$

Tale relazione definisce il **metodo di rilassamento**: il metodo è detto di **sottorilassamento** se $\omega < 1$, mentre è detto di **sovrarilassamento** se $\omega > 1$ (se $\omega = 1$ si ha il metodo di Gauss-Seidel). Quest'ultimo è detto metodo **SOR**, dalle iniziali di Successive Over Relaxation.
Per riscrivere il metodo in forma matriciale, introduciamo le seguenti matrici che, per comodità, definiamo con i seguenti comandi MATLAB:

```
D=diag(diag(A)), L=tril(A,-1), U=triu(A,1).
```

Si ha pertanto $\mathbf{A} = \mathbf{L} + \mathbf{D} + \mathbf{U}$ e riscrivendo in forma matriciale le iterazioni di Gauss-Seidel

$$(\mathbf{D} + \mathbf{L})\mathbf{x}^{(k)} = \mathbf{b} - \mathbf{U}\mathbf{x}^{(k-1)}$$
$$\mathbf{D}\mathbf{x}^{(k)} = \mathbf{b} - \mathbf{L}\mathbf{x}^{(k)} - \mathbf{U}\mathbf{x}^{(k-1)}$$

Sottraendo $\mathbf{D}\mathbf{x}^{(k-1)}$ da ambo i membri, si ha

$$\underbrace{\mathbf{D}\mathbf{x}^{(k)} - \mathbf{D}\mathbf{x}^{(k-1)}}_{\mathbf{D}\mathbf{r}^{(k-1)}} = \mathbf{b} - \mathbf{L}\mathbf{x}^{(k)} - (\mathbf{D} + \mathbf{U})\mathbf{x}^{(k-1)}$$

ovvero

$$\mathbf{r}^{(k-1)} = \mathbf{D}^{-1}\left[\mathbf{b} - \mathbf{L}\mathbf{x}^{(k)} - (\mathbf{D} + \mathbf{U})\mathbf{x}^{(k-1)}\right]$$

da cui rilassando si ottiene

$$\begin{aligned}\mathbf{x}^{(k)} &= \mathbf{x}^{(k-1)} + \omega\mathbf{r}^{(k-1)} \\ &= \mathbf{x}^{(k-1)} + \omega\mathbf{D}^{-1}\left[\mathbf{b} - \mathbf{L}\mathbf{x}^{(k)} - (\mathbf{D} + \mathbf{U})\mathbf{x}^{(k-1)}\right]\end{aligned}$$

In definitiva, abbiamo

$$(\mathbf{D} + \omega\mathbf{L})\mathbf{x}^{(k)} = \omega\mathbf{b} + \left[(1-\omega)\mathbf{D} - \omega\mathbf{U}\right]\mathbf{x}^{(k-1)}$$

Ricordando il risultato di convergenza dei metodi iterativi, abbiamo che la matrice di iterazione è data da:

$$\mathbf{B} = (\mathbf{D} + \omega\mathbf{L})^{-1}\left[(1-\omega)\mathbf{D} - \omega\mathbf{U}\right]$$

e il metodo di rilassamento risulterà convergente se e solo se $\rho(\mathbf{B}) < 1$. Notiamo che in Gauss-Seidel la matrice di iterazione trovata era

$$\mathbf{B} = -(\mathbf{D} + \mathbf{L})^{-1}$$

che corrisponde di nuovo al caso $\omega = 1$.
La scelta del parametro ω deve rendere $\rho(\mathbf{B})$ quanto più piccolo è possibile. In alcuni casi è possibile determinare il valore di ω_{opt} per il quale $\rho(\mathbf{B})$ assume valore minimo.
Si dimostra che $\rho(\mathbf{B}) \geq 1$ quando $\omega \leq 0$ oppure $\omega \geq 2$ e, pertanto, per questi valori di ω, il metodo non può convergere.
La **convergenza** è invece **assicurata** per tutti i valori di $0 < \omega < 2$ quando $\mathbf{A}$ è simmetrica e definita positiva.
Segue un'implmentazione in MATLAB del SOR.

```
function [x,k,ier] = SOR(A,b,x0,omega,tol,kmax)
if prod(diag(A)) == 0
    x = []; k = []; ier = -1;
    return
end
D = diag(diag(A));
L = tril(A,-1);
U = triu(A,1);
k = 0;  err = Inf;
while k <= kmax && err > tol
    x = (D+omega*L)\(omega*b+((1-omega)*D-omega*U)*x0);
    err = norm(x-x0)/norm(x);
    k = k + 1;
    x0 = x;
end
if k == kmax+1 && err > tol
    ier = 0;
else
    ier = 1;
end
```

2.6.5 Metodi di discesa

Se la matrice dei coefficienti $\mathbf{A} \in \mathbb{R}^{n,n}$ del sistema lineare $\mathbf{Ax} = \mathbf{b}$ è **simmetrica e definita positiva**, allora il sistema lineare può essere risolto con un **metodo di discesa**.
Per definire un metodo di discesa, associamo al sistema lineare $\mathbf{Ax} = \mathbf{b}$ il seguente funzionale (quadratico)

$$\phi : \mathbb{R}^n \to \mathbb{R}, \quad \phi(\mathbf{x}) = \frac{1}{2}\mathbf{x}^T\mathbf{Ax} - \mathbf{x}^T\mathbf{b}$$

definito come **energia del sistema**.
I metodi di discesa sono metodi iterativi che si basano sulla seguente proprietà.

Teorema 2.6.8. Sia $\mathbf{A}$ simmetrica e definita positiva. Allora

$$\mathbf{x}^* \text{ è soluzione del sistema } \mathbf{Ax} = \mathbf{b} \iff \mathbf{x}^* \text{ è il punto di minimo del funzionale } \phi(\mathbf{x})$$

Dimostrazione. Il minimo di $\phi(\mathbf{x})$ deve necessariamente annullare il suo gradiente.

$$\boldsymbol{\nabla}\phi(\mathbf{x}) = \left(\frac{\partial\phi}{\partial x_1}, \dots, \frac{\partial\phi}{\partial x_n}\right)^T.$$

Tenendo conto che $\mathbf{A}^T = \mathbf{A}$ e che

$$\frac{\partial \phi}{\partial x_k} = \frac{\partial}{\partial x_k}\left(\frac{1}{2}\sum_{i=1}^{n} x_i \left(\sum_{j=1}^{n} a_{ij}x_j\right) - \sum_{i=1}^{n} x_i b_i\right)$$

$$= \frac{1}{2}\left(\sum_{j=1}^{n} a_{kj}x_j + \sum_{i=1}^{n} a_{ik}x_i\right) - b_k$$

$$= \sum_{j=1}^{n} a_{kj}x_j - b_k = (\mathbf{Ax})_k - (\mathbf{b})_k,$$

risulta

$$\boldsymbol{\nabla}\phi(\mathbf{x}) = \mathbf{Ax} - \mathbf{b}$$

Pertanto, se $\mathbf{x}^*$ è un punto di minimo per $\phi(\mathbf{x})$, allora $\boldsymbol{\nabla}\phi(\mathbf{x}^*) = \mathbf{o}$ e, quindi, $\mathbf{Ax}^* - \mathbf{b} = \mathbf{o}$, cioè $\mathbf{x}^*$ è soluzione del sistema.
Viceversa, poiché $\mathbf{A}$ è non singolare, essendo simmetrica e definita positiva, denotiamo con $\mathbf{x}^*$ l'unica soluzione del sistema $\mathbf{Ax} = \mathbf{b}$.
Abbiamo allora che $\boldsymbol{\nabla}\phi(\mathbf{x}^*) = \mathbf{o}$ e $\mathbf{x}^*$ è l'unico punto di estremo relativo per $\phi(\mathbf{x})$. Per verificare che esso è un punto di minimo, calcoliamo la matrice Hessiana del funzionale $\phi(\mathbf{x})$,

$$\mathbf{H}\phi(\mathbf{x}) = \begin{pmatrix} \phi_{x_1x_1} & \phi_{x_1x_2} & \cdots & \phi_{x_1x_n} \\ \phi_{x_2x_1} & \phi_{x_2x_2} & \cdots & \phi_{x_2x_n} \\ \vdots & \vdots & \ddots & \vdots \\ \phi_{x_nx_1} & \phi_{x_nx_2} & \cdots & \phi_{x_nx_n} \end{pmatrix}$$

con

$$\phi_{x_hx_k} := \frac{\partial^2 \phi}{\partial x_h \partial x_k}.$$

Poiché

$$(\mathbf{H})_{hk} = \phi_{x_hx_k} = \frac{\partial}{\partial x_h}\left(\sum_{j=1}^{n} a_{kj}x_j - b_k\right) = a_{kh} = a_{hk} = (\mathbf{A})_{hk},$$

risulta

$$\mathbf{H}\phi(\mathbf{x}) = \mathbf{A}.$$

Tenendo conto che $\mathbf{A}$ è simmetrica e definita positiva, $\mathbf{x}^*$ è dunque un punto di minimo per ϕ. □

Per lo studio del metodo è utile introdurre il concetto di vettore residuo.

Definizione 2.6.9. Si definisce **residuo** dell'equazione $\mathbf{Ax} = \mathbf{b}$ relativo a un vettore $\mathbf{x}$, il vettore

$$\mathbf{r}(\mathbf{x}) = \mathbf{b} - \mathbf{Ax}$$

Osservazione: Dalla precedente dimostrazione si deduce che:

$$\boldsymbol{\nabla}\phi(\mathbf{x}) = \mathbf{Ax} - \mathbf{b} = -\mathbf{r}(\mathbf{x})$$

Esempio 2.6.10. Per $n = 1$ il funzionale $\phi(\mathbf{x})$ rappresenta una parabola con la concavità rivolta verso l'alto.

$$\phi(x) = \frac{1}{2}ax^2 - b, \qquad a > 0$$

Per $n = 2$, $\phi(\mathbf{x})$ rappresenta un paraboloide ellittico, e così via.
Per rappresentare graficamente il funzionale $\phi(\mathbf{x}) = \frac{1}{2}\mathbf{x}^T\mathbf{A}\mathbf{x} - \mathbf{x}^T\mathbf{b}$ per $n = 2$, scegliamo

$$\mathbf{A} = \begin{pmatrix} \lambda_1 & 0 \\ 0 & \lambda_2 \end{pmatrix} \qquad \text{e} \qquad \mathbf{b} = \begin{pmatrix} \lambda_1 \\ \lambda_2 \end{pmatrix},$$

in modo tale che il minimo coincida con $\mathbf{x}^* = (1, 1)^T$.
Il corrispondente funzionale è allora

$$\phi(x_1, x_2) = \frac{1}{2}\left(\lambda_1 x_1^2 + \lambda_2 x_2^2\right) - \lambda_1 x_1 - \lambda_2 x_2$$

Nelle Figure 1 e 2 sono rappresentati i funzionali per i valori $\lambda_1 = \lambda_2 = 1$ e $\lambda_1 = 1$, $\lambda_2 = 5$, rispettivamente. I grafici rappresentano un paraboloide circolare e uno ellittico.

Figura 1: $\lambda_1 = \lambda_2 = 1$

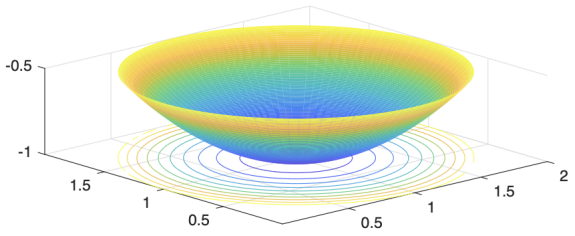
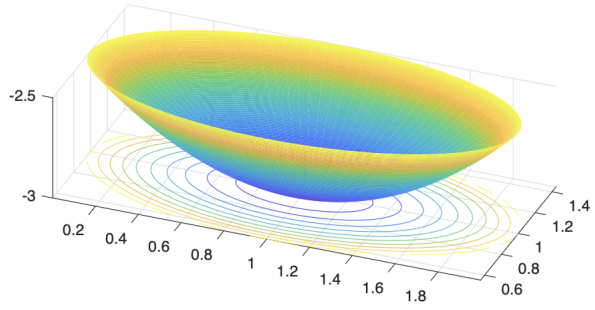


Figura 2: $\lambda_1 = 1$, $\lambda_2 = 5$



Le **curve di livello**, ovvero le proiezioni verticali sul piano (x_1, x_2) delle curve di intersezione della superficie $x_3 = \phi(x_1, x_2)$ con i piani orizzontali di equazione $x_3 = C$, sono delle ellissi concentriche, oppure delle circonferenze se $\lambda_1 = \lambda_2$, con centro $\mathbf{x}^*$, minimo del funzionale ϕ.

La struttura del funzionale ϕ, e in particolare la proprietà

$$\phi(\mathbf{x}^*) = \min_{\mathbf{x}\in\mathbb{R}^n} \phi(\mathbf{x}),$$

suggerisce una strategia per approssimare $\mathbf{x}^*$: costruire una successione $\mathbf{x}^{(k)}$ avente la proprietà che il valore del funzionale decresca nel passaggio da $\mathbf{x}^{(k)}$ a $\mathbf{x}^{(k+1)}$, avvicinandosi via via al valore minimo $\mathbf{x}^*$. Un metodo costruito in base a tale strategia è detto **metodo di discesa**.
Un metodo di discesa è definito da un relazione ricorsiva del tipo:

$$\mathbf{x}^{(k+1)} = \mathbf{x}^{(k)} + \alpha_k \mathbf{d}^{(k)}, \qquad k = 0, 1, 2, \ldots$$

ove il vettore $\mathbf{d}^{(k)}$ rappresenta il **vettore di discesa**, mentre lo scalare α_k rappresenta il **passo di discesa**.

Per minimizzare

$$\Phi(\mathbf{x}^{(k+1)}) < \Phi(\mathbf{x}^{(k)})$$

Un vettore $\mathbf{d}^{(k)} \neq \mathbf{o}$ è ammissibile come vettore di discesa se

$$\frac{\partial \phi(\mathbf{x}^{(k)})}{\partial \mathbf{d}^{(k)}} = (\mathbf{d}^{(k)})^T \boldsymbol{\nabla} \phi(\mathbf{x}^{(k)}) < 0, \qquad = (\mathbf{d}^{(k)})^T \left(\mathbf{A}\mathbf{x}^{(k)} - \mathbf{b}\right) < 0$$

ossia se muovendosi di un passo sufficientemente piccolo nella direzione e nel verso di $\mathbf{d}^{(k)}$ il funzionale $\phi(\mathbf{x})$ decresce.
I vari metodi si differenziano per la scelta del vettore di discesa $\mathbf{d}^{(k)}$. Infatti, una volta fissato $\mathbf{d}^{(k)}$, il passo di discesa α_k è univocamente determinato dalla condizione che il funzionale abbia la massima riduzione in quella direzione, cioè tale che

$$\phi\left(\mathbf{x}^{(k)} + \alpha_k \mathbf{d}^{(k)}\right) = \min_{\alpha \in \mathbb{R}} \phi\left(\mathbf{x}^{(k)} + \alpha \mathbf{d}^{(k)}\right)$$

Calcoliamo α_k. A tale scopo riscriviamo la funzione

$$\varphi(\alpha) = \phi\left(\mathbf{x}^{(k)} + \alpha \mathbf{d}^{(k)}\right) :$$

$$\begin{aligned}
\varphi(\alpha) &= \frac{1}{2}\left(\mathbf{x}^{(k)} + \alpha \mathbf{d}^{(k)}\right)^T \mathbf{A}\left(\mathbf{x}^{(k)} + \alpha \mathbf{d}^{(k)}\right) - \left(\mathbf{x}^{(k)} + \alpha \mathbf{d}^{(k)}\right)^T \mathbf{b} \\
&= \frac{1}{2}\left((\mathbf{x}^{(k)})^T \mathbf{A}\mathbf{x}^{(k)} + \alpha (\mathbf{d}^{(k)})^T \mathbf{A}\mathbf{x}^{(k)} + \alpha (\mathbf{x}^{(k)})^T \mathbf{A}\mathbf{d}^{(k)} + \alpha^2 (\mathbf{d}^{(k)})^T \mathbf{A}\mathbf{d}^{(k)}\right) \\
&\quad - (\mathbf{x}^{(k)})^T \mathbf{b} - \alpha (\mathbf{d}^{(k)})^T \mathbf{b} \\
&= \frac{1}{2}(\mathbf{x}^{(k)})^T \mathbf{A}\mathbf{x}^{(k)} - (\mathbf{x}^{(k)})^T \mathbf{b} + \alpha (\mathbf{d}^{(k)})^T \left(\mathbf{A}\mathbf{x}^{(k)} - \mathbf{b}\right) + \frac{1}{2}\alpha^2 (\mathbf{d}^{(k)})^T \mathbf{A}\mathbf{d}^{(k)} \\
&= \phi(\mathbf{x}^{(k)}) - \alpha (\mathbf{d}^{(k)})^T \mathbf{r}^{(k)} + \frac{1}{2}\alpha^2 \underbrace{(\mathbf{d}^{(k)})^T \mathbf{A}\mathbf{d}^{(k)}}_{>0}
\end{aligned}$$

avendo usato $(\mathbf{d}^{(k)})^T \mathbf{A}\mathbf{x}^{(k)} = (\mathbf{x}^{(k)})^T \mathbf{A}\mathbf{d}^{(k)}$ e posto $\mathbf{r}^{(k)} := \mathbf{r}(\mathbf{x}^{(k)})$.
Osserviamo che $\varphi(\alpha)$ è una parabola con la concavità verso l'alto, essendo

$$(\mathbf{d}^{(k)})^T \mathbf{A}\mathbf{d}^{(k)} > 0.$$

Per determinare il punto di minimo calcoliamo $\varphi'(\alpha)$:

$$\varphi'(\alpha) = -(\mathbf{d}^{(k)})^T \mathbf{r}^{(k)} + \alpha (\mathbf{d}^{(k)})^T \mathbf{A}\mathbf{d}^{(k)}$$

Da $\varphi'(\alpha) = 0$ si deduce che il punto di minimo è dato da

$$\alpha_k = \frac{(\mathbf{d}^{(k)})^T \mathbf{r}^{(k)}}{(\mathbf{d}^{(k)})^T \mathbf{A}\mathbf{d}^{(k)}}$$

Da $(\mathbf{d}^{(k)})^T \boldsymbol{\nabla} \phi(\mathbf{x}^{(k)}) < 0$ segue che

$$(\mathbf{d}^{(k)})^T \mathbf{r}^{(k)} = -(\mathbf{d}^{(k)})^T \boldsymbol{\nabla} \phi(\mathbf{x}^{(k)}) > 0$$

e, quindi, $\alpha_k > 0$.
I diversi metodi di discesa si distinguono per la diversa scelta del vettore $\mathbf{d}^{(k)}$.

Metodo del gradiente

Per effettuare una scelta appropriata del vettore $\mathbf{d}^{(k)}$, ricordiamo che la direzione del gradiente è quella lungo la quale la funzione varia più rapidamente: la massima crescita si ha nella direzione e verso del gradiente, la massima decrescita si ha nella direzione del gradiente, ma nel verso opposto a esso.
Pertanto una scelta naturale per il vettore $\mathbf{d}^{(k)}$ consiste nel porre

$$\mathbf{d}^{(k)} = -\boldsymbol{\nabla}\phi(\mathbf{x}^{(k)}) = \mathbf{r}^{(k)}$$

vale a dire muoversi nella direzione di massima decrescita per ϕ (**"steepest descent"**). Il corrispondente metodo di discesa è detto **metodo del gradiente** oppure **metodo della massima pendenza**.
Per il metodo del gradiente si ha allora

$$\alpha_k = \frac{(\mathbf{r}^{(k)})^T\mathbf{r}^{(k)}}{(\mathbf{r}^{(k)})^T\mathbf{A}\mathbf{r}^{(k)}}$$

$$\begin{aligned}\mathbf{r}^{(k+1)} &= \mathbf{b} - \mathbf{A}\mathbf{x}^{(k+1)} \\ &= \mathbf{b} - \mathbf{A}\left(\mathbf{x}^{(k)} + \alpha_k\mathbf{r}^{(k)}\right) \\ &= \mathbf{r}^{(k)} - \alpha_k\mathbf{A}\mathbf{r}^{(k)},\end{aligned}$$

e, tenendo conto dell'espressione di α_k, si ha

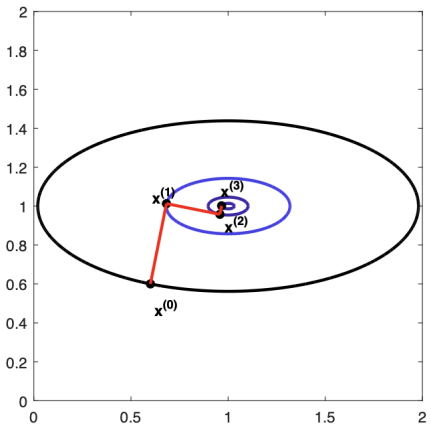
$$\begin{aligned}(\mathbf{r}^{(k+1)})^T\mathbf{r}^{(k)} &= \left(\mathbf{r}^{(k)} - \alpha_k\mathbf{A}\mathbf{r}^{(k)}\right)^T\mathbf{r}^{(k)} \\ &= (\mathbf{r}^{(k)})^T\mathbf{r}^{(k)} - \alpha_k(\mathbf{r}^{(k)})^T\mathbf{A}\mathbf{r}^{(k)} = 0\end{aligned}$$

Pertanto, a ogni passo, $\mathbf{d}^{(k+1)} = \mathbf{r}^{(k+1)}$ è ortogonale al vettore $\mathbf{d}^{(k)} = \mathbf{r}^{(k)}$ del passo precedente, ossia i gradienti sono a due a due ortogonali, ma non tutti tra loro ortogonali. Vedremo un miglioramento del metodo del gradiente che renderà le direzioni di discesa tutte tra loro ortogonali e questo darà al metodo fortissime proprietà di convergenza.
La scelta $\mathbf{d}^{(k)} = \mathbf{r}^{(k)}$, anche se in ciascun punto $\mathbf{x}^{(k)}$ sfrutta la direzione della massima pendenza, come vedremo, può non essere la migliore, in particolare quando la matrice è mal condizionata.

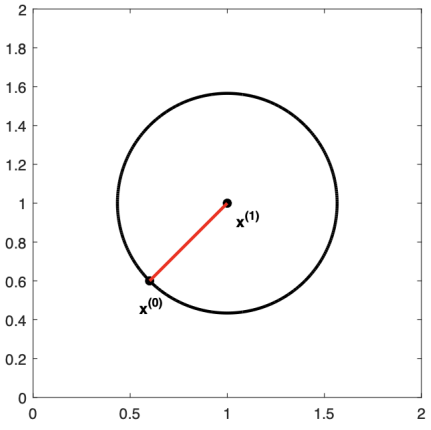
Esempio 2.6.11. Per mostrare il comportamento del metodo nel caso $n = 2$, scegliamo

$$\mathbf{A} = \begin{pmatrix}\lambda_1 & 0 \\ 0 & \lambda_2\end{pmatrix} \qquad \text{e} \qquad \mathbf{b} = \begin{pmatrix}\lambda_1 \\ \lambda_2\end{pmatrix},$$

in modo tale che il minimo coincida con $\mathbf{x}^* = (1, 1)^T$.
Partendo da $\mathbf{x}^{(0)} = (0.6, 0.6)^T$, per $\lambda_1 = 1$ e $\lambda_2 = 5$ il metodo del gradiente converge seguendo una traiettoria a zig-zag e graficamente si ha



Nel caso $n = 2$, il metodo del gradiente ha dunque il seguente comportamento:
si parte da $\mathbf{x}^{(0)}$ e si scende lungo la direzione del gradiente, che è ortogonale alla curva di livello passante per $\mathbf{x}^{(0)}$.
Si giunge in $\mathbf{x}^{(1)} = \mathbf{x}^{(0)} + \alpha_0 \mathbf{d}^{(0)}$ che rappresenta il punto in cui il funzionale $\phi(\mathbf{x}^{(0)} + \alpha \mathbf{d}^{(0)})$ assume valore minimo al variare di α; questo valore si raggiunge per $\alpha = \alpha_0$. In $\mathbf{x}^{(1)}$ la direzione del gradiente $\mathbf{d}^{(0)} = -\boldsymbol{\nabla}\phi(\mathbf{x}^{(0)})$ è tangente alla curva di livello $z = \phi(\mathbf{x}^{(1)})$.
Quindi, si riparte da $\mathbf{x}^{(1)}$ lungo la direzione $\mathbf{d}^{(1)} = -\boldsymbol{\nabla}\phi(\mathbf{x}^{(1)})$ del gradiente, ortogonale alla direzione $\mathbf{d}^{(0)}$, per raggiungere $\mathbf{x}^{(2)}$; e così via....
Pertanto, si ha che due direzioni consecutive $\mathbf{d}^{(k)}$ e $\mathbf{d}^{(k+1)}$ sono ortogonali tra loro, e due direzioni alternate $\mathbf{d}^{(k)}$ e $\mathbf{d}^{(k+2)}$ sono parallele tra loro.
Per $\lambda_1 = \lambda_2 = 1$ il metodo del gradiente converge in una sola iterazione, perchè la direzione del gradiente passa per il centro ($\mathbf{x}^{(1)} = \mathbf{x}^*$).



Nell'implementazione del metodo del gradiente, al posto del criterio di arresto precedentemente utilizzato $\|\mathbf{x}^{(k+1)} - \mathbf{x}^{(k)}\|_2 \leq tol \|\mathbf{x}^{(k+1)}\|_2$, considereremo il seguente criterio:

$$\|\mathbf{r}^{(k)}\|_2 \leq tol \|\mathbf{b}\|_2, \qquad tol \geq 2\varepsilon_m$$

Tale criterio di arresto scaturisce dalle seguenti motivazioni.

- Poiché $\mathbf{r}^{(k)} = \mathbf{b} - \mathbf{A}\mathbf{x}^{(k)} = \mathbf{A}\mathbf{x}^* - \mathbf{A}\mathbf{x}^{(k)} = \mathbf{A}\mathbf{e}^{(k)}$, si ha che, essendo $\mathbf{A}$ non singolare, $\mathbf{e}^{(k)} = \mathbf{o}$ se e solo se $\mathbf{r}^{(k)} = \mathbf{o}$. Quindi ha senso richiedere che $\mathbf{r}^{(k)}$ sia "piccolo".
- Poiché $\mathbf{A}\mathbf{x}^{(k)} = \mathbf{b} - \mathbf{r}^{(k)}$, si ha che $\mathbf{r}^{(k)}$ è quella quantità che sommata a $\mathbf{b}$ consente di ottenere $\mathbf{x}^{(k)}$. In precisione finita di calcolo, $\mathbf{r}^{(k)}$ dà contributo a $\mathbf{b}$ solo se sufficientemente grande,

ovvero $\|\mathbf{r}^{(k)}\|_2 > eps\|\mathbf{b}\|_2$. Ne consegue che se $\|\mathbf{r}^{(k)}\|_2 \leq eps\|\mathbf{b}\|_2$, non ha senso procedere con il metodo iterativo in quanto l'approssimazione $\mathbf{x}^{(k)}$ rimane invariata. Da tale osservazione scaturisce il criterio d'arresto $\|\mathbf{r}^{(k)}\|_2 \leq tol\|\mathbf{b}\|_2$, con $tol \geq eps = 2\varepsilon_m$.

- Anche le function MATLAB che implementano i metodi di discesa utilizzano il criterio di arresto sul residuo $\|\mathbf{r}^{(k)}\|_2$, e non sulla quantità $\|\mathbf{x}^{(k+1)} - \mathbf{x}^{(k)}\|_2$.

Scriviamo la function MATLAB che implementa il metodo del gradiente, tenendo conto che per esso le formule caratterizzanti sono le seguenti:

$$\mathbf{x}^{(k+1)} = \mathbf{x}^{(k)} + \alpha_k \mathbf{r}^{(k)}, \qquad \alpha_k = \frac{(\mathbf{r}^{(k)})^T \mathbf{r}^{(k)}}{(\mathbf{r}^{(k)})^T \mathbf{A}\mathbf{r}^{(k)}}, \qquad \mathbf{r}^{(k+1)} = \mathbf{r}^{(k)} - \alpha_k \mathbf{A}\mathbf{r}^{(k)}$$

Function MATLAB

```
function [x,k,ier] = gradiente(A,b,x,tol,kmax)
ier = 0;
r = b-A*x;
tolb = tol*norm(b);
while k <= kmax && norm(r) > tolb
    s = A*r;
    alpha = r'*r/(r'*s);
    x = x+alpha*r;
    r = r-alpha*s;
end
if k == kmax+1 && norm(r) > tolb
    ier = 0;
else
    ier = 1;
end
```

Osservazione:

- A ogni passo occorre calcolare un prodotto matrice per vettore, il cui costo in termini delle operazioni aritmetiche di moltiplicazione (e di addizione), è essenzialmente n^2 per n grande. Gli altri passi dell'algoritmo (calcolo di prodotti scalari e aggiornamento di vettori) richiedono un numero di operazioni dell'ordine di n. Pertanto il costo complessivo del metodo del gradiente è essenzialmente dato da kn^2 per n grande, ove k è il numero di iterazioni eseguite.
- Per ridurre il costo computazionale si è usata l'espressione di $\mathbf{r}^{(k+1)}$ in funzione di $\mathbf{r}^{(k)}$, anche se in pratica per garantire una migliore stabilità dell'algoritmo è opportuno dopo un certo numero di passi calcolare il residuo con la relazione

$$\mathbf{r}^{(k+1)} = \mathbf{b} - \mathbf{A}\mathbf{x}^{(k+1)}.$$

Per fornire un risultato di convergenza per il metodo del gradiente, definiamo al passo k l'errore

$$\mathbf{e}^{(k)} := \mathbf{x}^* - \mathbf{x}^{(k)}$$

e introduciamo la norma vettoriale, detta **norma dell'energia**,

$$\|\mathbf{x}\|_{\mathbf{A}} = \sqrt{\mathbf{x}^T \mathbf{A}\mathbf{x}}$$

che altro non è che la norma indotta dal prodotto scalare $(\mathbf{x}, \mathbf{y}) \mapsto \mathbf{x}^T \mathbf{A}\mathbf{y}$ a sua volta indotto da $\mathbf{A}$. Si può dimostrare il seguente teorema.

Teorema 2.6.12. Siano $\lambda_{\max}$ e $\lambda_{\min}$ il massimo e il minimo autovalore della matrice simmetrica e definita positiva $\mathbf{A}$ di ordine n. Sia inoltre $K_2(\mathbf{A}) = \lambda_{\max}/\lambda_{\min} \geq 1$ il numero di condizionamento spettrale della matrice $\mathbf{A}$. La successione $\{\mathbf{x}^{(k)}\}$ generata dal metodo del gradiente converge qualunque sia il vettore iniziale $\mathbf{x}^{(0)}$ e il corrispondente errore $\mathbf{e}^{(k)}$ soddisfa per ogni k la seguente stima:

$$\|\mathbf{e}^{(k)}\|_{\mathbf{A}} \leq \left(\frac{K_2(\mathbf{A})-1}{K_2(\mathbf{A})+1}\right)^k \|\mathbf{e}^{(0)}\|_{\mathbf{A}}$$

Osservazione: Osserviamo che il metodo converge perché il fattore di riduzione

$$\frac{K_2(\mathbf{A})-1}{K_2(\mathbf{A})+1} = 1 - \frac{2}{K_2(\mathbf{A})+1}$$

è minore di 1.
Inoltre la velocità di convergenza dipende da tale fattore: quanto più esso è piccolo, tanto più è rapida la convergenza. Poiché il fattore di riduzione è tanto più vicino a zero quanto più $K_2(\mathbf{A})$ si avvicina a 1, la convergenza della successione $\{\mathbf{x}^{(k)}\}$ è tanto più rapida quanto più la matrice è ben condizionata.
Quando $K_2(\mathbf{A}) = 1$, cioè quando gli autovalori sono tutti uguali, il metodo converge in una sola iterazione. Quando, invece $K_2(\mathbf{A}) \gg 1$, la convergenza del metodo può essere molto lenta e il numero delle iterazioni è proporzionale a $K_2(\mathbf{A})$.

Osservazione: Il criterio d'arresto sul residuo può non essere affidabile quando la matrice $\mathbf{A}$ è malcondizionata. Infatti, si può dimostrare che

$$\frac{\|\mathbf{e}^{(k)}\|_2}{\|\mathbf{x}^*\|_2} \leq K_2(\mathbf{A}) \frac{\|\mathbf{r}^{(k)}\|_2}{\|\mathbf{b}\|_2}$$

e pertanto, pur essendo piccolo il residuo, l'errore potrebbe non essere altrettanto piccolo.
In caso di cattivo condizionamento, anche il criterio di arresto $\|\mathbf{x}^{(k+1)} - \mathbf{x}^{(k)}\|_2 \leq tol\|\mathbf{x}^{(k+1)}\|_2$, potrebbe non essere affidabile; infatti, in questo caso, a causa della convergenza lenta del metodo, due iterate successive potrebbero essere vicine tra loro (e quindi il criterio risulta soddisfatto) ma entrambe lontane dalla soluzione esatta.

Metodo del gradiente coniugato

La scelta della direzione di massima decrescita locale del funzionale ϕ come direzione di discesa, può dunque non essere la migliore, in particolare quando la matrice è malcondizionata. In tal caso, per $n = 2$ per esempio, le curve di livello sono ellissi molto allungate e muoversi lungo la direzione del gradiente può portare a una modesta riduzione del funzionale. Un esame del problema mostra che la direzione

$$\mathbf{d}^{(k)} = \begin{cases} \mathbf{r}^{(0)}, & \text{se } k = 0 \\ \mathbf{r}^{(k)} + \beta_k \mathbf{d}^{(k-1)}, & \text{se } k \geq 1 \end{cases}$$

dove β_k è determinato in modo tale che

$$(\mathbf{d}^{(k)})^T \mathbf{A} \mathbf{d}^{(k-1)} = 0$$

Nel caso di $d^{(1)}$, imponendo la condizione di ortogonalità rispetto ad A, si ha

$$\left[r^{(1)} + \beta_1 d^{(0)}\right]^T A d^{(0)} = 0.$$

Poiché $d^{(0)} = r^{(0)}$, segue che

$$\left[r^{(1)} + \beta_1 r^{(0)}\right]^T A r^{(0)} = 0,$$

da cui

$$r^{(1)T} A r^{(0)} + \beta_1 r^{(0)T} A r^{(0)} = 0.$$

Ricavando β_1, si ottiene

$$\boxed{\beta_1 = -\frac{r^{(1)T} A r^{(0)}}{r^{(0)T} A r^{(0)}}.}$$

Il procedimento si generalizza al caso k-esimo, ottenendo

$$\boxed{\beta_k = -\frac{r^{(k)T} A d^{(k-1)}}{d^{(k-1)T} A d^{(k-1)}}.}$$

I vettori $\mathbf{d}^{(k)}$ e $\mathbf{d}^{(k-1)}$ che soddisfano $(\mathbf{d}^{(k)})^T \mathbf{A} \mathbf{d}^{(k-1)} = 0$ sono detti **A-coniugati** e da questa definizione discende il nome del corrispondente metodo di discesa, che è detto **metodo del gradiente coniugato**. In particolare stiamo richiedendo che $\mathbf{d}^{(k)}$ e $\mathbf{d}^{(k-1)}$ siano ortogonali rispetto al prodotto scalare indotto da $\mathbf{A}$.
Si dimostra che la direzione $\mathbf{d}^{(k)}$ è $\mathbf{A}$-coniugata non solo con $\mathbf{d}^{(k-1)}$ ma con tutte le precedenti, cioè

$$(\mathbf{d}^{(j)})^T \mathbf{A} \mathbf{d}^{(k)} = 0 \qquad \forall j, k \geq 0, \quad j \neq k$$

e che i residui sono tutti ortogonali (nel senso del prodotto scalare standard) tra loro

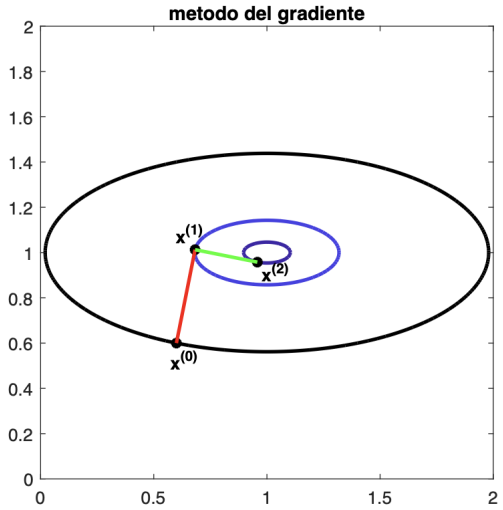
$$(\mathbf{r}^{(j)})^T \mathbf{r}^{(k)} = 0 \qquad \forall j, k \geq 0, \quad j \neq k$$

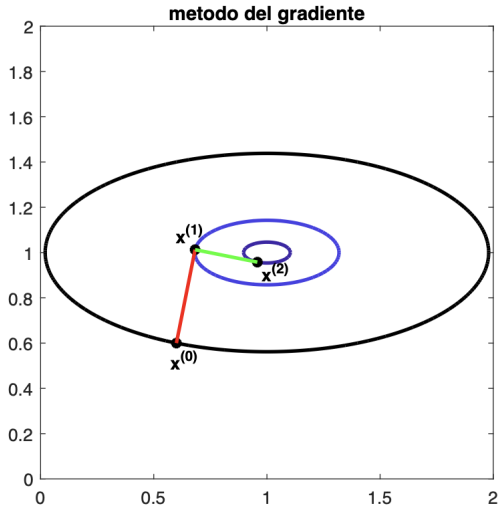
Osservazione: Ricordiamo che nel caso del metodo del gradiente solo due residui consecutivi sono ortogonali tra loro, ovvero solo due direzioni consecutive sono ortogonali. Due direzioni alternate sono invece parallele. Pertanto, esistono solo due direzioni linearmente indipendenti.
Nel caso del metodo del gradiente coniugato, i residui $\mathbf{r}^{(0)}, \mathbf{r}^{(1)}, \ldots, \mathbf{r}^{(n-1)}$ formano un sistema ortogonale in $\mathbb{R}^n$; se essi sono tutti diversi dal vettore nullo formano una base in tale spazio.
Pertanto, l'$n+1$-esimo residuo $\mathbf{r}^{(n)}$, essendo ortogonale a ciascuno di essi, sarà nullo, e dunque $\mathbf{x}^{(n)} = \mathbf{x}^*$. In altri termini, il metodo del gradiente coniugato fornisce, in aritmetica infinita e quindi in assenza degli errori dovuti all'aritmetica di macchina, la soluzione del sistema lineare in al più n iterazioni.
In effetti tale metodo fu proposto come metodo diretto. Esso viene considerato come un metodo iterativo perché se n è molto grande e la matrice non è malcondizionata, si ottiene una buona approssimazione della soluzione dopo un numero di iterazioni molto minore di n. Pertanto il metodo del gradiente coniugato risulta molto conveniente per trattare problemi di grosse dimensioni.

Esempio 2.6.13. Per confrontare, nel caso $n = 2$, il comportamento del metodo del gradiente e quello del gradiente coniugato scegliamo

$$\mathbf{A} = \begin{pmatrix} \lambda_1 & 0 \\ 0 & \lambda_2 \end{pmatrix}, \qquad \mathbf{b} = \begin{pmatrix} \lambda_1 \\ \lambda_2 \end{pmatrix}$$

in modo tale che il minimo coincida con $\mathbf{x}^* = (1, 1)^T$.
Partendo da $\mathbf{x}^{(0)} = (0.6, 0.6)^T$, per $\lambda_1 = 1$ e $\lambda_2 = 5$ graficamente si ha





Il metodo del gradiente coniugato converge in sole due iterazioni.

Osservazione:

- Come per il metodo del gradiente, anche per il metodo del gradiente coniugato, l'operazione che presenta in generale un maggiore costo computazionale è quella relativa alla moltiplicazione della matrice $\mathbf{A}$ per il vettore $\mathbf{d}^{(k)}$. Essendo il costo di quest'ultimo pari a n^2 moltiplicazioni (e addizioni), il costo complessivo è circa kn^2, ove k è il numero di iterazioni eseguite.
- Se il metodo richiedesse un numero di passi dell'ordine di n e se $\mathbf{A}$ non fosse sparsa, il costo complessivo sarebbe dell'ordine di n^3, superiore a quella del metodo di Choleski. Tuttavia, nella risoluzione di sistemi di equazioni lineari che scaturiscono dalla discretizzazione di problemi differenziali, il numero di iterazioni richieste è di solito molto inferiore alla dimensione della matrice e la matrice è sparsa.

Per quanto riguarda la stima di convergenza dell'errore $\mathbf{e}^{(k)}$ associato alla successione $\{\mathbf{x}^{(k)}\}$ generata dal metodo del gradiente coniugato, vale il seguente teorema.

Teorema 2.6.14. La successione $\{\mathbf{x}^{(k)}\}$ generata dal metodo del gradiente coniugato converge qualunque sia il vettore iniziale $\mathbf{x}^{(0)}$ in al più n iterazioni e il corrispondente errore $\mathbf{e}^{(k)}$ soddisfa per ogni k la seguente stima:

$$\|\mathbf{e}^{(k)}\|_{\mathbf{A}} \leq 2 \left(\frac{\sqrt{K_2(\mathbf{A})} - 1}{\sqrt{K_2(\mathbf{A})} + 1} \right)^k \|\mathbf{e}^{(0)}\|_{\mathbf{A}}$$

Osservazione:

- Se $K_2(\mathbf{A}) \approx 1$ sono sufficienti pochi passi, mentre se $K_2(\mathbf{A}) \gg 1$ è possibile che siano necessari n passi per ottenere un'approssimazione accettabile della soluzione, e se n è molto grande è possibile che non si riesca ad ottenere un'approssimazione accettabile a causa degli errori di arrotondamento.
- I metodi del gradiente e del gradiente coniugato ammettono lo stesso costo computazionale; mentre, per quanto riguarda la velocità di convergenza, il fattore di riduzione del metodo del

gradiente dipende da $K_2(\mathbf{A})$ e quello del gradiente coniugato da $\sqrt{K_2(\mathbf{A})}$. Poiché quest'ultimo valore risulta più piccolo, il metodo del gradiente coniugato converge più velocemente del metodo del gradiente. Tuttavia, se la matrice è malcondizionata la convergenza di entrambi i metodi potrebbe risultare lenta.

Comandi MATLAB

- `x = pcg(A,b)` determina la soluzione $\mathbf{x}$ del sistema lineare $\mathbf{Ax} = \mathbf{b}$ simmetrico e definito positivo mediante il metodo del gradiente coniugato, a partire dal vettore iniziale $\mathbf{x}^{(0)} = \mathbf{o}$.
- `x = pcg(A,b,tol,kmax)` determina la soluzione $\mathbf{x}$ imponendo il criterio d'arresto $\|\mathbf{r}\|_2/\|\mathbf{b}\|_2 \leq$ `tol` e il massimo numero di iterazioni `kmax`. Se `tol = []` allora di default `tol` $= 10^{-6}$; se `kmax = []` allora `kmax` $= \min\{n, 20\}$.

2.7 Sistemi lineare precondizionati

L'analisi della convergenza dei metodi di discesa per un sistema lineare $\mathbf{Ax} = \mathbf{b}$ mette in evidenza il ruolo cruciale del numero di condizionamento $K_2(\mathbf{A})$ della matrice $\mathbf{A}$: i suddetti metodi convergono lentamente se la matrice $\mathbf{A}$ è malcondizionata.
Per ovviare a questo inconveniente, si può sostituire il sistema $\mathbf{Ax} = \mathbf{b}$ con il sistema equivalente

$$\mathbf{P}^{-1}\mathbf{Ax} = \mathbf{P}^{-1}\mathbf{b}$$

dove $\mathbf{P}$ è una matrice non singolare avente le seguenti proprietà:

1. $K_2(\mathbf{P}^{-1}\mathbf{A}) \ll K_2(\mathbf{A})$;
2. gli elementi di $\mathbf{P}$ sono calcolabili in modo poco costoso;
3. il prodotto matrice-vettore $\mathbf{y} = \mathbf{P}^{-1}\mathbf{z}$ (ovvero la risoluzione del sistema $\mathbf{Py} = \mathbf{z}$) deve avere un costo confrontabile con quello del prodotto matrice-vettore per il calcolo di $\mathbf{y} = \mathbf{Az}$, tipicamente $\longleftrightarrow O(n^2)$, in modo tale che il costo risulti simile a quello necessario per risolvere il problema mal condizionato.

Definizione 2.7.1. Una matrice $\mathbf{P}$ avente le suddette proprietà si dice **matrice di precondizionamento** per $\mathbf{A}$.

Osservazione: Osserviamo che la matrice che realizza in modo ottimale la prima condizione è la matrice $\mathbf{P} = \mathbf{A}$, per la quale si ha

$$K_2(\mathbf{P}^{-1}\mathbf{A}) = K_2(\mathbf{I}) = 1$$

Ma $\mathbf{P} = \mathbf{A}$ non soddisfa l'ultima delle precedenti condizioni. Tuttavia l'osservazione suggerisce di scegliere $\mathbf{P}$ come una opportuna approssimazione della matrice $\mathbf{A}$.

Nel caso di una matrice $\mathbf{A}$ simmetrica e definita positiva, una strategia che porta a una matrice di precondizionamento efficace consiste nel costruire una **fattorizzazione incompleta di Choleski** della matrice $\mathbf{A}$.

Precisamente, si sceglie $\mathbf{P} = \bar{\mathbf{R}}^T\bar{\mathbf{R}}$, con $\bar{\mathbf{R}}$ triangolare superiore ottenuta applicando l'algoritmo di Choleski ad $\mathbf{A}$, ma calcolando soltanto gli elementi nelle posizioni in cui $\mathbf{A}$ è non nulla e trascurando tutti gli altri ($\bar{r}_{ij} = 0$ se $a_{ij} = 0$). Ne segue che

$$\mathbf{A} = \bar{\mathbf{R}}^T\bar{\mathbf{R}} + \mathbf{E}, \qquad e_{ij} = 0 \quad \text{se } a_{ij} \neq 0,$$

cioè

$$\left(\bar{\mathbf{R}}^T\bar{\mathbf{R}}\right)_{ij} = a_{ij} \qquad \text{se } a_{ij} \neq 0.$$

Cioè $\bar{\mathbf{R}}^T\bar{\mathbf{R}}$ coincide con $\mathbf{A}$ in tutte le posizioni in cui $\mathbf{A}$ è non nulla; la matrice $\mathbf{E}$ contiene i contributi trascurati (fill-in).

$$\mathbf{P} = \bar{\mathbf{R}}^T\bar{\mathbf{R}}$$

Pertanto $\mathbf{P} = \mathbf{A} - \mathbf{E}$ è un'approssimazione di $\mathbf{A}$ e $\mathbf{P}^{-1}\mathbf{A} = \mathbf{I} + \mathbf{P}^{-1}\mathbf{E}$ è tanto più vicina all'identità quanto più $\mathbf{E}$ è "piccola".
Gli elementi non nulli di $\bar{\mathbf{R}}$ sono, in generale, diversi dai corrispondenti elementi del fattore $\mathbf{R}$ della fattorizzazione completa $\mathbf{A} = \mathbf{R}^T\mathbf{R}$; inoltre $\bar{\mathbf{R}}$ è sparsa tanto quanto la parte triangolare superiore di $\mathbf{A}$.
Per un'ampia classe di metodi di discretizzazione che conducono a matrici $\mathbf{A}$ simmetriche e definite positive, risulta $K_2(\mathbf{P}^{-1}\mathbf{A}) \ll K_2(\mathbf{A})$ e un sistema $\mathbf{Py} = \mathbf{z}$ può essere facilmente risolto mediante la risoluzione dei due sistemi triangolari $\bar{\mathbf{R}}^T\mathbf{w} = \mathbf{z}$ e $\bar{\mathbf{R}}\mathbf{y} = \mathbf{w}$.

Osservazione: A differenza della fattorizzazione completa, la fattorizzazione incompleta di Choleski di una matrice simmetrica e definita positiva non sempre esiste: l'algoritmo può incontrare un elemento diagonale non positivo e interrompersi. Essa esiste certamente, per esempio, per le matrici simmetriche e definite positive con elementi fuori diagonale non positivi, come quelle che si ottengono dalla discretizzazione dell'equazione di Poisson.

2.7.1 Metodo del gradiente coniugato precondizionato

Si noti che la matrice $\mathbf{P}^{-1}\mathbf{A}$ non è simmetrica e, quindi, i metodi di discesa non possono essere applicati direttamente al sistema $\mathbf{P}^{-1}\mathbf{Ax} = \mathbf{P}^{-1}\mathbf{b}$.
Per ovviare a questo inconveniente si può sostituire il problema originario simmetrico e definito positivo $\mathbf{Ax} = \mathbf{b}$ con il problema

$$(\bar{\mathbf{R}}^T)^{-1}\mathbf{Ax} = (\bar{\mathbf{R}}^T)^{-1}\mathbf{b}$$
$$\Longrightarrow \underbrace{(\bar{\mathbf{R}}^T)^{-1}\mathbf{A}\bar{\mathbf{R}}^{-1}}_{\bar{\mathbf{A}}}\underbrace{\bar{\mathbf{R}}\mathbf{x}}_{\bar{\mathbf{x}}} = \underbrace{(\bar{\mathbf{R}}^T)^{-1}\mathbf{b}}_{\bar{\mathbf{b}}}$$

dove $\bar{\mathbf{R}}^{-1}\bar{\mathbf{R}} = \mathbf{I}$.

$$\Longrightarrow \begin{cases} \bar{\mathbf{A}}\bar{\mathbf{x}} = \bar{\mathbf{b}} \overset{\text{grad coniug}}{\Longrightarrow} \bar{\mathbf{x}} \\ \bar{\mathbf{R}}\mathbf{x} = \bar{\mathbf{x}} \Longrightarrow \mathbf{x} \end{cases}$$

Osserviamo che $\bar{\mathbf{A}}$ è simmetrica,

$$\begin{aligned} \bar{\mathbf{A}}^T &= \left((\bar{\mathbf{R}}^T)^{-1}\mathbf{A}\bar{\mathbf{R}}^{-1}\right)^T \\ &= (\bar{\mathbf{R}}^{-1})^T\mathbf{A}^T\bar{\mathbf{R}}^{-1} \\ &= (\bar{\mathbf{R}}^T)^{-1}\mathbf{A}\bar{\mathbf{R}}^{-1} = \bar{\mathbf{A}}, \end{aligned}$$

e definita positiva:

$$\mathbf{x}^T\bar{\mathbf{A}}\mathbf{x} = \left(\bar{\mathbf{R}}^{-1}\mathbf{x}\right)^T \mathbf{A}\left(\bar{\mathbf{R}}^{-1}\mathbf{x}\right) = \mathbf{y}^T\mathbf{A}\mathbf{y} > 0,$$

essendo $\mathbf{A}$ simmetrica e definita positiva e $\mathbf{y} = \bar{\mathbf{R}}^{-1}\mathbf{x} \neq 0$ per ogni $\mathbf{x} \neq 0$ ($\bar{\mathbf{R}}$ è non singolare, perché triangolare con elementi diagonali positivi). È pertanto possibile applicare il metodo del gradiente coniugato al sistema $\bar{\mathbf{A}}\bar{\mathbf{x}} = \bar{\mathbf{b}}$.
Inoltre, da $\mathbf{A} = \bar{\mathbf{R}}^T\bar{\mathbf{R}} + \mathbf{E}$ segue

$$\bar{\mathbf{A}} = (\bar{\mathbf{R}}^T)^{-1}(\bar{\mathbf{R}}^T\bar{\mathbf{R}} + \mathbf{E})\bar{\mathbf{R}}^{-1} = \mathbf{I} + (\bar{\mathbf{R}}^T)^{-1}\mathbf{E}\bar{\mathbf{R}}^{-1}$$

cioè $\bar{\mathbf{A}}$ differisce dall'identità per una perturbazione dovuta al solo fill-in trascurato: se il precondizionatore è efficace, $K_2(\bar{\mathbf{A}}) < K_2(\mathbf{A})$ e il metodo converge più rapidamente. Ottenuto $\bar{\mathbf{x}}$, la soluzione si ricava da $\bar{\mathbf{R}}\mathbf{x} = \bar{\mathbf{x}}$.
Tale procedura prende il nome di **metodo del gradiente coniugato precondizionato**.
In MATLAB esistono i seguenti comandi

- `L = ichol(A)` calcola il fattore $\mathbf{L} = \bar{\mathbf{R}}^T$ del precondizionatore $\mathbf{P} = \bar{\mathbf{R}}^T\bar{\mathbf{R}}$ che si ottiene mediante la fattorizzazione incompleta di Choleski di $\mathbf{A}$. La matrice $\mathbf{A}$ deve essere definita in formato sparso; quindi, nel caso non lo fosse, usare la seguente sintassi

  `L = ichol(sparse(A)).`

- `x = pcg(A,b,tol,kmax,M,N,x0)` determina la soluzione $\mathbf{x}$ del sistema lineare $\mathbf{Ax} = \mathbf{b}$ con il metodo del gradiente coniugato precondizionato a partire dal vettore iniziale `x0`. Il precondizionatore utilizzato è $\mathbf{P} = \mathbf{MN}$. Se si vuole utilizzare il precondizionatore che si ottiene mediante la fattorizzazione incompleta di Choleski, allora `M=L` e `N=L'`. Se `M=[]` e `N=[]` il metodo non utilizza precondizionatori.

Osservazione: Se la matrice $\mathbf{A}$ non è simmetrica e definita positiva, non è possibile applicare i metodi di discesa al sistema $\mathbf{Ax} = \mathbf{b}$.
Tenendo conto che $\mathbf{A}^T\mathbf{A}$ è simmetrica e definita positiva se e solo se $\mathbf{A}$ non è singolare, un modo per ricondursi a un sistema simmetrico e definito positivo consiste nel risolvere il sistema

$$\mathbf{A}^T\mathbf{Ax} = \mathbf{A}^T\mathbf{b}$$

Tuttavia questa procedura non è conveniente in quanto si dimostra che $K_2(\mathbf{A}^T\mathbf{A}) = (K_2(\mathbf{A}))^2$ e il prodotto matrice-vettore $\mathbf{y} = \mathbf{A}^T\mathbf{Az}$ costa il doppio rispetto al prodotto $\mathbf{y} = \mathbf{Az}$.
Sono stati quindi proposti metodi alternativi tra i quali molto popolare è il metodo GMRes.

Capitolo 3

Approssimazione di funzioni e di dati

Siano assegnati $n+1$ dati (x_i, y_i), con $x_i \in \mathbb{R}$ e $y_i \in \mathbb{R}$, $i = 1, \ldots, n+1$, ove y_i è il valore assunto da una certa funzione $f(x)$ in x_i oppure la misura di un dato sperimentale in corrispondenza di x_i.

Definizione 3.0.1. Il **criterio dell'interpolazione** consiste nello scegliere, come approssimante dei dati (x_i, y_i), $i = 1, \ldots, n+1$, una funzione $\tilde{f}(x)$ soddisfacente le seguenti condizioni:

$$\tilde{f}(x_i) = y_i \qquad i = 1, \ldots, n+1,$$

che impongono il passaggio di $\tilde{f}$ per i punti (x_i, y_i).
Le suddette equazioni sono dette **condizioni di interpolazione** e i punti x_i sono detti **nodi di interpolazione**; inoltre, si dice che $\tilde{f}$ **interpola i dati** (x_i, y_i) oppure, se $y_i = f(x_i)$, **interpola la funzione** f nei punti x_i.

In seguito verranno considerate:

- funzioni interpolanti $\tilde{f}(x)$ **polinomiali**, generalmente utilizzate per lo sviluppo di algoritmi nella differenziazione, nell'integrazione e nella soluzione di problemi differenziali;
- funzioni interpolanti $\tilde{f}(x)$ **polinomiali a tratti**, largamente utilizzate nella grafica e nella soluzione di problemi differenziali.

3.1 Interpolazione mediante polinomi

3.1.1 Esistenza, unicità e costruzione del polinomio interpolante

Teorema 3.1.1. Assegnati $n+1$ punti (x_i, y_i), con le ascisse distinte tra loro ($x_i \neq x_j$ per $i \neq j$), esiste uno e un sol polinomio $p_n(x)$ di grado minore o uguale a n interpolante i dati assegnati, ovvero soddisfacente le condizioni di interpolazione:

$$p_n(x_i) = y_i, \qquad i = 1, \ldots, n+1.$$

Dimostrazione. Consideriamo il generico polinomio di grado n

$$p_n(x) = c_1 x^n + c_2 x^{n-1} + \ldots + c_n x + c_{n+1},$$

definito dagli $n+1$ coefficienti $c_1, \ldots, c_{n+1}$ e imponiamo le condizioni di interpolazione:

$$c_1 x_i^n + c_2 x_i^{n-1} + \ldots + c_n x_i + c_{n+1} = y_i, \qquad i = 1, \ldots, n+1.$$

Otteniamo così il sistema lineare

$$\begin{pmatrix} x_1^n & \cdots & x_1 & 1 \\ x_2^n & \cdots & x_2 & 1 \\ \vdots & \vdots & \vdots & \vdots \\ x_{n+1}^n & \cdots & x_{n+1} & 1 \end{pmatrix} \begin{pmatrix} c_1 \\ c_2 \\ \vdots \\ c_{n+1} \end{pmatrix} = \begin{pmatrix} y_1 \\ y_2 \\ \vdots \\ y_{n+1} \end{pmatrix} \iff \mathbf{V}_n \mathbf{c} = \mathbf{y}$$

la cui matrice dei coefficienti $\mathbf{V}_n$ è la matrice di Vandermonde associata al vettore $\mathbf{x} = (x_1, x_2, \ldots, x_{n+1})^T$. Poiché

$$|\det(\mathbf{V}_n)| = \prod_{i<j} |x_i - x_j|$$

e i nodi x_i sono distinti, la matrice $\mathbf{V}_n$ è non singolare e dunque

$$\exists!\, \mathbf{c}^* = (c_1^*, \ldots, c_{n+1}^*)^T : \quad \mathbf{V}_n \mathbf{c}^* = \mathbf{y}.$$

□

Rappresentazione monomiale

In effetti cercare un polinomio vuol dire essenzialmente cercare una base $\{p_1, \ldots, p_n\}$ di $\mathbb{P}^n = \{\text{polinomi di grado al massimo n}\}$, che è uno spazio vettoriale di dimensione $n+1$, e trovare le cooordinate del polinimio rispetto a tale base.
La base più facile e intuitiva è quella dei monomi monici $\{1, x, x^2, \ldots, x^n\}$.

Definizione 3.1.2. Si definisce **rappresentazione monomiale** del polinomio interpolante $p_n(x)$ l'espressione:

$$p_n(x) = c_1 x^n + c_2 x^{n-1} + \ldots + c_n x + c_{n+1},$$

ove i coefficienti c_i, $i = 1, \ldots, n+1$, sono determinati imponendo che $p_n(x)$ soddisfi le condizioni di interpolazione.

I seguenti comandi MATLAB consentono di ottenere i coefficienti della rappresentazione monomiale e di valutare il polinomio interpolante in qualsiasi punto o vettore di punti assegnato.

- `c = polyfit(x,y,n)` calcola e memorizza in `c`, i coefficienti della rappresentazione monomiale del polinomio di grado n,

  $$p_n(x) = c(1)x^n + c(2)x^{n-1} + \ldots + c(n)x + c(n+1),$$

  interpolante i dati (x_i, y_i), $i = 1, \ldots, n+1$, le cui ascisse x_i e ordinate y_i sono memorizzate nei vettori `x` e `y`, rispettivamente.

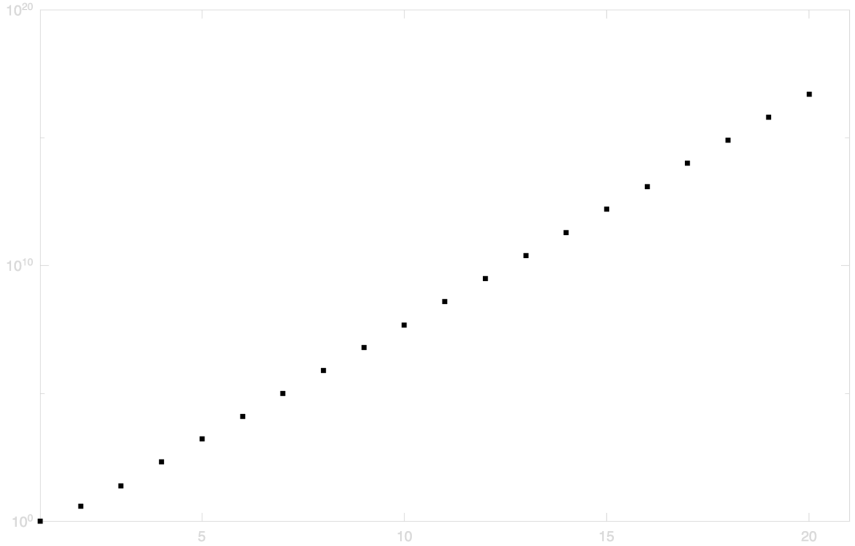
- `p = polyval(c,z)` calcola e memorizza nel vettore `p`, i valori che un **qualsiasi** polinomio

  $$p_n(x) = c(1)x^n + c(2)x^{n-1} + \ldots + c(n)x + c(n+1),$$

  i cui coefficienti sono memorizzati in `c`, assume nelle componenti del vettore `z`. I coefficienti devono essere ordinati dal termine di grado massimo al termine di grado zero.

Tuttavia la determinazione del polinomio interpolante mediante la sua rappresentazione monomiale comporta la risoluzione di un problema mal condizionato, ovvero la risoluzione di un sistema lineare $\mathbf{V}_n\mathbf{c} = \mathbf{y}$ la cui matrice dei coefficienti è una matrice di Vandermonde.
In figura è riportato il numero di condizionamento K_∞ (in norma infinito) della matrice di Vandermonde generata dal vettore $\mathbf{x}$ di $n = 2, \dots, 20$.
Si nota che il condizionamento della matrice peggiora al crescere dell'ordine n e, per questo motivo, non è possibile utilizzare la rappresentazione monomiale del polinomio interpolante quando n è grande.

Figura: $K_\infty(\mathbf{V}_n)$ per $n = 2, 3, \dots, 20$.



Pertanto, quando n è grande, è necessario calcolare il polinomio interpolante attraverso rappresentazioni alternative.

Rappresentazione di Lagrange

Sia $V = \mathbb{R}^n$ e supponiamo di voler trovare le coordinate di $X = (x_1, \dots, x_n)$ rispetto alla base canonica $\{e_1, \dots, e_n\}$ di $\mathbb{R}^n$. Questo ha costo computazionale nullo in quanto x_i è già la coordinata di X su e_i. Possiamo formalizzare questo fatto definendo i funzionali

$$\varphi_i : \mathbb{R}^n \longrightarrow \mathbb{R}, \qquad X = (x_1, \dots, x_n) \longmapsto x_i.$$

L'insieme $\{\varphi_1, \dots, \varphi_n\}$ costituisce un sistema di riferimento per $\mathbb{R}^n$ ovvero una base del duale algebrico $(\mathbb{R}^n)^*$. Tale base è proprio la base duale di $\{e_1, \dots, e_n\}$ infatti $\varphi_j(e_i) = \delta_{ij}$.
Consideriamo ora un insieme di coppie di numeri reali $\{(x_1, y_1), \dots, (x_{n+1}, y_{n+1})\}$. Vorremmo trovare una base di $\mathbb{P}^n$ rispetto alla quale il polinomio che vale y_i in x_i, $\forall i = 1, \dots, n+1$, abbia coordinate $(y_1, \dots, y_n)$.
Definiamo allora le applicazioni

$$\psi_i : \mathbb{P}^n \longrightarrow \mathbb{R}, \qquad p \longmapsto p(x_i), \qquad \forall i = 1, \dots, n+1.$$

Come nel caso euclideo $\beta^* = \{\psi_1, \dots, \psi_{n+1}\}$ formano un sistema di riferimento per $\mathbb{P}^n$ ovvero formano una base del duale $(\mathbb{P}^n)^*$. Trovando quindi la base primale β di $\mathbb{P}^n$ associata a β^* otteniamo la caratteristica cercata. Sia ℓ_j il generico elemento di β. Questo deve soddisfare

$$\psi_i(\ell_j) = \delta_{ij} \iff \ell_j(x_i) = \delta_{ij} \iff \ell_j(x_i) = 0 \quad \forall i \neq j, \qquad \ell_j(x_j) = 1 \quad i = j.$$

Per il teorema di esistenza ed unicità

$$\ell_j(x) = C \prod_{\substack{i=1 \\ i \neq j}}^{n+1} (x - x_i).$$

Calcoliamo C in modo tale che $\ell_j(x_j) = 1$.

$$1 = \ell_j(x_j) = C \prod_{\substack{i=1 \\ j \neq i}}^{n+1} (x_j - x_i) \implies C = \frac{1}{\prod_{\substack{i=1 \\ j \neq i}}^{n+1} (x_j - x_i)} \implies \ell_j(x) = \frac{\prod_{\substack{i=1 \\ i \neq j}}^{n+1} (x - x_i)}{\prod_{\substack{i=1 \\ j \neq i}}^{n+1} (x_j - x_i)}.$$

Definizione 3.1.3. Si definisce **rappresentazione di Lagrange** del polinomio p_n interpolante i dati (x_i, y_i), $i = 1, \ldots, n+1$ con $x_i \neq x_j$ per $i \neq j$, l'espressione

$$p_n(x) = \sum_{j=1}^{n+1} y_j \ell_j(x)$$

I polinomi

$$\ell_j(x) = \frac{(x - x_1) \cdots (x - x_{j-1})(x - x_{j+1}) \cdots (x - x_{n+1})}{(x_j - x_1) \cdots (x_j - x_{j-1})(x_j - x_{j+1}) \cdots (x_j - x_{n+1})}$$

$j = 1, \ldots, n+1$, di grado n sono detti **polinomi fondamentali di Lagrange** associati ai nodi x_i.

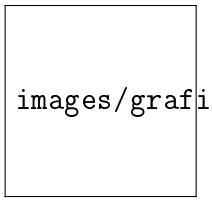
Esempio 3.1.4. Scriviamo la rappresentazione di Lagrange del polinomio $p_3(x)$ interpolante i dati $(0, 1)$, $(1, -1)$, $(2, 1)$, $(1/2, 2)$, precedentemente considerati.
Per i polinomi $\ell_j(x)$, $j = 1, 2, 3, 4$, si ha:

$$\ell_1(x) = \frac{(x-1)(x-2)(x-1/2)}{(0-1)(0-2)(0-1/2)}$$

$$\ell_2(x) = \frac{(x-0)(x-2)(x-1/2)}{(1-0)(1-2)(1-1/2)}$$

$$\ell_3(x) = \frac{(x-0)(x-1)(x-1/2)}{(2-0)(2-1)(2-1/2)}$$

$$\ell_4(x) = \frac{(x-0)(x-1)(x-2)}{(1/2-0)(1/2-1)(1/2-2)}$$



Quindi la rappresentazione di Lagrange del polinomio $p_3(x)$ interpolante è

$$p_3(x) = 1 \cdot \ell_1(x) - 1 \cdot \ell_2(x) + 1 \cdot \ell_3(x) + 2 \cdot \ell_4(x)$$

Osservazione:

- La rappresentazione di Lagrange del polinomio di interpolazione è interessante perché rappresenta la base per la costruzione di formule di integrazione numerica e di metodi per l'approssimazione della soluzione di problemi differenziali.
- Dal punto di vista del calcolo numerico del polinomio, la rappresentazione di Lagrange del polinomio interpolante è di interesse pratico solo per un numero piccolo di dati di interpolazione oppure quando i nodi x_i sono fissati e si calcolano i polinomi di interpolazione corrispondenti a diversi insiemi di valori $\{y_i\}$. Infatti, poiché ciascun polinomio $\ell_j(x)$ dipende da tutti i nodi $x_1, \ldots, x_{n+1}$, se si desidera aggiungere un dato di interpolazione (x_{n+2}, y_{n+2}) all'insieme dei dati $\{(x_i, y_i)\}_{i=1,\ldots,n+1}$, occorre ricalcolare tutti i polinomi $\ell_j(x)$. In generale il costo computazionale per il calcolo degli $l_j(x)$ è $O(n^2)$ ma se supponiamo di avere un flusso di dati incrementale per il quale ci viene fornita una coppia (x_i, y_i) per volta possiamo ricalcolare i nuovi polinimi dai vecchi con un costo di $O(n)$. Il vero problema sta nella rivalutazione del nuovo polinomio interpolante che ha un costo di $O(n^2)$ per valutazione.
- Per il calcolo effettivo del polinomio interpolante si utilizza una rappresentazione, alternativa a quella monomiale e di Lagrange, nota sotto il nome di rappresentazione di Newton.

Differenze divise e rappresentazione di Newton

Il teorema di Lagrange fornisce un legame tra una quantità costruita a partire dai valori di una funzione in due punti e una proprietà differenziale della funzione stessa. Siano $x_1 < x_2$ e sia f continua in $[x_1, x_2]$ e derivabile in (x_1, x_2). Il rapporto incrementale

$$\frac{f(x_2) - f(x_1)}{x_2 - x_1} \tag{3.1}$$

misura la variazione media di f per unità di variazione della variabile indipendente. Il teorema di Lagrange assicura che questa pendenza media coincide con la derivata prima in almeno un punto intermedio $\xi \in (x_1, x_2)$:

$$\frac{f(x_2) - f(x_1)}{x_2 - x_1} = f'(\xi). \tag{3.2}$$

Ci si può chiedere se sia possibile ripetere questa costruzione per descrivere non soltanto la pendenza, ma anche la variazione della pendenza e, più in generale, il comportamento delle derivate successive. Per rispondere, si introducono le *differenze divise*, definite ricorsivamente a partire dai valori della funzione.

Definizione 3.1.5. Siano $x_1, \dots, x_{n+1}$ nodi distinti e sia f definita in tali punti. Le **differenze divise** di f sono definite ponendo

$$f[x_i] = f(x_i) \tag{3.3}$$

e, per ogni $k \geq 1$,

$$f[x_i, \dots, x_{i+k}] = \frac{f[x_{i+1}, \dots, x_{i+k}] - f[x_i, \dots, x_{i+k-1}]}{x_{i+k} - x_i}. \tag{3.4}$$

La differenza divisa $f[x_i, \dots, x_{i+k}]$ è detta **di ordine** k.

Osservazione: Le differenze divise sono invarianti rispetto a permutazioni degli argomenti, cioè

$$f[x_1, x_2] = f[x_2, x_1],$$
$$f[x_1, x_2, x_3] = f[x_2, x_1, x_3] = f[x_3, x_2, x_1], \dots$$

Prima di ricavare la relazione tra la differenza divisa di ordine k e la $k-esima$ derivata di f concentriamoci sul motivo per il quale le abbiamo introdotte, ovvero ricavare una nuova base per il polinomio interpolante che superi i limiti della base dei polinomi di Lagrange.
Voglio interpolare le coppie $(x_1, f(x_1)), (x_2, f(x_2)), \dots, (x_{n+1}, f(x_{n+1}))$.
È facile verificare che il seguente polinomio di grado 0

$$p_0(x) = f(x_1)$$

interpola $f(x)$ in x_1 e il polinomio di grado 1

$$\begin{aligned} p_1(x) &= \underbrace{f(x_1)}_{p_0(x)} + f[x_1, x_2](x - x_1) \\ &= f(x_1) + \frac{f(x_2) - f(x_1)}{x_2 - x_1}(x - x_1) \end{aligned}$$

interpola $f(x)$ in x_1 e in x_2.
Così proseguendo, denotato con $p_{n-1}(x)$ il polinomio di grado $n-1$ interpolante $f(x)$ in $x_1, \dots, x_n$, il polinomio di grado n

$$p_n(x) = \underbrace{p_{n-1}(x)}_{\substack{\text{interpola le prime} \\ n \text{ coppie}}} + f[x_1, x_2, \dots, x_{n+1}] \underbrace{(x - x_1) \cdots (x - x_n)}_{\text{si annulla nei primi } n \text{ nodi}}$$

interpola $f(x)$ in $x_1, \dots, x_{n+1}$.
Dall'unicità del polinomio, di grado n e interpolante $n+1$ punti, segue allora che quest'ultima espressione del polinomio p_n è una terza rappresentazione del polinomio interpolante (già, precedentemente definito, nelle rappresentazioni monomiale e di Lagrange).

Definizione 3.1.6. Si definisce **rappresentazione di Newton** del polinomio interpolante i dati (x_i, y_i) con $y_i = f(x_i)$, la seguente espressione

$$\begin{aligned} p_n(x) = f(x_1) &+ f[x_1, x_2](x - x_1) \\ &+ f[x_1, x_2, x_3](x - x_1)(x - x_2) \\ &+ \dots + f[x_1, x_2, \dots, x_{n+1}](x - x_1) \cdots (x - x_n) \end{aligned}$$

I polinomi 1, $x - x_1$, …, $(x - x_1) \cdots (x - x_n)$ sono detti **polinomi fondamentali di Newton**.

Complementi 3.1.7.

Teorema 3.1.8 (Lagrange generalizzato per le differenze divise)**.** Sia $f \in C^n([a, b])$, con $n \geq 1$, e siano

$$a \leq x_1 < x_2 < \cdots < x_{n+1} \leq b.$$

Allora esiste almeno un punto $\xi \in (x_1, x_{n+1})$ tale che

$$\boxed{f[x_1, \ldots, x_{n+1}] = \frac{f^{(n)}(\xi)}{n!}.} \tag{3.5}$$

Dimostrazione. Sia p_n il polinomio di grado al più n che interpola f nei nodi $x_1, \ldots, x_{n+1}$. Nella rappresentazione di Newton esso si scrive

$$p_n(x) = \sum_{k=0}^{n} f[x_1, \ldots, x_{k+1}] \prod_{j=1}^{k} (x - x_j), \tag{3.6}$$

dove, per $k = 0$, il prodotto è convenzionalmente uguale a 1. Tutti i termini della somma, a eccezione dell'ultimo, hanno grado strettamente minore di n. Poiché il prodotto $\prod_{j=1}^{n}(x - x_j)$ è un polinomio monico di grado n, il coefficiente del termine di grado massimo di p_n è precisamente $f[x_1, \ldots, x_{n+1}]$. Derivando n volte si ottiene quindi

$$p_n^{(n)}(x) = n!\, f[x_1, \ldots, x_{n+1}]. \tag{3.7}$$

Consideriamo la funzione $g(x) = f(x) - p_n(x)$. Dalle condizioni di interpolazione segue che

$$g(x_1) = g(x_2) = \cdots = g(x_{n+1}) = 0.$$

La funzione g possiede dunque almeno $n + 1$ zeri distinti. Applicando il teorema di Rolle a ciascuno dei n intervalli compresi tra due zeri consecutivi, g' possiede almeno n zeri. Ripetendo il ragionamento su g', poi su g'' e così via, dopo n applicazioni successive del teorema di Rolle si trova un punto $\xi \in (x_1, x_{n+1})$ per il quale

$$g^{(n)}(\xi) = 0.$$

Di conseguenza,

$$0 = f^{(n)}(\xi) - p_n^{(n)}(\xi) = f^{(n)}(\xi) - n!\, f[x_1, \ldots, x_{n+1}],$$

da cui segue (3.5). □

Il teorema chiarisce la natura della differenza divisa di ordine n: essa è un valore della derivata n-esima, assunto in un punto dell'intervallo individuato dai nodi, diviso per $n!$. In particolare,

$$n!\, f[x_1, \ldots, x_{n+1}] = f^{(n)}(\xi). \tag{3.8}$$

Per $n = 1$ si ritrova il teorema di Lagrange ordinario; per $n = 2$ si ottiene $2\, f[x_1, x_2, x_3] = f''(\xi)$. Fino a questo punto i nodi sono stati assunti distinti. Studiamo ora ciò che accade quando essi si avvicinano tutti a uno stesso punto a. Sia $f \in C^n$ in un intorno di a e consideriamo, per ogni parametro $\varepsilon > 0$, $n + 1$ nodi distinti $x_1(\varepsilon), \ldots, x_{n+1}(\varepsilon)$ tali che

$$x_j(\varepsilon) \longrightarrow a \qquad \text{per } \varepsilon \longrightarrow 0, \quad j = 1, \ldots, n + 1. \tag{3.9}$$

Applicando il teorema di Lagrange generalizzato ai primi $k+1$ nodi, per ogni $k = 1, \dots, n$ esiste un punto $\xi_{k,\varepsilon}$ compreso tra il minimo e il massimo di tali nodi per cui

$$f[x_1(\varepsilon), \dots, x_{k+1}(\varepsilon)] = \frac{f^{(k)}(\xi_{k,\varepsilon})}{k!}. \tag{3.10}$$

Poiché tutti i nodi convergono ad a, anche ogni punto $\xi_{k,\varepsilon}$, essendo compreso fra essi, deve convergere ad a. Per continuità di $f^{(k)}$ si conclude che

$$\boxed{\lim_{\varepsilon \to 0} f[x_1(\varepsilon), \dots, x_{k+1}(\varepsilon)] = \frac{f^{(k)}(a)}{k!}.} \tag{3.11}$$

La stessa relazione vale per $k = 0$, interpretando $f[x_1] = f(x_1)$.
Il risultato fornisce una spiegazione del ruolo dei coefficienti di Taylor: essi sono precisamente i limiti delle differenze divise quando i nodi confluiscono nel centro dello sviluppo. In particolare, si può estendere per continuità la notazione delle differenze divise anche al caso di nodi coincidenti, ponendo

$$\boxed{f[\underbrace{a, \dots, a}_{k+1 \text{ volte}}] = \frac{f^{(k)}(a)}{k!}.} \tag{3.12}$$

Il legame appena stabilito diventa particolarmente trasparente nella rappresentazione di Newton. Per i nodi $x_1(\varepsilon), \dots, x_{n+1}(\varepsilon)$, il polinomio interpolante è

$$p_{n,\varepsilon}(x) = \sum_{k=0}^{n} f[x_1(\varepsilon), \dots, x_{k+1}(\varepsilon)] \prod_{j=1}^{k} \big(x - x_j(\varepsilon)\big). \tag{3.13}$$

Quando $\varepsilon \to 0$, il coefficiente di ciascun termine converge, in virtù di (3.11), a $f^{(k)}(a)/k!$. Contemporaneamente, per ogni x fissato,

$$\prod_{j=1}^{k} \big(x - x_j(\varepsilon)\big) \longrightarrow (x-a)^k. \tag{3.14}$$

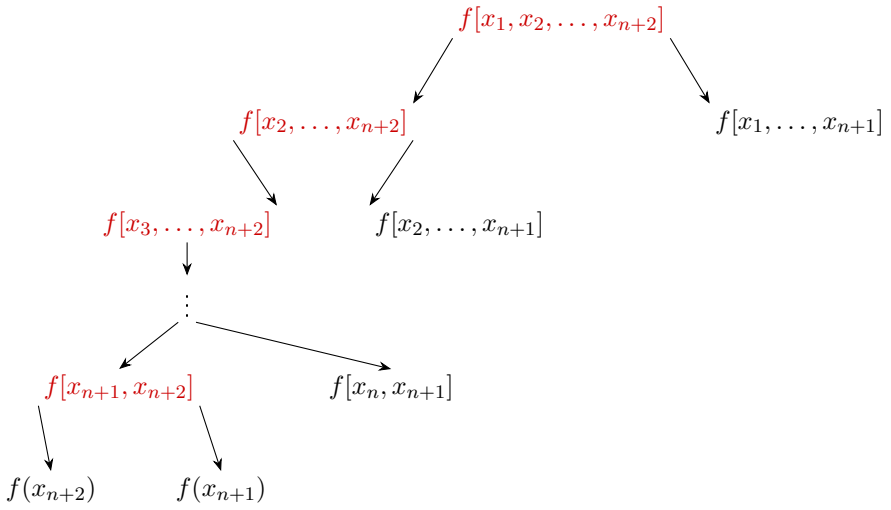
Il numero di addendi in (3.13) è finito, quindi è possibile passare al limite termine a termine e ottenere

$$\begin{aligned} \lim_{\varepsilon \to 0} p_{n,\varepsilon}(x) &= \sum_{k=0}^{n} \frac{f^{(k)}(a)}{k!} (x-a)^k \\ &= T_{n,a}(x). \end{aligned} \tag{3.15}$$

Il polinomio $T_{n,a}$ è il **polinomio di Taylor di ordine n centrato in a**.
Da questo punto di vista l'interpolazione di Newton e lo sviluppo di Taylor sono due manifestazioni della stessa costruzione. Nel primo caso si impongono i valori della funzione in nodi distinti; nel secondo, facendo coincidere progressivamente i nodi, le condizioni sui valori si trasformano in condizioni sulle derivate nello stesso punto. Lo sviluppo di Taylor può così essere interpretato come il limite dell'interpolazione polinomiale quando tutti i nodi di interpolazione confluiscono nel centro a.
Supponiamo di nuovo di ricevere dei dati $\{(x_i, y_i)\}$ uno alla volta e di voler calcolare volta per volta il polinomio interpolante. Abbiamo visto che dato il polinomio $p_n(x)$ che interpola $\{(x_1, y_1), \dots, (x_{n+1}, y_{n+1})\}$ il polinomio che interpola $\{(x_1, y_1), \dots, (x_{n+2}, y_{n+2})\}$ è

$$p_{n+1}(x) = p_n(x) + f[x_1, x_2, \dots, x_{n+2}](x - x_1) \cdots (x - x_{n+1}).$$

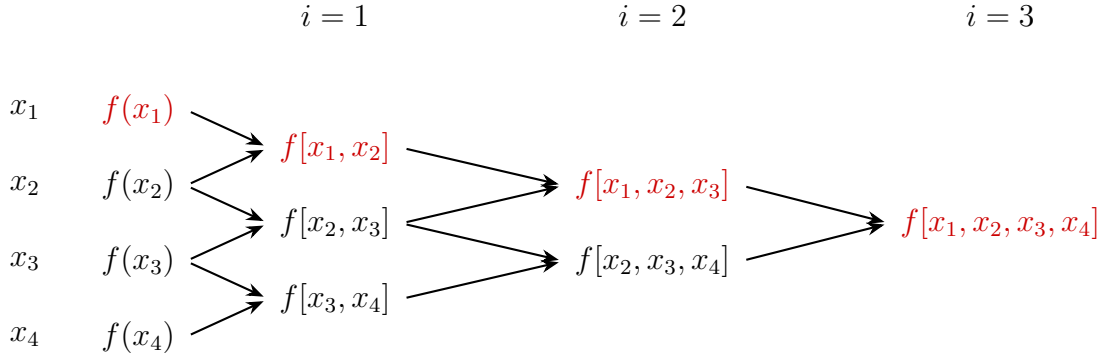
E dobbiamo quindi solo effettuare il calcolo di $f[x_1, x_2, \dots, x_{n+2}]$. Essendo questa una procedura ricorsiva possiamo costruire un albero di "calcolo"



I nodi rossi vanno effettivamente calcolati mentre quelli neri sono già stati calcolati perché già utilizzati in $p_n(x)$. Il costo di aggiornamento è quindi circa $O(n+1)$. Fino a qui abbiamo rilevato lo stesso costo della rappresentazione tramite polinomi di Lagrange. Calcoliamo però il costo di valutazione. Supponiamo di volere valutare $p_{n+1}(x)$ in x^* avendo già $p_n(x^*)$. Allora

$$p_{n+1}(x^*) = \underbrace{p_n(x^*)}_{\text{già noto}} + f[x_1, \dots, x_{n+2}] \underbrace{(x^* - x_1) \cdots (x^* - x_{n+1})}_{\text{da calcolare}}.$$

Ovvero il numero di operazioni da effettuare è $O(n+1)$ contro le $O(n^2)$ richieste dalla rappresentazione di Lagrange richieste per lo stesso task.
Più in generale chiediamoci quale sia il costo computazionale di calcolo e valutazione del polinomio interpolante costruito tramite le differenze divise senza presupporre di aver già precalcolato alcun che. È sufficiente calcolare la diagonale del seguente triangolo. Per comodità prendiamo $n+1 = 4$.



Inizializziamo quindi $x = (x_1, \dots, x_{n+1})^T$ e $y = (f(x_1), \dots, f(x_{n+1}))^T$. Partendo dall'ordine $i = 1$, calcoliamo le differenze divise in modo ricorsivo, tenendo solo quelle che ci servono. Nello specifico, vogliamo creare la successione

$$y^{(0)} = \begin{pmatrix} f(x_1) \\ \vdots \\ f(x_{n+1}) \end{pmatrix}, \qquad y^{(1)} = \begin{pmatrix} f(x_1) \\ f[x_1, x_2] \\ \vdots \\ f[x_n, x_{n+1}] \end{pmatrix},$$

$$y^{(2)} = \begin{pmatrix} f(x_1) \\ f[x_1, x_2] \\ f[x_1, x_2, x_3] \\ \vdots \\ f[x_{n-1}, x_n, x_{n+1}] \end{pmatrix}, \qquad \ldots, \qquad y^{(n)} = \begin{pmatrix} f(x_1) \\ f[x_1, x_2] \\ f[x_1, x_2, x_3] \\ \vdots \\ f[x_1, \ldots, x_n] \\ f[x_1, \ldots, x_{n+1}] \end{pmatrix}.$$

In MATLAB tale approccio è implementato dal seguente codice.

```
1 function a = difdiv(x,y)
2     n = length(x)-1;
3     for i = 1:n
4         j = n+1:-1:i+1;
5         y(j) = (y(j)-y(j-1))./(x(j)-x(j-i));
6     end
7     a = y;
8 end
```

Osservazione: Notiamo che, per la disposizione degli elementi in $y^{(k)}$, la formula di riga 5 corrisponde a calcolare le differenze finite di ordine $k+2$ e a posizionarle nelle entrate $i+1 : n+1$ di $y^{(k+1)}$. Inoltre, il termine $x(j-i)$ garantisce che la differenza a denominatore sia corretta.
Il costo computazionale è dell'ordine di $O(n^2)$. Se avessimo usato l'algoritmo ricorsivo banale

```
1 function a = difdiv_banale(x)
2     a = (difdiv_banale[x(2:end)] - ...
3          difdiv_banale(x(1:length(x)-1))) / ...
4          (x(length(x))-x(1))
5 end
```

avremmo avuto un costo di $O(2^n)$, corrispondente a percorrere l'intero grafo di ricorsione, ricalcolando più volte gli stessi coefficienti.
Concentriamoci ora su come costruire un algoritmo efficiente per la valutazione del polinomio interpolante scritto nella base di Newton. Partiamo dal polinomio interpolante di grado 3 nei punti $(x_1, f(x_1))$, $(x_2, f(x_2))$, $(x_3, f(x_3))$, $(x_4, f(x_4))$. Indichiamo con a_i, $i = 1, \ldots, 4$, la differenza finita di ordine $i-1$ presente nel polinomio. Allora

$$\begin{aligned} p(z) &= a_1 + a_2(z - x_1) + a_3(z - x_1)(z - x_2) \\ &\quad + a_4(z - x_1)(z - x_2)(z - x_3) \\ &= \big[a_4(z - x_3) + a_3\big](x - z_1)(x - z_2) + a_2(z - x_1) + a_1 \\ &= \big\{[a_4(z - x_3) + a_3](z - x_2) + a_2\big\}(z - x_1) + a_1. \end{aligned}$$

Possiamo quindi procedere con il seguente schema di calcolo:

$$\begin{aligned} p^{(4)} &= a_4 \\ &\downarrow \\ p^{(3)} &= p^{(4)}(z - x_3) + a_3 \\ &\downarrow \\ p^{(2)} &= p^{(3)}(z - x_2) + a_2 \\ &\downarrow \\ p^{(1)} &= p^{(2)}(z - x_1) + a_1 = p(z). \end{aligned}$$

Più in generale, per il polinomio interpolante di grado n in $n+1$ punti utilizziamo

$$\begin{cases} p^{(n+1)} = a_{n+1}, \\ p^{(k)} = p^{(k+1)} \cdot (z - x_k) + a_k, \quad k = n, \dots, 1. \end{cases}$$

Tale algoritmo è noto come **algoritmo di Horner**. Vengono effettuate $n+1$ moltiplicazioni e $2(n+1)$ addizioni. L'algoritmo ha dunque complessità $O(n)$.

```
function p = interp(x,a,z)
    n = length(x)-1;
    p = a(n+1)*ones(1,length(z));
    for k = n:-1:1
        p = p.*(z-x(k))+a(k);
    end
```

3.1.2 Condizionamento del polinomio di intepolazione

Nel caso del polinomio di intepolazione costruito tramite la base di monomi monici il condizionamento è quello della matrice di Vandermonde ovvero, come abbiamo già visto, è un problema fortemente mal condizionato. In questa sezione vediamo come calcolare il condizionamento del polinimio intepolante scritto in base dei polinomi di Lagrange.
Per studiare il condizionamento del problema di interpolazione polinomiale introduciamo la seguente norma di funzione.

Definizione 3.1.9. Sia $g(x)$ una funzione continua in $[a,b]$. Si definisce **norma uniforme** o **norma del massimo** della funzione $g(x)$ la quantità

$$\|g\|_\infty = \max_{x \in [a,b]} |g(x)|.$$

Esaminiamo quindi la reazione del polinomio interpolante $p_n(x)$ alla presenza di perturbazioni nelle ordinate $f(x_j)$, $j = 1, \dots, n+1$.
Introdotti i valori perturbati $\bar{f}_j = f(x_j) + \epsilon_j$, $j = 1, \dots, n+1$, denotiamo con

$$\bar{p}_n(x) = \sum_{j=1}^{n+1} \bar{f}_j \ell_j(x), \qquad x \in [a,b],$$

il polinomio interpolante i dati perturbati, ricordando che

$$p_n(x) = \sum_{j=1}^{n+1} f(x_j) \ell_j(x).$$

Si ha allora

$$p_n(x) - \bar{p}_n(x) = \sum_{j=1}^{n+1} \overbrace{\left(f(x_j) - \bar{f}_j\right)}^{\epsilon_j} \ell_j(x),$$

da cui

$$\begin{aligned} |p_n(x) - \bar{p}_n(x)| &\le \sum_{j=1}^{n+1} |f(x_j) - \bar{f}_j| \, |\ell_j(x)| \\ &\le \max_{1 \le j \le n+1} |\epsilon_j| \sum_{j=1}^{n+1} |\ell_j(x)|. \end{aligned}$$

Denotando con

$$\Lambda_n = \left\| \sum_{j=1}^{n+1} |\ell_j| \right\|_\infty$$

e con $\boldsymbol{\epsilon} = (\epsilon_1, \epsilon_2, \ldots, \epsilon_{n+1})^T$, si ottiene

$$|p_n(x) - \bar{p}_n(x)| \leq \Lambda_n \underbrace{\|\boldsymbol{\epsilon}\|_\infty}_{\text{max perturbazione}} \qquad \forall x \in [a,b]$$

e quindi

$$\|p_n - \bar{p}_n\|_\infty \leq \Lambda_n \|\boldsymbol{\epsilon}\|_\infty.$$

Poiché

$$\begin{aligned} \|p_n\|_\infty &= \max_{x\in[a,b]} |p_n(x)| \\ &\geq \max_{1\leq j\leq n+1} |p_n(x_j)| \\ &= \max_{1\leq j\leq n+1} |f(x_j)| \\ &= \|\mathbf{f}\|_\infty, \end{aligned}$$

con $\mathbf{f} = (f(x_1), f(x_2), \ldots, f(x_{n+1}))^T$, possiamo scrivere

$$\frac{\|p_n - \bar{p}_n\|_\infty}{\|p_n\|_\infty} \leq \Lambda_n \frac{\|\boldsymbol{\epsilon}\|_\infty}{\|p_n\|_\infty}$$

e dunque

$$\boxed{\frac{\|p_n - \bar{p}_n\|_\infty}{\|p_n\|_\infty} \leq \underbrace{\Lambda_n}_{\substack{\text{condizionamento} \\ \text{del problema}}} \frac{\|\boldsymbol{\epsilon}\|_\infty}{\|\mathbf{f}\|_\infty}}.$$

Definizione 3.1.10. Il numero Λ_n è detto **costante di Lebesgue** associata ai nodi $x_1, x_2, \ldots, x_{n+1}$ e all'intervallo $[a,b]$.

Osserviamo che la costante di Lebesgue rappresenta una maggiorazione del coefficiente di amplificazione degli errori ϵ_j, $j = 1, \ldots, n+1$, nei dati sul risultato.
Si dimostra che, **qualunque sia la scelta dei nodi di interpolazione**,

$$\lim_{n\to\infty} \Lambda_n = +\infty.$$

Nel caso "ottimale" l'ordine di divergenza è di tipo logaritmico.
In particolare, per le seguenti scelte di nodi, tutte appartenenti al generico intervallo $[a,b]$, si hanno le seguenti costanti di Lebesgue.
Per i **nodi di Chebyshev**

$$x_i = \frac{b-a}{2} \cos\left(\frac{(2i-1)\pi}{2(n+1)}\right) + \frac{b+a}{2}, \qquad i = 1, \ldots, n+1,$$

e per i **nodi di Chebyshev-Lobatto**

$$x_i = \frac{b-a}{2} \cos\left(\frac{(i-1)\pi}{n}\right) + \frac{b+a}{2}, \qquad i = 1, \ldots, n+1,$$

si ha

$$\Lambda_n = O(\log n) \qquad \text{per } n \to \infty.$$

Per i **nodi equispaziati**:

$$x_i = a + (i-1)\frac{b-a}{n}, \qquad i = 1, \dots, n+1.$$

Si ha

$$\Lambda_n = O(e^n) \qquad \text{per } n \to \infty.$$

3.1.3 Convergenza del polinomio di interpolazione

Definizione 3.1.11. Denotato con $p_n(x)$ il polinomio interpolante una funzione continua $f(x)$ nei punti $x_1, \dots, x_{n+1}$, si definisce **errore di interpolazione** la funzione

$$E_n(x) = f(x) - p_n(x).$$

Si dice che la successione di polinomi interpolanti $\{p_n\}_{n\in\mathbb{N}}$ **converge uniformemente** a f se, e solo se,

$$\lim_{n\to\infty} \|E_n\|_\infty = 0.$$

Osservazione: La successione $\{p_n\}_{n\in\mathbb{N}}$ La successione è definita dalla scelta dei nodi di interpolazione.

Nell'ipotesi $f(x) \in C^{n+1}([a,b])$ è possibile ricavare un'espressione per l'errore di interpolazione $E_n(x)$. Vale infatti il seguente teorema.

Teorema 3.1.12. Assegnati $n+1$ punti (x_i, y_i), con le ascisse distinte tra loro e tutte appartenenti ad $[a,b]$ e $y_i = f(x_i)$ per $i = 1, \dots, n+1$, se $f(x) \in C^{n+1}([a,b])$, allora per ogni fissato x esiste $\xi \in (a,b)$ tale che

$$E_n(x) = \frac{f^{(n+1)}(\xi)}{(n+1)!}\omega_{n+1}(x),$$

con

$$\omega_{n+1}(x) = (x - x_1) \cdot (x - x_2) \cdot \ldots \cdot (x - x_{n+1}).$$

Notiamo che ω_{n+1} è un polinomio che si annulla.

Dimostrazione. Scriviamo

$$E_n(x) = f(x) - p_n(x) = R_n(x)\omega_{n+1}(x).$$

dove $R_n(x)$ è un coefficiente incognito che dobbiamo determinare.
Introduciamo la funzione ausiliaria, fissato $x \neq x_i$,

$$G(t) = \underbrace{f(t) - p_n(t)}_{E_n(t)} - R_n(x)\omega_{n+1}(t), \qquad t \in [a,b],$$

Ricordando che $p_n(x_i) = f(x_i)$, $i = 1, \dots, n+1$, poiché $G(t) \in C^{n+1}([a,b])$, $G(x_i) = 0$, per $i = 1, \dots, n+1$, e $G(x) = 0$, possiamo applicare il teorema di Rolle e affermare che esistono $n+1$ punti distinti

$\xi_1^{(1)}, \xi_2^{(1)}, \ldots, \xi_{n+1}^{(1)}$ in (a,b), nei quali

$$G^{(1)}\left(\xi_i^{(1)}\right) = 0, \qquad i = 1, \ldots, n+1.$$

Applicando nuovamente il teorema di Rolle alla funzione $G^{(1)}(x) \in C^n([a,b])$, possiamo affermare che esistono n punti distinti $\xi_1^{(2)}, \xi_2^{(2)}, \ldots, \xi_n^{(2)}$ in (a,b), nei quali

$$G^{(2)}\left(\xi_i^{(2)}\right) = 0, \qquad i = 1, \ldots, n.$$

Proseguendo allo stesso modo con le derivate successive, sino alla $(n+1)$-esima, possiamo affermare che esiste $\xi := \xi_1^{(n+1)}$ in (a,b) per il quale si ha

$$G^{(n+1)}(\xi) = 0.$$

Tenendo infine conto che $\omega_{n+1}^{(n+1)}(x) = (n+1)!$ e che $p_n^{(n+1)}(x) = 0$, possiamo scrivere

$$0 = G^{(n+1)}(\xi) = f^{(n+1)}(\xi) - (n+1)! R_n(x) \quad \Longrightarrow \quad R_n(x) = \frac{f^{(n+1)}(\xi)}{(n+1)!},$$

e quindi la tesi, ricordando che $E_n(x) = R_n(x)\omega_{n+1}(x)$. □

Osservazione:

- La rappresentazione dell'errore $E_n(x)$ che abbiamo costruito riveste un'importanza soprattutto teorica. Infatti, il suo utilizzo richiede non solo la determinazione della derivata $f^{(n+1)}$, ma anche la valutazione di quest'ultima in un punto $\xi = \xi_1^{(n+1)}$ (dipendente da x) non noto.
- Quando la funzione $f(x)$ è un polinomio di grado non superiore a n, la derivata $f^{(n+1)}$ è identicamente nulla e dalla relazione precedente deduciamo che $f(x) \equiv p_n(x)$.
- Supponiamo che gli $n+1$ punti distinti appartengano all'intervallo $I = [\bar{x}, \bar{x}+h]$ e che si voglia approssimare in I una funzione $f(x) \in C^{n+1}(I)$ con un polinomio interpolante di grado n fissato. Denotato con

  $$M_{n+1} = \max_{x \in I} |f^{(n+1)}(x)|$$

  Supponiamo $b - a = h$ e sia $x \in (a,b)$. Dati i nodi $x_1, \ldots, x_{n+1} \in (a,b)$, si ha

  $$|x - x_i| \leq h.$$

  Allora

  $$|\omega_{n+1}(x)| = \prod_{i=1}^{n+1} |x - x_i| \leq h^{n+1}.$$

  Si ha

  $$\max_{x \in I} |f(x) - p_n(x)| \leq \frac{h^{n+1}}{(n+1)!} M_{n+1} \quad \Rightarrow \quad \|f - p_n\|_\infty = O(h^{n+1}), \qquad h \to 0.$$

  Quest'ultima relazione giustifica l'interesse per la costruzione di funzioni polinomiali a tratti.

Determiniamo ora un'altra espressione dell'errore di interpolazione a partire dalla rappresentazione di Newton del polinomio interpolante.

Teorema 3.1.13. Assegnati $n+1$ punti (x_i, y_i), con le ascisse distinte tra loro e tutte appartenenti ad $[a,b]$ e $y_i = f(x_i)$, per ogni $x \in [a,b]$ si ha

$$E_n(x) = f[x, x_1, \dots, x_{n+1}]\, \omega_{n+1}(x),$$

con

$$\omega_{n+1}(x) = (x - x_1) \cdot (x - x_2) \cdot \ldots \cdot (x - x_{n+1}).$$

Dimostrazione. Riprendiamo la definizione di differenza divisa e, fissato $x \neq x_i$, scriviamo

$$f[x, x_1] = \frac{f(x_1) - f(x)}{x_1 - x},$$

da cui deduciamo

$$f(x) = \underbrace{f(x_1)}_{p_0(x)} + f[x, x_1](x - x_1).$$

Il termine $p_0(x) = f(x_1)$ rappresenta il polinomio interpolante la funzione $f(x)$ nel nodo $x = x_1$, mentre $E_0(x) = (x - x_1) f[x, x_1]$ denota l'errore $f(x) - p_0(x)$.
Successivamente consideriamo

$$f[x, x_1, x_2] = \frac{f[x_1, x_2] - f[x, x_1]}{x_2 - x},$$

ovvero

$$f[x, x_1] = f[x_1, x_2] + f[x, x_1, x_2](x - x_2),$$

e sostituiamo quest'ultima nell'espressione precedentemente determinata per $f(x)$:

$$f(x) = f(x_1) + f[x_1, x_2](x - x_1) + \underbrace{f[x, x_1, x_2](x - x_1)(x - x_2)}_{E_1(x)}.$$

Il polinomio $p_1(x) = f(x_1) + f[x_1, x_2](x - x_1)$, di grado 1, è il polinomio interpolante la funzione $f(x)$ in x_1 e in x_2, mentre $E_1(x) = f[x, x_1, x_2](x - x_1)(x - x_2)$ denota l'errore di interpolazione $f(x) - p_1(x)$.
Così proseguendo, sino a includere tutti i nodi $x_1, \dots, x_{n+1}$, si perviene alla rappresentazione:

$$f(x) = p_n(x) + f[x, x_1, x_2, \dots, x_{n+1}](x - x_1)(x - x_2) \cdot \ldots \cdot (x - x_{n+1}),$$

da cui segue immediatamente la tesi. □

Osservazione: Notiamo che nel Teorema 3.1.13 non si è fatta alcuna ipotesi sulla regolarità di f a differenza del Teorema 3.1.12. Inoltre ricordando il Teorema 3.1.8 possiamo effettivamente dire che i coefficienti dei due teoremi coincidono quando f è sufficientemente regolare.

Sfortunatamente, a un aumento del grado n del polinomio interpolante non corrisponde necessariamente un miglioramento nell'approssimazione di una funzione f!
Enunciamo un risultato di non convergenza.

Teorema 3.1.14. Data una qualunque successione di nodi distinti, tutti situati in $[a, b]$,

$$\begin{array}{cccc} x_1^{(0)} & & & \\ x_1^{(1)} & x_2^{(1)} & & \\ x_1^{(2)} & x_2^{(2)} & x_3^{(2)} & \\ \cdots & \cdots & \cdots & \cdots \\ x_1^{(n)} & x_2^{(n)} & \cdots & x_{n+1}^{(n)} \\ \cdots & \cdots & \cdots & \cdots \end{array}$$

esiste sempre una funzione continua $f(x)$ in $[a, b]$ che, interpolata su quei nodi, genera una successione di polinomi di interpolazione $\{p_n(x)\}$ non uniformemente convergente a $f(x)$ in $[a, b]$.

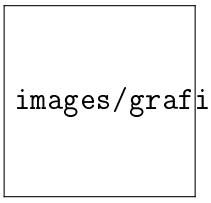
Esempio 3.1.15. Se si considerano i nodi equispaziati in $[a, b] = [-5, 5]$,

$$x_i = a + (i-1)h, \qquad i = 1, \ldots, n+1, \qquad h = \frac{b-a}{n},$$

come nodi di interpolazione, l'esempio che segue mostra che la successione dei polinomi interpolanti la funzione continua (infinitamente derivabile)

$$f(x) = \frac{1}{1+x^2},$$

detta **funzione di Runge**, non converge uniformemente a $f(x)$ in $[a, b]$, al tendere del numero dei nodi all'infinito.
Il limite della norma uniforme dell'errore di interpolazione è addirittura infinito!



Enunciamo un risultato di convergenza.

Teorema 3.1.16. Sia

$$\begin{array}{cccc} x_1^{(0)} & & & \\ x_1^{(1)} & x_2^{(1)} & & \\ x_1^{(2)} & x_2^{(2)} & x_3^{(2)} & \\ \cdots & \cdots & \cdots & \cdots \\ x_1^{(n)} & x_2^{(n)} & \cdots & x_{n+1}^{(n)} \\ \cdots & \cdots & \cdots & \cdots \end{array}$$

una successione qualsiasi di nodi distinti $x_i \in \mathbb{R}$, tutti in $[a, b]$, e sia $f(z)$ una funzione analitica in una regione D contenente $[a, b]$. Si denoti con $p_n(x)$ il polinomio di grado n interpolante f

nei nodi $x_1^{(n)}, x_2^{(n)}, \dots, x_{n+1}^{(n)}$.
Se la distanza delle singolarità di $f(z)$ da $[a,b]$ è maggiore di $b-a$, allora

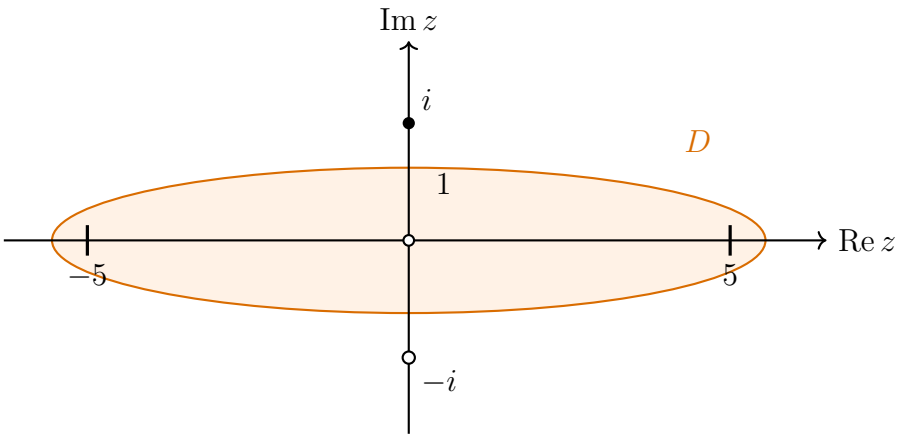
$$\lim_{n\to\infty} \|f - p_n\|_\infty = 0.$$

Osservazione: Si tratta di una condizione sufficiente.

Esempio 3.1.17. Consideriamo la funzione

$$f(x) = \frac{1}{1+x^2} \quad \longrightarrow \quad f(z) = \frac{1}{1+z^2},$$

che presenta singolarità in $\pm i$.
Nel caso $[a,b] = [-5,5]$, si ha $|b-a| = b-a = 10$ e $|i-(-i)| = 2$. Dalla contronominale del teorema precedente sapevamo già che doveva accadare che $|b-a| > |i-(-i)|$.



Esempio 3.1.18. Denotato con p_n il polinomio interpolante $f(x) = \frac{1}{1+x^2}$ nei nodi x_i, $i = 1, \dots, n+1$, equispaziati in $[1,2]$, in virtù del precedente teorema possiamo affermare che

$$\lim_{n\to\infty} \|E_n\|_\infty = 0.$$

images/grafici/runge_intervallo_1_2.pdf

In questo caso $b-a = 1$ mentre $|i-1| = |-i-1| = \sqrt{2}$

Denotiamo con $\mathbb{P}_n$ lo spazio lineare dei polinomi algebrici di grado minore o uguale a n. La costante di Lebesgue riveste un ruolo cruciale nella stima dell'errore di interpolazione. Per dimostrare quanto affermato, introduciamo la seguente definizione.

Definizione 3.1.19. Sia $f \in C^0([a,b])$. Si definisce **polinomio di migliore approssimazione uniforme algebrica** per la funzione f il polinomio q_n^* per il quale si ha

$$\|f - q_n^*\|_\infty = \min_{q_n \in \mathbb{P}_n} \|f - q_n\|_\infty.$$

La quantità

$$E_n^*(x) = f(x) - q_n^*(x)$$

è detta **errore di migliore approssimazione uniforme algebrica**.

Definizione 3.1.20. Una funzione di variabile reale $f : [a,b] \to \mathbb{R}$ soddisfa la condizione di Hölder di ordine μ, con $0 < \mu \leq 1$, se esiste una costante $C > 0$ tale che, per ogni $x, y \in [a,b]$, si ha $|f(x) - f(y)| \leq C|x - y|^\mu$. Il numero μ si dice esponente di Hölder, mentre f si dice Hölder-continua o hölderiana.

Osservazione: Se $\mu = 1$, allora f è lipschitziana.

Osservazione: Il polinomio q_n^* non è in generale un polinomio interpolante.

Si può dimostrare (teoremi di Jackson) che:

- Se $f \in C^0([a,b])$, allora
$$\lim_{n\to\infty} \|E_n^*\|_\infty = 0.$$
Non è noto l'ordine di convergenza rispetto a n.
- Se f è hölderiana di ordine $\mu > 0$ in $[a,b]$, allora
$$\|E_n^*\|_\infty = O\left(\frac{1}{n^\mu}\right) \qquad \text{per } n \to \infty.$$
- Se $f \in C^{k,\mu}([a,b])$ (ovvero $f^{(k)}$ è hölderiana di ordine μ), $k \geq 1$ e $0 \leq \mu \leq 1$, allora
$$\|E_n^*\|_\infty = O\left(\frac{1}{n^{k+\mu}}\right) \qquad \text{per } n \to \infty.$$

Ciò premesso, possiamo dare il seguente risultato di convergenza.

Teorema 3.1.21. Denotato con $p_n(x)$ il polinomio interpolante una funzione continua f nei nodi distinti $x_1, x_2, \ldots, x_{n+1}$ e con

$$E_n^*(x) = f(x) - q_n^*(x)$$

l'errore di migliore approssimazione uniforme algebrica, risulta

$$\|E_n^*\|_\infty \leq \underbrace{\|f - p_n\|_\infty}_{\|E_n\|_\infty} \leq (1 + \Lambda_n)\|E_n^*\|_\infty.$$

Dimostrazione. Per l'unicità del polinomio interpolante, essendo $q_n^* \in \mathbb{P}_n$, possiamo scrivere

$$q_n^*(x) = \sum_{j=1}^{n+1} \ell_j(x) q_n^*(x_j).$$

Pertanto, si ha

$$\begin{aligned} E_n(x) &= f(x) - p_n(x) \\ &= f(x) - q_n^*(x) + q_n^*(x) - p_n(x) \\ &= \underbrace{f(x) - q_n^*(x)}_{E_n^*(x)} + \sum_{j=1}^{n+1} \ell_j(x) \underbrace{\left(q_n^*(x_j) - f(x_j)\right)}_{-E_n^*(x_j)}. \end{aligned}$$

La tesi discende allora dalla seguente disuguaglianza, ottenuta applicando la disuguaglianza triangolare:

$$\begin{aligned} |f(x) - p_n(x)| &\leq |f(x) - q_n^*(x)| + \sum_{j=1}^{n+1} |\ell_j(x)| \, |q_n^*(x_j) - f(x_j)| \\ &\leq \|f - q_n^*\|_\infty + \|f - q_n^*\|_\infty \sum_{j=1}^{n+1} |\ell_j(x)| \\ &\leq \|f - q_n^*\|_\infty (1 + \Lambda_n), \end{aligned}$$

dove

$$\Lambda_n = \left\| \sum_{j=1}^{n+1} |\ell_j| \right\|_\infty.$$

□

Tenendo conto delle stime di convergenza per l'errore di migliore approssimazione uniforme algebrica e del comportamento della costante di Lebesgue associata ai nodi di Chebyshev e di Chebyshev-Lobatto, è immediato dimostrare il seguente teorema.

Teorema 3.1.22. Sia $\{p_n(x)\}$ la successione dei polinomi interpolanti $f(x)$ nei nodi $x_i \in [a, b]$ di Chebyshev oppure di Chebyshev-Lobatto, per i quali

$$\Lambda_n = O(\log n).$$

Se f è hölderiana di ordine $\mu > 0$ in $[a, b]$, allora

$$\|f - p_n\|_\infty = O\left(\frac{\log n}{n^\mu}\right), \qquad n \to \infty.$$

Se $f \in C^{k,\mu}([a, b])$, $k \geq 1$ e $0 \leq \mu \leq 1$, allora

$$\|f - p_n\|_\infty = O\left(\frac{\log n}{n^{k+\mu}}\right), \qquad n \to \infty.$$

Osservazione: Dal precedente teorema si deduce che la convergenza uniforme della successione dei polinomi interpolanti alla funzione interpolata f è tanto più rapida quanto più è regolare la funzione f.

Osservazione: Si precisa che la convergenza uniforme **non è garantita** se la funzione f è solo continua, mentre è garantita se la funzione f è almeno hölderiana.

Esempio 3.1.23. Denotato con p_n il polinomio interpolante $f(x) = \frac{1}{1+x^2}$ nei nodi di Chebyshev x_i, $i = 1, \dots, n+1$, $n = 9, 13$, relativi all'intervallo $[-5, 5]$, si ha

$$\lim_{n\to\infty} \|f - p_n\|_\infty = 0.$$

al contrario della stessa funzione sullo stesso intervallo ma con nodi equispaziati.



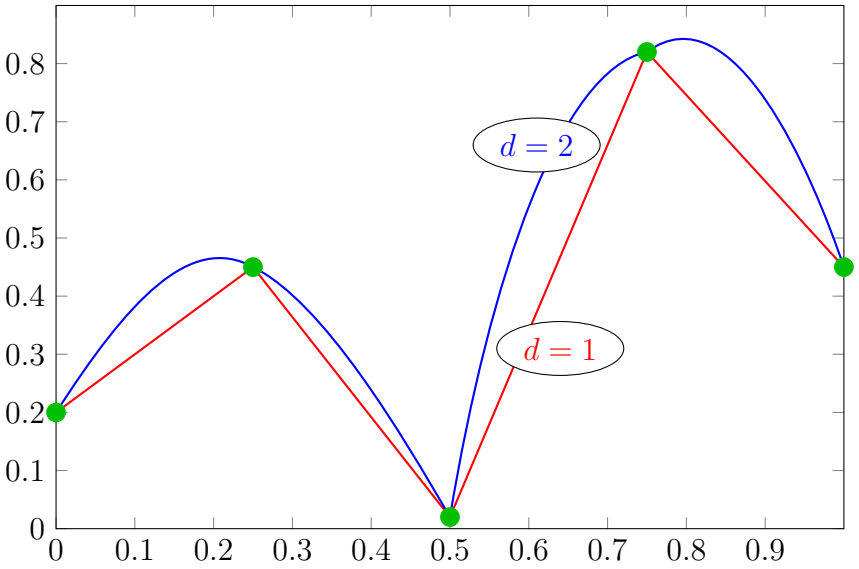
I precedenti risultati di convergenza mostrano dunque che:

- A un aumento del grado n del polinomio interpolante non corrisponde necessariamente un miglioramento nell'approssimazione di una funzione f;
- la convergenza uniforme del polinomio interpolante alla funzione interpolata dipende dalla scelta dei nodi e dalla regolarità della funzione f;
- la scelta dei nodi equispaziati non garantisce la convergenza uniforme, neppure per funzioni dotate di derivata continua di qualunque ordine;
- se f è hölderiana, la scelta dei nodi di Chebyshev oppure di Chebyshev-Lobatto, invece, garantisce la convergenza uniforme.

3.2 Interpolazione mediante funzioni polinomiali a tratti

Definizione 3.2.1. Si definisce **funzione polinomiale a tratti** di grado d, associata a una partizione dell'intervallo $[a, b]$, una funzione continua in $[a, b]$ e definita da un'unione di tratti contigui di polinomi algebrici diversi, ciascuno di grado d.

In generale, il grado d viene scelto piccolo ($d = 1, 2, 3$).



Si osservi che le funzioni polinomiali a tratti sono generalmente continue, ma non sono derivabili nei punti di raccordo.
Data una partizione $a \equiv x_1 < \ldots < x_{n+1} \equiv b$ di $[a, b]$ e, in ciascuno intervallo $[x_i, x_{i+1}]$, assegnate $d + 1$ ascisse

$$x_i \equiv x_{i,1} < \ldots < x_{i,d+1} \equiv x_{i+1},$$

supponiamo di conoscere le corrispondenti ordinate $y_{i,j} = f(x_{i,j})$, $j = 1, \ldots, d + 1$. A partire dai dati $(x_{i,j}, y_{i,j})$ è possibile costruire la funzione polinomiale a tratti

$$f_n(x) = p_{d,i}(x), \qquad x \in [x_i, x_{i+1}],$$

dove $[x_i, x_{i+1}]$ è l'intervallo i-esimo della partizione, costituita da n intervalli.
Si ha

$$p_{d,i}(x) = \sum_{j=1}^{d+1} \ell_j(x) f(x_{i,j}), \qquad p_{d,i}(x_{i,j}) = f(x_{i,j}), \qquad i = 1, \ldots, n, \quad j = 1, \ldots, d + 1,$$

dove i valori $f(x_{i,j})$ devono essere calcolabili.
Osserviamo che:

- se $x_{i,j} \in (x_i, x_{i+1})$, per ogni j, allora $f_n(x)$ è discontinua in x_i e in x_{i+1};
- se $x_{i,1} = x_i$ e $x_{i,d+1} = x_{i+1}$, per ogni i, allora $f_n(x) \in C([a, b])$.

Se $f(x) \in C^{d+1}([a, b])$, con $d \geq 1$, utilizzando un precedente risultato di convergenza dell'interpolazione polinomiale e denotando con

$$M_{d+1} = \|f^{(d+1)}\|_\infty, \qquad h_i = x_{i+1} - x_i, \qquad h = \max_{1 \leq i \leq n} h_i,$$

dove h_i rappresenta l'ampiezza dell'intervallo $[x_i, x_{i+1}]$ e h determina l'ordine di convergenza, si dimostra facilmente che

$$\|f - f_n\|_\infty = O(h^{d+1}), \qquad h \to 0.$$

Questo risultato è infatti una conseguenza immediata del fatto che

$$\begin{aligned} \|f - f_n\|_\infty &= \max_{1 \leq i \leq n} \|f - f_n\|_{\infty, [x_i, x_{i+1}]} \\ &= \max_{1 \leq i \leq n} \max_{x \in [x_i, x_{i+1}]} |f(x) - p_{d,i}(x)| \end{aligned}$$

e della seguente stima dell'errore nell'intervallo $[x_i, x_{i+1}]$:

$$|f(x) - p_{d,i}(x)| \leq \frac{M_{d+1}}{(d+1)!} h_i^{d+1} \leq \frac{M_{d+1}}{(d+1)!} h^{d+1}, \qquad \forall x \in [x_i, x_{i+1}],\ \forall i.$$

Osserviamo che, anche quando $f \in C^n([a,b])$ con $n > d+1$, si ha sempre

$$\|f - f_n\|_\infty = O(h^{d+1}), \qquad h \to 0.$$

perché la convergenza dipende dal grado dei polinomi interpolanti in ciascun intervallo.

3.2.1 Spline interpolanti

L'approccio precedente presenta un limite: per costruire i polinomi interpolanti nei singoli sottointervalli è necessario conoscere i valori della funzione in ulteriori punti, oltre ai nodi della partizione. Un'alternativa consiste nell'imporre condizioni sulle derivate, anziché vincolare esclusivamente i valori assunti dalla funzione in punti distinti.
Inoltre, è desiderabile che la rappresentazione polinomiale a tratti presenti una maggiore regolarità nei punti di raccordo. Come abbiamo visto, infatti, la continuità non garantisce necessariamente la derivabilità. Da queste esigenze nasce il concetto di **funzione spline**.
Le funzioni **spline** sono particolari funzioni polinomiali a tratti che soddisfano opportune proprietà di regolarità nei punti di raccordo. Se denotiamo con $S_d(x)$ la generica spline di grado locale d definita in $[a,b]$, allora $S_d(x) \in C^{d-1}([a,b])$. Di seguito la definizione di spline interpolante.

Definizione 3.2.2. Sia $a \equiv x_1 < x_2 < \ldots < x_{n+1} \equiv b$ una partizione dell'intervallo $[a,b]$ e $y_i = f(x_i)$. Si definisce **spline di ordine d interpolante** f nei nodi assegnati x_i una funzione $S_d(x)$ soddisfacente le seguenti condizioni:

i) $S_d(x)$ è un polinomio di grado (al più) d per $x \in [x_i, x_{i+1}]$, $i = 1, \ldots, n$;

ii) la derivata $S_d^{(k)}(x)$ di ordine k di $S_d(x)$ è una funzione continua in $[a,b]$ per $k = 0, 1, \ldots, d-1$;

iii) $S_d(x_i) = y_i$, $i = 1, \ldots, n+1$.

Osservazione: La condizione iii) impone soltanto l'interpolazione nei nodi e non la continuità. Le spline più utilizzate sono quelle di ordine $d = 1, 2, 3$. In particolare, per $d = 3$ si ha $S_3 \in C^2([a,b])$.

Osservazione: I nodi sono $x_1, \ldots, x_{n+1}$, con $x_1 = a$ e $x_{n+1} = b$, e individuano n intervalli. Poiché S_d è un polinomio di grado d in ogni intervallo, sono necessari $d+1$ coefficienti per ciascuno di essi. La spline S_d è quindi definita da $(d+1)n$ parametri.
Dalla condizione iii) si hanno $n+1$ equazioni, corrispondenti a $S_d(x_i) = y_i$.
La condizione ii) impone $S_d \in C^{d-1}([a,b])$, ossia la continuità delle derivate nei nodi interni $x_2, x_3, \ldots, x_n$, che sono $n-1$. Si impongono quindi

$$S_d^{(k)}(x_i^-) = S_d^{(k)}(x_i^+), \qquad \forall i = 2, \ldots, n, \quad \forall k = 0, \ldots, d-1.$$

ottendo $d(n-1)$ equazioni. Complessivamente, il numero di equazioni è

$$\begin{aligned} n + 1 + d(n-1) &= n + 1 + nd - d \\ &= n(d+1) - d + 1 \\ &= n(d+1) - (d-1). \end{aligned}$$

Ma le incognite sono $n(d+1)$. Pertanto, quando $d > 1$, mancano $d-1$ condizioni libere.

Osservazione: La derivata di una spline di ordine d è un'altra spline di ordine $d-1$, mentre la primitiva di una spline di ordine d è una spline di ordine $d+1$.

Osservazione: Per le spline $S_1(x)$ di ordine $d = 1$ non occorre imporre nessuna condizione aggiuntiva $(d-1=0)$.

Osservazione: Per le spline $S_2(x)$ di ordine $d = 2$ occorre imporre una sola condizione aggiuntiva $(d-1=1)$. Tipicamente si impone $S_2^{(1)}(a)$ oppure $S_2^{(1)}(b)$ uguale a un certo valore, oppure, nel caso di funzioni f periodiche con periodo $b-a$, si impone

$$S_2^{(1)}(a) = S_2^{(1)}(b).$$

Spline lineari

La spline $S_1(x)$ di ordine 1, interpolante f nei punti x_i, $i = 1, \dots, n+1$, ha la seguente espressione:

$$S_1(x) = f(x_i) + \frac{f(x_{i+1}) - f(x_i)}{x_{i+1} - x_i}(x - x_i), \qquad x \in [x_i, x_{i+1}].$$

Per calcolare e valutare la spline $S_1(x)$ si può utilizzare il seguente comando MATLAB:

```
s = interp1(x,y,z)
```

Il comando calcola e memorizza in `s` i valori che la spline lineare interpolante i dati (x_i, y_i), $i = 1, \dots, n+1$, assume in `z`; i valori x_i e y_i sono memorizzati in `x` e in `y`, rispettivamente.
Per le spline lineari vale il seguente teorema di convergenza.

Teorema 3.2.3. Data una partizione $a = x_1 < x_2 < \dots < x_{n+1} = b$, sia S_1 la spline interpolante una funzione $f \in C^2([a,b])$ nei nodi della partizione x_i, $i = 1, \dots, n+1$. Denotato con

$$h = \max_{1 \le i \le n}(x_{i+1} - x_i),$$

allora

$$\|f - S_1\|_\infty = O(h^2), \qquad h \to 0.$$

Osservazione: La convergenza uniforme è comunque garantita qualunque sia la funzione continua f e qualunque sia la scelta dei nodi x_i, per $h \to 0$.
In particolare, si ha:

$$f \in C^0([a,b]) \implies \|f - S_1\|_\infty \xrightarrow{h \to 0} 0,$$
$$f \in C^1([a,b]) \implies \|f - S_1\|_\infty = O(h).$$

Per $f \in C^k([a,b])$, con $k > 2$, si ha

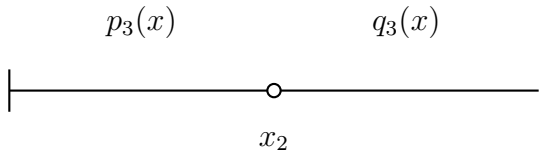
$$\|f - S_1\|_\infty = O(h^2).$$

Spline cubiche

Per l'unicità di una spline cubica è necessario imporre due ulteriori condizioni aggiuntive ($d-1=2$). Fra tutte le possibili scelte, tipiche condizioni aggiuntive sono:

1) $S_3^{(2)}(x_1)=0$, $S_3^{(2)}(x_{n+1})=0$: **spline naturale**;
2) $S_3^{(2)}(x_1)=f^{(2)}(x_1)$, $S_3^{(2)}(x_{n+1})=f^{(2)}(x_{n+1})$;
3) $S_3^{(3)}(x_2^-)=S_3^{(3)}(x_2^+)$, $S_3^{(3)}(x_n^-)=S_3^{(3)}(x_n^+)$: **spline "not-a-knot"**;
4) $S_3^{(1)}(x_1)=f^{(1)}(x_1)$, $S_3^{(1)}(x_{n+1})=f^{(1)}(x_{n+1})$: **spline vincolata**;
5) $S_3^{(1)}(a)=S_3^{(1)}(b)$, $S_3^{(2)}(a)=S_3^{(2)}(b)$: **spline periodica** (tali condizioni si impongono nel caso di fenomeni periodici di periodo $b-a$, per cui le condizioni di interpolazione implicano $S_3(a)=S_3(b)$).

Imponendo la condizione di *not-a-knot* i due tratti (contigui) di polinomi (di terzo grado) che definiscono la spline cubica nell'intervallo (x_1,x_3) e (x_{n-1},x_{n+1}) appartengono a uno stesso polinomio di terzo grado. In questo caso, i nodi x_2 e x_n sono nodi di interpolazione ma non più di partizione, nel senso che non separano più tratti di polinomi diversi.
Infatti siano $p_3(x)$ e $q_3(x)$ i due polinomi che definiscono la spline nei tratti contigui separati dal nodo x_2.



Si ha

$$p_3 = S_3\big|_{[x_1,x_2]}, \qquad q_3 = S_3\big|_{[x_2,x_3]}.$$

Dalla definizione di spline, si ha

$$p_3^{(k)}(x_2) = q_3^{(k)}(x_2), \qquad k=0,1,2.$$

Aggiungendo la condizione

$$p_3^{(3)}(x_2) = q_3^{(3)}(x_2)$$

e osservando che

$$p_3^{(4)}(x_2) = 0 = p_3^{(4)}(x_2),$$

si ottiene

$$p_3^{(k)}(x_2) = q_3^{(k)}(x_2), \qquad \forall k,$$

ossia

$$(p_3^{(k)} - q_3^{(k)})(x_2) = 0, \qquad \forall k.$$

Consideriamo ora le espressioni dei due polinomi:

$$p_3(x) = a_1x^3 + a_2x^2 + a_3x + a_4,$$
$$q_3(x) = b_1x^3 + b_2x^2 + b_3x + b_4.$$

Dall'uguaglianza delle derivate terze si ha

$$q_3^{(3)} = 6b_1 = p_3^{(3)} = 6a_1 \quad \Longrightarrow \quad a_1 = b_1.$$

Dall'uguaglianza delle derivate seconde segue

$$q_3^{(2)}(x_2) = 6a_1x_2 + 2b_2 = 6a_1x_2 + 2a_2 \quad \Longrightarrow \quad a_2 = b_2.$$

Proseguendo analogamente per i coefficienti rimanenti, si conclude che

$$p_3 = q_3.$$

Per il caso di x_n si procede in modo analogo.
Imponendo le condizioni 1), oppure 2), oppure 3), oppure 4), oppure 5), la spline corrispondente esiste ed è unica. Per determinare una spline cubica non si ricorre alla risoluzione di un sistema lineare nelle $4n$ incognite $\{c_{i,1}, c_{i,2}, c_{i,3}, c_{i,4}\}$, ma alla risoluzione di un sistema lineare nelle $n+1$ incognite

$$M_i = S_3^{(2)}(x_i), \qquad i = 1, \ldots, n+1,$$

che gode di proprietà particolari.
In particolare, se si impone una delle condizioni aggiuntive 1)–4), il sistema lineare nelle $n+1$ (al massimo) incognite M_i risulta simmetrico, a diagonale dominante e tridiagonale, per la cui risoluzione è possibile utilizzare l'algoritmo di Thomas.
Per la scelta 5), il sistema lineare nelle $n+1$ incognite M_i non è più esattamente tridiagonale, ma è ancora simmetrico e a diagonale dominante.
Per quanto riguarda la convergenza uniforme delle spline cubiche interpolanti alla funzione interpolata, si può dimostrare il seguente teorema.

Teorema 3.2.4. Sia $S_3(x)$ la spline cubica interpolante i dati $(x_i, f(x_i))$, con $a \le x_i \le b$, e soddisfacente una delle condizioni aggiuntive 1)–5).
Denotato con

$$h_i = x_{i+1} - x_i, \qquad h = \max_{1 \le i \le n} h_i,$$

se $f \in C^4([a,b])$ e se

$$\frac{h}{h_i} \le \gamma < \infty \qquad \text{per } h \to 0,$$

si ha

$$\|f^{(p)} - S_3^{(p)}\|_\infty = O(h^{4-p}), \qquad h \to 0, \quad p = 0, 1, 2, 3.$$

Osservazione: La condizione sulla partizione richiede che tutti gli intervalli si riducano proporzionalmente. In particolare, si considera

$$\lim_{h \to 0} \frac{\max h_i}{h_i} = C \Rightarrow h_i = O\left(\max h_i\right) \quad \forall i.$$

Osservazione: La convergenza uniforme delle spline cubiche interpolanti, a differenza dei polinomi interpolanti, è comunque garantita qualunque sia la funzione continua f e qualunque sia la scelta dei nodi x_i per $h \to 0$.
Il precedente teorema assicura l'approssimazione uniforme in $[a,b]$ anche delle stesse derivate di $f(x)$ (**simultanea approssimazione**).

Per calcolare e valutare le spline "not-a-knot" o le spline vincolate si possono utilizzare i seguenti comandi MATLAB.
Il comando

```
s = spline(x,y,z)
```

calcola e memorizza in `s` i valori che la spline cubica interpolante i dati (x_i, y_i), $i = 1, \ldots, n+1$, e soddisfacente la condizione 3) "not-a-knot", assume in `z`; i valori x_i e y_i sono memorizzati in `x` e in `y`, rispettivamente.
Il comando

```
s = spline(x,[yd1 y ydn],z)
```

calcola e memorizza in `s` i valori che la spline cubica interpolante i dati (x_i, y_i), $i = 1, \ldots, n+1$, e soddisfacente la condizione 4), con `yd1` $= f^{(1)}(x_1)$ e `ydn` $= f^{(1)}(x_{n+1})$, assume in `z`; i valori x_i e y_i sono memorizzati in `x` e in `y`, rispettivamente.

Esempio 3.2.5. In figura sono rappresentati graficamente il polinomio e la spline cubica "not-a-knot" interpolanti la funzione di Runge in 13 nodi di Chebyshev definiti nell'intervallo $[-5, 5]$.

